

**BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ**



ĐỒ ÁN TỐT NGHIỆP
**NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG THÔNG TIN
HỖ TRỢ QUẢN LÝ VĂN BẰNG CHỨNG CHỈ
DỰA TRÊN NỀN TẢNG BLOCKCHAIN**

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:
Phạm Đức Khải – MSSV: AT190226

Người hướng dẫn:
TS. Trần Trung
Khoa CNTT – Trường Đại học Điện lực

Hà Nội, 2026

**BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ**



ĐỒ ÁN TỐT NGHIỆP
NGHIÊN CỨU PHÁT TRIỂN HỆ THỐNG THÔNG TIN
HỖ TRỢ QUẢN LÝ VĂN BẰNG CHỨNG CHỈ
DỰA TRÊN NỀN TẢNG BLOCKCHAIN

Ngành: An toàn thông tin
Mã số: 7.48.02.02

Sinh viên thực hiện:
Phạm Đức Khải – MSSV: AT190226

Người hướng dẫn:
TS. Trần Trung
Khoa CNTT – Trường Đại học Điện lực

Hà Nội, 2026

MỤC LỤC

LỜI CẢM ƠN.....	ix
LỜI NÓI ĐẦU.....	x
CHƯƠNG I. CƠ SỞ LÝ THUYẾT BLOCKCHAIN.....	1
1.1. Tổng quan về công nghệ Blockchain.....	1
1.1.1. Khái niệm và mục tiêu hình thành.....	1
1.1.2. Quá trình phát triển.....	1
1.1.3. Đặc trưng, giới hạn và những cách hiểu cần tránh.....	2
1.1.4. Phân biệt với DLT và cơ sở dữ liệu phân tán.....	3
1.2. Cấu trúc dữ liệu và nền tảng mật mã.....	3
1.2.1. Giao dịch, khối và liên kết băm.....	3
1.2.2. Hàm băm mật mã và cây Merkle.....	4
1.2.3. Mật mã khóa công khai và chữ ký số.....	5
1.2.4. Định danh, dấu thời gian và nonce.....	6
1.3. Kiến trúc và nguyên lý hoạt động.....	6
1.3.1. Mạng ngang hàng và vai trò của nút.....	6
1.3.2. Vòng đời giao dịch.....	7
1.3.3. Sao chép trạng thái và thực thi xác định.....	7
1.3.4. Hợp đồng thông minh và dữ liệu ngoài hệ thống.....	8
1.4. Cơ chế đồng thuận.....	8
1.4.1. Bài toán đồng thuận và mô hình lỗi.....	8
1.4.2. Proof of Work và Proof of Stake.....	9
1.4.3. Đồng thuận CFT và BFT trong mạng định danh.....	9
1.4.4. Tính hoàn tất và đánh đổi thiết kế.....	10
1.5. Phân loại Blockchain và điều kiện lựa chọn.....	11
1.5.1. Các trục phân loại.....	11
1.5.2. So sánh các mô hình triển khai.....	11
1.5.3. Khi nên và không nên sử dụng Blockchain.....	12
1.6. An toàn, quyền riêng tư và khả năng mở rộng.....	13
1.6.1. Mô hình đe dọa và bảo vệ nhiều lớp.....	13
1.6.2. Quản lý khóa, hợp đồng thông minh và nguồn dữ liệu ngoài.....	13
1.6.3. Quyền riêng tư và dữ liệu cá nhân.....	14
1.6.4. Khả năng mở rộng và khả năng tương tác.....	14
1.7. Định hướng ứng dụng trong quản lý văn bản, chứng chỉ.....	15
1.7.1. Bài toán, tác nhân và yêu cầu kiểm chứng.....	15
1.7.2. Mô hình dữ liệu khái niệm trên chuỗi và ngoài chuỗi.....	15
1.7.3. Giá trị kỳ vọng, giới hạn và ranh giới trách nhiệm.....	16

1.7.4. Mô hình Blockcerts V3.....	16
1.7.5. Bitcoin OP_RETURN và môi trường regtest.....	17
1.7.6. So sánh các hướng quản lý văn bằng số.....	17
1.8. Tổng kết Chương I.....	18
1.8.1. Kết quả lý luận chính.....	18
1.8.2. Cơ sở chuyển sang Chương II.....	19
CHƯƠNG II. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	20
2.1. Phân tích bài toán.....	20
2.1.1. Bối cảnh nghiệp vụ.....	20
2.1.2. Phạm vi.....	20
2.2. Tác nhân và yêu cầu chức năng.....	20
2.2.1. Tác nhân.....	20
2.2.2. Yêu cầu chức năng.....	21
2.2.3. Quy tắc nghiệp vụ.....	22
2.3. Yêu cầu phi chức năng và mô hình đe dọa.....	23
2.3.1. Yêu cầu phi chức năng.....	23
2.3.2. Tài sản và đối thủ.....	23
2.4. Lựa chọn công nghệ và kiến trúc.....	24
2.4.1. Lựa chọn mô hình Blockcerts–Bitcoin.....	24
2.4.2. Kiến trúc thành phần.....	24
2.4.3. Luồng dữ liệu.....	25
2.5. Thiết kế Maker–Checker và máy trạng thái.....	26
2.5.1. Luồng phát hành.....	26
2.5.2. Trạng thái.....	26
2.6. Thiết kế phát hành Blockcerts V3.....	27
2.6.1. Cấu trúc chứng thư.....	27
2.6.2. Batching và ánh xạ kết quả.....	28
2.6.3. Tách nhóm yêu cầu và lô phát hành.....	28
2.7. Thiết kế xác minh và thu hồi.....	29
2.7.1. Xác minh.....	29
2.7.2. Thu hồi.....	30
2.7.3. Quy tắc hợp nhất kết quả xác minh.....	31
2.8. Thiết kế dữ liệu ngoài chuỗi.....	33
2.8.1. Bảng users.....	33
2.8.2. Bảng issued_certificates.....	33
2.8.3. verification_logs và audit_logs.....	34
2.9. Thiết kế API và phân quyền.....	34

2.10. Thiết kế ba cổng giao diện.....	35
2.11. Thiết kế an toàn và triển khai.....	36
2.11.1. Ranh giới tin cậy.....	36
2.11.2. Quản lý bí mật và dữ liệu nhạy cảm.....	37
2.11.3. Kiểm soát SSRF trong dịch vụ xác minh.....	37
2.11.4. Bảo vệ trạng thái và chống cạnh tranh.....	38
2.12. Tiêu chí chấp nhận và kế hoạch kiểm thử.....	38
2.12.1. Ma trận truy vết.....	39
2.12.2. Nguyên tắc diễn giải bằng chứng.....	40
2.13. Kết luận Chương 2.....	40
CHƯƠNG III. XÂY DỰNG, KIỂM THỬ VÀ ĐÁNH GIÁ.....	41
3.1. Môi trường và công nghệ triển khai.....	41
3.1.1. Môi trường thực nghiệm.....	41
3.1.2. Tổ chức triển khai.....	42
3.2. Hiện thực backend và cơ sở dữ liệu.....	43
3.2.1. Các module.....	43
3.2.2. Migration và transaction.....	43
3.2.3. Audit và verification log.....	43
3.3. Hiện thực quy trình phát hành.....	44
3.3.1. Phát hành đơn và Maker–Checker.....	44
3.3.2. Tối ưu phát hành theo lô.....	44
3.3.3. Cấu trúc chứng thư đầu ra.....	45
3.4. Hiện thực xác minh và thu hồi.....	45
3.4.1. Xác minh công khai.....	45
3.4.2. Thu hồi.....	46
3.5. Hiện thực ba frontend.....	46
3.6. Hardening B12.....	47
3.7. Phương pháp kiểm thử.....	47
3.7.1. Mục tiêu và nguyên tắc.....	47
3.7.2. Kiểm thử tự động và chất lượng mã.....	48
3.8. Quy trình triển khai và tái lập chi tiết.....	48
3.8.1. B1 – Kiểm tra điều kiện tiên quyết.....	49
3.8.2. B2 – Cố định mốc mã nguồn.....	50
3.8.3. B3 – Chuẩn bị cấu hình an toàn.....	50
3.8.4. B4 – Dựng hạ tầng phụ thuộc.....	51
3.8.5. B5 – Migration và build.....	52
3.8.6. B6 – Khởi tạo tài khoản và kiểm tra RBAC.....	52

3.8.7. B7 – Tạo nhóm yêu cầu phát hành.....	53
3.8.8. B8 – Duyệt từng phần và phát hành.....	53
3.8.9. B9 – Kiểm tra cổng Holder.....	54
3.8.10. B10 – Xác minh công khai.....	54
3.8.11. B11 – Thu hồi và kiểm tra sau thu hồi.....	54
3.8.12. B12 – Kiểm thử chức năng, an toàn và phục hồi.....	55
3.8.13. B13 – Benchmark và đóng băng evidence.....	55
3.9. Kết quả Kịch bản 1 – tính toán vẹn và Maker–Checker.....	56
3.10. Kết quả Kịch bản 2 – hiệu năng Merkle batching.....	58
3.10.1. Kết quả từng lần.....	58
3.10.2. Giá trị tổng hợp.....	59
3.10.3. Tài nguyên.....	61
3.10.4. Kiểm toán artifact.....	61
3.11. Kết quả Kịch bản 3 – thu hồi, RBAC và audit.....	62
3.12. Đánh giá kết quả.....	64
3.12.1. Mức đáp ứng.....	64
3.12.2. Hạn chế.....	64
3.12.3. Hướng phát triển.....	65
3.13. Kết luận Chương 3.....	65
KẾT LUẬN VÀ KIẾN NGHỊ.....	66
1. Kết quả của đề án.....	66
2. Đóng góp chính.....	66
3. Kiến nghị.....	66
TÀI LIỆU THAM KHẢO.....	67
PHỤ LỤC.....	69
Phụ lục 1. Danh mục API và quyền truy cập.....	69
Phụ lục 2. Cấu hình triển khai tái lập.....	71
2.1. Nguyên tắc cấu hình.....	71
2.2. Khung biến môi trường.....	71
2.3. Các service triển khai.....	72
Phụ lục 3. Quy trình vận hành và xử lý lỗi.....	72
3.1. Trình tự chuẩn.....	72
3.2. Backend không kết nối PostgreSQL.....	73
3.3. Redis hoặc BullMQ không hoạt động.....	73
3.4. Bitcoin RPC không phản hồi.....	73
3.5. Migration chưa đủ.....	73
3.6. Phát hành dừng trước broadcast.....	73

3.7. Phát hành dừng sau broadcast.....	74
3.8. Verifier trả INDETERMINATE.....	74
3.9. Checksum không khớp.....	74
Phụ lục 4. Bộ bằng chứng thực nghiệm.....	74
Phụ lục 5. Ma trận truy vết yêu cầu và bằng chứng.....	76
Phụ lục 6. Danh mục hình và tệp nguồn.....	77

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

- API**— Giao diện lập trình ứng dụng
- BFT**— Khả năng chịu lỗi Byzantine
- CFT**— Khả năng chịu lỗi dừng
- DLT**— Công nghệ sổ cái phân tán
- E2E**— Kiểm thử đầu cuối
- HTTP**— Giao thức truyền siêu văn bản
- JWT**— Mã thông báo Web JSON
- MFA**— Xác thực đa yếu tố
- OP_RETURN**— Trường dữ liệu trong giao dịch Bitcoin
- PoS**— Cơ chế đồng thuận bằng cổ phần
- PoW**— Cơ chế đồng thuận bằng công việc
- RBAC**— Kiểm soát truy cập dựa trên vai trò
- RPC**— Cơ chế gọi thủ tục từ xa
- SSRF**— Giả mạo yêu cầu phía máy chủ
- TLS**— Bảo mật tầng truyền tải
- UUID**— Định danh duy nhất toàn cục
- WIF**— Định dạng nhập khóa ví

DANH MỤC HÌNH VẼ

Hình 1.1. Cấu trúc khối và liên kết mật mã giữa các khối liên tiếp.....	4
Hình 1.2. Cây Merkle và đường dẫn kiểm chứng một giao dịch.....	5
Hình 1.3. Vòng đời giao dịch Blockchain ở mức khái quát.....	7
Hình 1.4. So sánh khái quát các nhóm cơ chế đồng thuận.....	10
Hình 1.5. Phân loại Blockchain theo phạm vi tham gia và quyền truy cập.....	11
Hình 1.6. Các lớp bảo vệ trong một hệ thống Blockchain.....	13
Hình 1.7. Mô hình khái niệm ứng dụng Blockchain trong quản lý văn bản.....	15
Hình 2.1. Tác nhân và nhóm chức năng của hệ thống.....	21
Hình 2.2. Kiến trúc thành phần của hệ thống.....	25
Hình 2.3. Trình tự phát hành chứng thư theo lô.....	26
Hình 2.4. Máy trạng thái của yêu cầu phát hành.....	27
Hình 2.5. Quan hệ giữa nhóm yêu cầu và lần phát hành.....	29
Hình 2.6. Trình tự xác minh chứng thư công khai.....	30
Hình 2.7. Quy tắc phân loại kết quả xác minh.....	31
Hình 2.8. Mô hình dữ liệu của hệ thống.....	33
Hình 2.9. Ranh giới tin cậy và kiểm soát an toàn.....	37
Hình 3.1. Kiến trúc triển khai môi trường thực nghiệm.....	42
Hình 3.2. Luồng phát hành một văn bản.....	44
Hình 3.3. Luồng phát hành chứng thư theo lô.....	45
Hình 3.4. Luồng thu hồi và xác minh chứng thư.....	46
Hình 3.5. Các cổng kiểm soát quy trình tái lập.....	49
Hình 3.6. Chuỗi bằng chứng từ mã nguồn đến kết luận.....	56
Hình 3.7. Tiến trình thực nghiệm Chương 3.....	58
Hình 3.8. Thời gian phát hành trung bình theo kích thước batch.....	60
Hình 3.9. Thông lượng và thời gian trung bình trên mỗi chứng thư.....	60
Hình 3.10. Mức sử dụng tài nguyên theo kích thước batch.....	61
Hình 3.11. Tổng hợp tỷ lệ đạt của ba kịch bản kiểm thử.....	63

DANH MỤC BẢNG

Bảng 1.1. So sánh khái quát các mô hình triển khai Blockchain.....	11
Bảng 1.2. So sánh các hướng quản lý và xác minh văn bằng.....	18
Bảng 2.1. Danh sách yêu cầu chức năng.....	21
Bảng 2.2. Đe dọa và kiểm soát.....	23
Bảng 2.3. Ý nghĩa trạng thái.....	27
Bảng 2.4. Ma trận hợp nhất kết quả xác minh.....	32
Bảng 2.5. Mô hình dữ liệu chính.....	34
Bảng 2.6. API chính và quyền.....	34
Bảng 2.7. Tiêu chí chấp nhận.....	38
Bảng 2.8. Ma trận truy vết thiết kế và kiểm thử.....	39
Bảng 3.1. Thành phần phần mềm.....	41
Bảng 3.2. Điều kiện chấp nhận B1.....	49
Bảng 3.3. Nhóm biến cấu hình runtime.....	51
Bảng 3.4. Sản phẩm build cần đạt.....	52
Bảng 3.5. Kiểm tra đầu ra B7.....	53
Bảng 3.6. Checkpoint của worker phát hành.....	54
Bảng 3.7. Kết quả kiểm soát cuối B13.....	56
Bảng 3.8. Kết quả Kịch bản 1.....	57
Bảng 3.9. Chín lần chạy hiệu năng tái kiểm chứng.....	58
Bảng 3.10. Tổng hợp theo kích thước batch.....	59
Bảng 3.11. Tài nguyên hệ thống.....	61
Bảng 3.12. Kết quả kiểm toán.....	62
Bảng 3.13. Kết quả Kịch bản 3.....	63
Bảng 3.14. Đối chiếu yêu cầu và bằng chứng.....	64
Bảng PL1.1. API xác thực và tài khoản.....	69
Bảng PL1.2. API phát hành, Holder và audit.....	69
Bảng PL1.3. API xác minh, thu hồi và tài nguyên công khai.....	70
Bảng PL2.1. Phân loại cấu hình.....	71
Bảng PL2.2. Vai trò service.....	72
Bảng PL4.1. Thành phần evidence chính.....	74
Bảng PL4.2. Số liệu chốt được phép sử dụng.....	75
Bảng PL5.1. Truy vết yêu cầu chức năng.....	76
Bảng PL5.2. Truy vết yêu cầu an toàn và phi chức năng.....	77
Bảng PL6.1. Nhóm tài sản hình ảnh.....	78

LỜI CẢM ƠN

Trong quá trình thực hiện đồ án tốt nghiệp này, em đã nhận được sự giúp đỡ tận tình của cán bộ hướng dẫn TS. Trần Trung – Giảng viên Khoa CNTT Trường Đại học Điện lực; cán bộ quản lý ThS. Nguyễn Đức Ngân – Giảng viên Khoa ATTT Học viện Kỹ thuật Mật Mã, cùng với sự quan tâm sâu sát của Hệ quản lý sinh viên, sự động viên của người thân và bạn bè.

Em xin trân trọng cảm ơn.

LỜI NÓI ĐẦU

Quá trình chuyển đổi số trong giáo dục làm tăng số lượng hồ sơ và văn bằng được tạo, trao đổi và kiểm tra trên môi trường mạng. Cách quản lý tập trung giúp cơ sở đào tạo kiểm soát nghiệp vụ, nhưng bên thứ ba thường phải truy cập công tra cứu của đơn vị cấp hoặc liên hệ thủ công để xác minh. Khi cổng dịch vụ gián đoạn, dữ liệu bị sửa trái phép hoặc bản sao văn bằng được tạo ngoài hệ thống, việc đánh giá tính xác thực trở nên khó khăn. Vì vậy, bài toán không chỉ là số hóa tài liệu mà còn phải bảo đảm tính toàn vẹn, nguồn gốc, trạng thái hiệu lực và khả năng truy vết.

Các mô hình chứng thư có thể kiểm chứng kết hợp chữ ký số, định danh đơn vị cấp và cơ chế trạng thái đã được nghiên cứu rộng rãi. Blockcerts bổ sung cách neo bằng chứng Merkle lên blockchain, cho phép nhiều chứng thư dùng chung một giao dịch và giúp người nhận tự lưu giữ tài liệu. Hướng tiếp cận này không loại bỏ hệ thống nghiệp vụ của cơ sở đào tạo; dữ liệu cá nhân và quy trình phê duyệt vẫn được quản lý ngoài chuỗi, còn blockchain chỉ lưu bằng chứng tối thiểu phục vụ kiểm tra độc lập.

Đề tài “Nghiên cứu phát triển hệ thống thông tin hỗ trợ quản lý văn bằng chứng chỉ dựa trên nền tảng Blockchain” đặt mục tiêu xây dựng một nguyên mẫu có thể phát hành, quản lý, xác minh và thu hồi văn bằng số. Hệ thống áp dụng quy trình Maker–Checker để tách quyền lập và duyệt, RBAC để kiểm soát API, Merkle batching để phát hành nhiều chứng thư trong một transaction, công Holder để sinh viên nhận chứng thư và cổng Verify công khai để kiểm tra tính toàn vẹn cùng trạng thái thu hồi.

Phạm vi thực nghiệm sử dụng Blockcerts V3, Bitcoin Core ở chế độ regtest, backend NestJS, PostgreSQL, Redis và BullMQ 6. Ba cổng giao diện được tách theo nhóm người dùng gồm Admin cho Maker và Checker, Client cho Student, Verify cho người xác minh công khai. Việc đánh giá tập trung vào tính đúng của quy trình, khả năng phục hồi sau lỗi, kiểm soát an toàn và hiệu quả phát hành theo lô. Kết quả regtest không được suy rộng thành tuyên bố sẵn sàng production hoặc hiệu năng của Bitcoin mainnet.

Báo cáo gồm ba chương. Chương I trình bày blockchain, mật mã, Blockcerts và văn bằng số. Chương II phân tích yêu cầu, kiến trúc, dữ liệu, phân quyền, xác minh và thu hồi. Chương III mô tả hiện thực, tái lập, kiểm thử, benchmark và kiểm toán 1.830 chứng thư. Sáu phụ lục cung cấp API, cấu hình, hướng dẫn xử lý lỗi, evidence và truy vết.

CHƯƠNG I. CƠ SỞ LÝ THUYẾT BLOCKCHAIN

1.1. Tổng quan về công nghệ Blockchain

1.1.1. Khái niệm và mục tiêu hình thành

Công nghệ chuỗi khối (Blockchain) là một dạng kiến trúc sổ cái phân tán (Distributed Ledger Technology - DLT), trong đó dữ liệu giao dịch được đóng gói thành các khối (blocks), liên kết tuần tự theo thời gian bằng các hàm băm mật mã và được đồng bộ giữa các nút mạng thông qua giao thức đồng thuận [1], [2]. Thay vì phụ thuộc vào một máy chủ trung tâm, sổ cái Blockchain được sao chép và duy trì độc lập tại nhiều nút tham gia. Khi dữ liệu quá khứ bị thay đổi, các liên kết băm phía sau không còn khớp; sự sai lệch vì thế có thể được phát hiện khi kiểm tra lại lịch sử. Kết hợp với sao chép sổ cái và đồng thuận, cơ chế này làm cho việc sửa đổi hồi tố mà không để lại dấu vết trở nên khó khăn trong các giả định an toàn của hệ thống [1], [10].

Mục tiêu cốt lõi của Blockchain là giúp các thực thể thuộc nhiều miền quản trị thống nhất một lịch sử trạng thái dùng chung và giảm mức phụ thuộc vào một bên trung gian duy nhất [1]. Trong các quy trình nghiệp vụ truyền thống, việc đối soát liên tổ chức qua trung gian thường làm phát sinh chi phí, độ trễ và nguy cơ xuất hiện điểm lỗi đơn lẻ (Single Point of Failure). Blockchain giải quyết bài toán này nhờ kết hợp mật mã khóa công khai, cây Merkle, mạng ngang hàng và thuật toán đồng thuận để tạo ra nhật ký kiểm toán minh bạch, cho phép các bên tự kiểm chứng tính toàn vẹn dữ liệu [2], [10].

Tuy nhiên, Blockchain không nhằm thay thế các hệ thống lưu trữ dung lượng lớn hay cơ sở dữ liệu quan hệ truyền thống. Mọi hệ thống Blockchain trong thực tế đều đòi hỏi một khung quản trị xác định, bao gồm quy định về quyền truy cập, chính sách quản lý khóa mật mã, quy tắc phê duyệt giao dịch và cơ chế nâng cấp giao thức [1]. Công nghệ này phù hợp hơn với bài toán cộng tác đa bên, trong đó các thực thể cần chia sẻ trạng thái có thể kiểm chứng nhưng không muốn một bên duy nhất toàn quyền quyết định lịch sử dữ liệu [10].

1.1.2. Quá trình phát triển

Nền tảng lý thuyết của Blockchain là kết quả kế thừa từ nhiều thập kỷ nghiên cứu về mật mã học, cấu trúc dữ liệu và hệ thống phân tán. Từ cuối những năm 1970 và đầu thập niên 1980, các công trình về hệ mật mã khóa công khai, cấu trúc cây băm Merkle [3] và các giải pháp đóng dấu thời gian số (digital timestamping) đã đặt nền móng cho việc xác thực tính toàn vẹn dữ liệu. Song song

với đó, lý thuyết hệ thống phân tán phát triển các mô hình đồng thuận chịu lỗi dừng (CFT) và chịu lỗi tùy ý Byzantine (BFT), cung cấp cơ sở toán học để duy trì tính nhất quán trạng thái giữa các nút độc lập khi xảy ra sự cố mạng hoặc có nút sai hỏng [6], [7].

Năm 2008, Satoshi Nakamoto kết hợp chuỗi liên kết băm, mạng ngang hàng và cơ chế Bằng chứng công việc (Proof of Work) để giới thiệu Bitcoin, qua đó đề xuất một cơ chế hạn chế chi tiêu kép mà không cần máy chủ thanh toán trung tâm [1], [2]. Các nền tảng về sau mở rộng khả năng lập trình trạng thái thông qua hợp đồng thông minh (smart contract), cho phép biểu diễn và tự động thực thi một số quy tắc nghiệp vụ [1], [10]. Đồng thời, các kiến trúc Blockchain cấp quyền (permissioned) ra đời, ứng dụng các thuật toán CFT và BFT để tối ưu hóa thông lượng và đáp ứng yêu cầu bảo mật, quyền riêng tư nghiêm ngặt trong môi trường liên tổ chức [1], [8].

1.1.3. Đặc trưng, giới hạn và những cách hiểu cần tránh

Hệ thống Blockchain sở hữu các đặc trưng kỹ thuật cốt lõi gồm phân tán dữ liệu, khả năng kiểm chứng độc lập, tính truy vết giao dịch và cơ chế chống can thiệp dữ liệu hồi tố [1]. Mọi giao dịch hợp lệ đều gắn liền với định danh mật mã của bên khởi tạo và được neo chặt vào cấu trúc chuỗi khối, giúp người tham gia đối soát lịch sử một cách minh bạch mà không phụ thuộc một chủ thể duy nhất [2], [10]. Mặc dù vậy, việc nghiên cứu và ứng dụng đòi hỏi phải nhận diện rõ các giới hạn kỹ thuật và tránh ba ngộ nhận phổ biến:

Thứ nhất, dữ liệu ghi trên chuỗi không đồng nghĩa với chân lý thực tế; hệ thống chỉ bảo đảm tính toàn vẹn của dữ liệu sau khi đã được chấp nhận vào sổ cái. Nếu dữ liệu đầu vào bị sai lệch hoặc cố tình gian lận từ khâu thu thập ngoại chuỗi (off-chain), hệ thống vẫn lưu trữ và bảo vệ dữ liệu sai đó một cách bền vững theo nguyên lý “rác vào, rác ra” (Garbage-In, Garbage-Out) [1]. Thứ hai, không nên tuyệt đối hóa tính chất dữ liệu thành “bất biến vĩnh cửu”. Về bản chất, dữ liệu trên chuỗi chỉ mang tính kháng sửa đổi dựa trên các giả định an toàn mật mã và ngưỡng tài nguyên đồng thuận; nếu kẻ xấu kiểm soát vượt ngưỡng năng lực tính toán hoặc vượt số nút lỗi cho phép của thuật toán đồng thuận, lịch sử giao dịch vẫn có nguy cơ bị phân nhánh hoặc đảo ngược [1], [6].

Thứ ba, Blockchain không đồng nghĩa với tính ẩn danh hoàn toàn hay việc loại bỏ trách nhiệm pháp lý. Đa phần các mạng lưới chỉ cung cấp tính năng bí danh (pseudonymity) dựa trên địa chỉ khóa công khai; thông qua phân tích đồ thị luồng giao dịch và đối chiếu siêu dữ liệu ngoại vi, danh tính thực của thực thể vẫn có thể bị truy vết và định danh [1]. Ngoài ra, việc nhiều nút cùng kiểm tra và lưu bản sao

thường làm tăng độ trễ, chi phí lưu trữ và chi phí phối hợp so với cơ sở dữ liệu tập trung được tối ưu cho cùng một bài toán [10].

1.1.4. Phân biệt với DLT và cơ sở dữ liệu phân tán

Công nghệ sổ cái phân tán (DLT) là khái niệm bao trùm, chỉ chung mọi kiến trúc hệ thống lưu trữ và đồng bộ hóa sổ cái ghi chép trạng thái trên nhiều thực thể mạng mà không phụ thuộc vào một máy chủ trung tâm [1]. DLT không bắt buộc dữ liệu phải tổ chức thành khối liên kết chuỗi tuyến tính, mà có thể sử dụng cấu trúc đồ thị có hướng không chu trình (DAG) hoặc các dạng cây phi tuyến khác. Do đó, Blockchain là một tập con đặc thù của DLT, nổi bật với cấu trúc phân khối tuần tự và các liên kết mật mã chặt chẽ giữa các khối [1], [10].

Cơ sở dữ liệu phân tán truyền thống (chẳng hạn các hệ quản trị RDBMS hoặc NoSQL phân tán) cũng thực hiện việc phân mảnh và nhân bản dữ liệu qua nhiều máy chủ nhằm tối ưu thông lượng, độ sẵn sàng (Availability) và tính chịu lỗi phần cứng (Fault Tolerance). Tuy nhiên, các hệ thống này vận hành trong cùng một miền quản trị tin cậy (Single Trust Domain) dưới sự kiểm soát của một tổ chức [10]. Quản trị viên hệ thống (DBA) có toàn quyền can thiệp, xóa bản ghi hoặc chỉnh sửa lược đồ dữ liệu trực tiếp mà không cần sự đồng thuận ngang hàng của các bên độc lập.

Blockchain hướng tới môi trường có nhiều chủ thể cùng duy trì lịch sử theo một bộ quy tắc đã thống nhất [1], [8]. Trong thiết kế liên tổ chức, quyền cập nhật và quản trị nên được phân bổ sao cho một tài khoản đơn lẻ không thể âm thầm sửa lịch sử đã xác nhận; tuy nhiên, mức phân quyền thực tế vẫn phụ thuộc số tổ chức vận hành, chính sách nâng cấp và khả năng thông đồng. Khác biệt cốt lõi so với cơ sở dữ liệu phân tán truyền thống vì thế nằm ở ranh giới tin cậy và quy trình chấp nhận thay đổi, không chỉ ở số lượng máy lưu bản sao [10].

1.2. Cấu trúc dữ liệu và nền tảng mật mã

1.2.1. Giao dịch, khối và liên kết băm

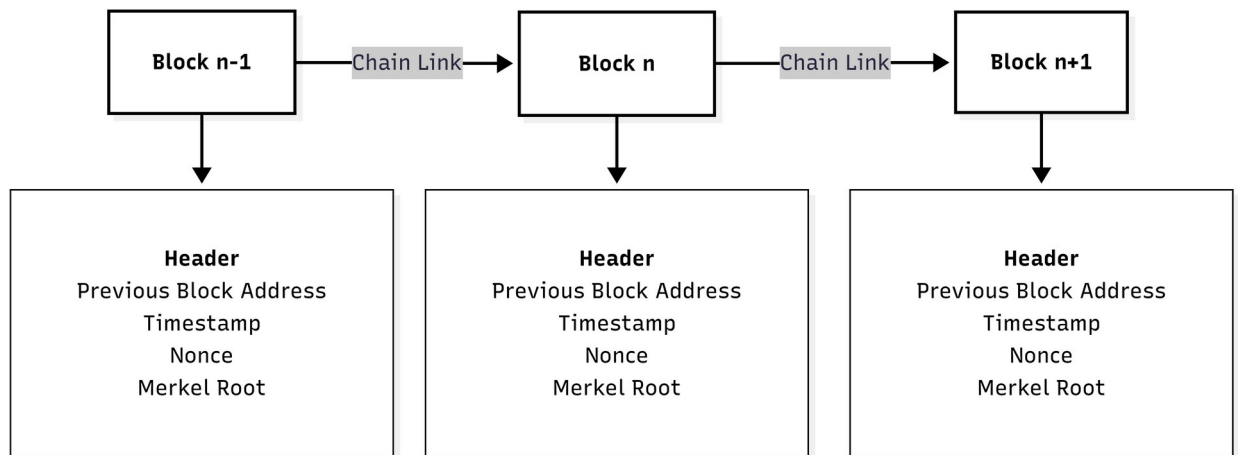
Giao dịch (Transaction) là đơn vị nguyên tử biểu diễn một yêu cầu thay đổi trạng thái trong hệ thống sổ cái phân tán [1], [10]. Tùy nền tảng, giao dịch có thể chứa định danh hoặc địa chỉ bên gửi, dữ liệu đầu vào, số thứ tự chống phát lại, tham số phí và chữ ký số. Khi được phát tán vào mạng lưới, các nút tiếp nhận tiến hành kiểm tra độc lập tính hợp lệ về mặt cú pháp, tính toàn vẹn chữ ký số và điều kiện logic trạng thái hiện hành trước khi chấp nhận đưa vào hàng đợi xử lý [1], [2].

Các giao dịch đã được sắp thứ tự có thể được đóng gói vào Khối (Block), gồm phần đầu chứa siêu dữ liệu giao thức và phần thân chứa giao dịch [1], [2].

Tùy thiết kế, phần đầu khối có thể bao gồm giá trị băm của khối trước H_{i-1} , gốc Merkle $MerkleRoot_i$, dấu thời gian $Timestamp_i$ và dữ liệu phục vụ đồng thuận $Metadata_i$. Mỗi liên kết mật mã giữa các khối liên tiếp có thể được biểu diễn khái quát như sau:

$$H_i = H(H_{i-1} \parallel MerkleRoot_i \parallel Timestamp_i \parallel Metadata_i) \quad (1.1)$$

Trong biểu thức (1.1), toán tử $\parallel$ biểu thị phép nối chuỗi. Đây là mô hình minh họa; tập trường và quy tắc mã hóa byte cụ thể do từng giao thức quy định. Giá trị H_{i-1} tạo liên kết kiểm chứng giữa hai khối liên tiếp.



Hình 1.1. Cấu trúc khối và liên kết mật mã giữa các khối liên tiếp

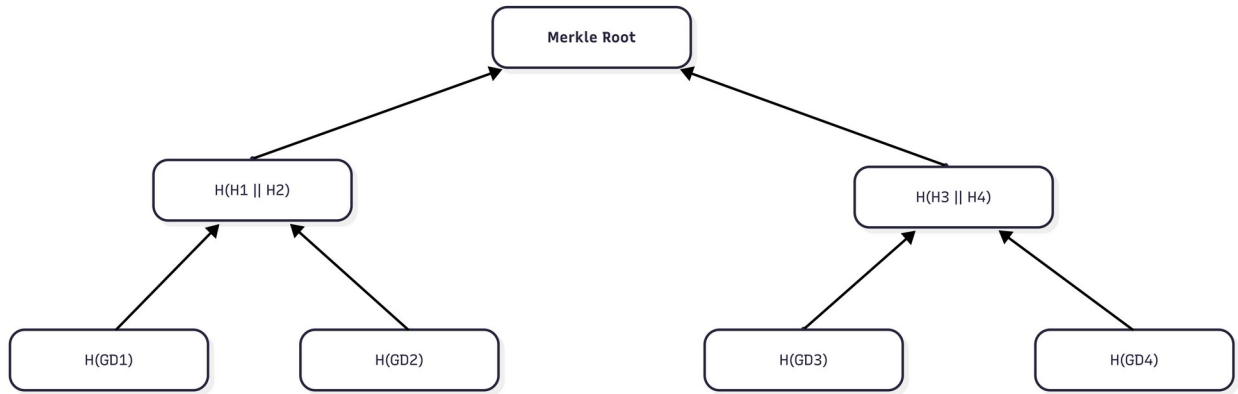
Nếu dữ liệu trong khối quá khứ k bị sửa, giá trị băm giao dịch, gốc Merkle và H_k thay đổi do tính nhạy của hàm băm [4]. Tham chiếu tại khối kế tiếp vì thế không còn khớp và sự can thiệp có thể được phát hiện. Để lịch sử đã sửa được các nút chấp nhận, đối phương còn phải đáp ứng quy tắc đồng thuận và chính sách quản trị của mạng; điều kiện cụ thể phụ thuộc loại Blockchain và mô hình đe dọa, không chỉ phụ thuộc liên kết băm [1], [2].

1.2.2. Hàm băm mật mã và cây Merkle

Hàm băm mật mã là công cụ toán học nền tảng để kiểm tra tính toàn vẹn dữ liệu trong kiến trúc chuỗi khối [1], [4]. Một hàm băm mật mã H ánh xạ thông điệp M có độ dài tùy ý thành chuỗi nhị phân cố định $h=H(M) \in \{0,1\}^n$; với SHA-256, $n=256$ [4]. Ba thuộc tính an toàn thường được xét gồm: kháng tiền ảnh (cho trước y , khó tìm M sao cho $H(M)=y$), kháng tiền ảnh thứ hai (cho trước M , khó tìm $M' \neq M$ sao cho $H(M')=H(M)$) và kháng va chạm (khó tìm hai thông điệp phân biệt có cùng giá trị băm). Hiệu ứng tuyết lở làm cho thay đổi nhỏ ở đầu vào dẫn đến thay đổi lớn, khó dự đoán ở đầu ra [4].

Để quản lý và xác thực danh sách giao dịch trong khối với chi phí tối ưu, giao thức sử dụng cấu trúc Cây Merkle (Merkle Tree) [3]. Cây Merkle thường

được biểu diễn dưới dạng cây nhị phân, trong đó mỗi nút lá lưu $L_j = H(Tx_j)$, còn mỗi nút trung gian được tính theo $N = H(N_L \parallel N_R)$. Tiến trình băm cặp này thực hiện đệ quy từ các nút lá lên đỉnh để tạo ra một giá trị đại diện duy nhất là Gốc Merkle (Merkle Root) được ghi vào phần đầu khối [3]. Cây nhị phân là mô hình minh họa phổ biến; cấu trúc và cách xử lý số nút lẻ có thể khác giữa các nền tảng.



Hình 1.2. Cây Merkle và đường dẫn kiểm chứng một giao dịch

Cấu trúc cây Merkle cho phép kiểm tra một giao dịch thuộc tập giao dịch đã cam kết thông qua Bảng chứng Merkle (Merkle Proof). Với cây cân bằng, kích thước bằng chứng và số phép băm khi xác minh ở mức $O(\log n)$, với n là tổng số giao dịch [3], [10]. Một nút nhẹ (Light Node) chỉ cần lưu trữ phần đầu khối chứa Merkle Root và yêu cầu mạng lưới cung cấp chuỗi các nút băm trung gian nằm dọc theo đường dẫn kiểm chứng (Audit Path) từ giao dịch cần kiểm tra lên gốc cây. Bằng cách thực hiện tối đa $\lceil \log_2 n \rceil$ phép băm đối với cây cân bằng, nút kiểm tra có thể tự xác thực tính hợp lệ của giao dịch mà không cần tải hay phân tích toàn bộ phần thân khối, giúp tối ưu hóa băng thông mạng và hiệu năng lưu trữ trong hệ thống phân tán [3], [10].

1.2.3. Mật mã khóa công khai và chữ ký số

Mật mã khóa công khai và chữ ký số thiết lập cơ chế xác thực nguồn gốc và kiểm soát ủy quyền đối với các giao dịch thay đổi trạng thái sổ cái [1], [5]. Theo NIST FIPS 186-5 [5], một lược đồ chữ ký số có thể được mô tả hình thức qua ba thuật toán: *KeyGen* sinh cặp khóa bí mật sk và khóa công khai pk ; *Sign* tạo chữ ký σ cho thông điệp m ; *Verify* kiểm tra chữ ký dựa trên pk và m :

$$KeyGen(1^\lambda) \rightarrow (sk, pk); \sigma = Sign(sk, m); Verify(pk, m, \sigma) \in \{0, 1\}. \quad (1.2)$$

Không nên diễn giải chữ ký số nói chung như việc “mã hóa giá trị băm bằng khóa bí mật rồi giải mã bằng khóa công khai”. Cách mô tả này không phản ánh mô hình của nhiều lược đồ hiện đại. Ở mức tổng quát, thuật toán *Verify* chỉ trả về kết quả hợp lệ hoặc không hợp lệ dựa trên khóa công khai, thông điệp và chữ ký; nó không khôi phục thông điệp bằng một phép giải mã [5].

Về phương diện an toàn thông tin, chữ ký số cung cấp ba thuộc tính cơ bản: xác thực nguồn gốc (Authentication), bảo toàn tính toàn vẹn (Integrity) và chống chối bỏ kỹ thuật (Non-repudiation) [1], [5]. Tuy nhiên, cần nhận thức rõ giới hạn biên của công nghệ: chữ ký số chỉ chứng minh tính hợp lệ về mặt toán học giữa bộ dữ liệu (m , σ , pk), chứ không thể chứng minh thực thể hợp pháp thực sự trực tiếp ký nếu khóa bí mật sk đã bị lộ lọt, bị mã độc đánh cắp hoặc trích xuất trái phép từ thiết bị lưu trữ [1], [5]. Do đó, giá trị chứng minh của chữ ký trong thực tế phụ thuộc vào quản lý vòng đời khóa, bảo vệ thiết bị ký, gắn khóa với đúng danh tính, ghi nhật ký sử dụng và thu hồi kịp thời khi có sự cố [1], [5].

1.2.4. Định danh, dấu thời gian và nonce

Định danh trong Blockchain cho phép gắn giao dịch với một khóa hoặc tư cách thành viên có quyền [1]. Ở mạng không cấp quyền, địa chỉ thường được tạo từ khóa công khai hoặc dữ liệu dẫn xuất và chủ yếu mang tính bí danh. Ở mạng cấp quyền, khóa công khai có thể được gắn với danh tính tổ chức thông qua chứng thư số và hạ tầng khóa công khai [8]. Trong cả hai trường hợp, chữ ký hợp lệ chỉ chứng minh việc sử dụng khóa tương ứng; mối liên hệ giữa khóa và cá nhân hoặc tổ chức ngoài đời vẫn phụ thuộc quy trình đăng ký, cấp phát và quản trị danh tính.

Dấu thời gian hỗ trợ sắp xếp sự kiện và áp dụng các quy tắc có yếu tố thời hạn, nhưng không nên mặc nhiên coi thời gian do một nút cung cấp là bằng chứng thời gian pháp lý. Mạng phân tán không có đồng hồ vật lý chung tuyệt đối; từng giao thức phải quy định nguồn thời gian, khoảng sai lệch chấp nhận và cách xử lý nút có đồng hồ sai [1], [2]. Khi nghiệp vụ cần bằng chứng thời gian có thẩm quyền, hệ thống phải kết hợp cơ chế xác nhận phù hợp ngoài Blockchain.

Nonce là giá trị được giao thức sử dụng để tạo tính duy nhất hoặc thứ tự. Trong PoW, nút khai thác thay đổi nonce để tìm giá trị băm thỏa điều kiện độ khó [2]. Trong mô hình tài khoản, nonce thường là bộ đếm tăng dần để các nút phát hiện giao dịch cũ hoặc sai thứ tự. Cơ chế này hỗ trợ chống phát lại khi được kiểm tra cùng chữ ký và ngữ cảnh giao dịch; nó không thay thế toàn bộ biện pháp bảo vệ phiên và giao thức [1], [10].

1.3. Kiến trúc và nguyên lý hoạt động

1.3.1. Mạng ngang hàng và vai trò của nút

Mạng ngang hàng (Peer-to-Peer - P2P) cho phép các nút trao đổi trực tiếp giao dịch, khối và thông tin trạng thái mà không bắt buộc mọi thông điệp phải đi qua một máy chủ trung tâm [1], [2]. Mỗi nút kiểm tra dữ liệu nhận được theo quy tắc giao thức trước khi chuyển tiếp cho các nút lân cận. Cách tổ chức này hỗ trợ

nhân bản dữ liệu, tăng khả năng tiếp tục hoạt động khi một số kết nối gặp sự cố và làm giảm phụ thuộc vào một điểm truyền tin duy nhất. Tuy nhiên, P2P không đồng nghĩa với phân quyền tuyệt đối: các nút vẫn có thể tập trung tại cùng nhà cung cấp hạ tầng, cùng miền quản trị hoặc chịu ảnh hưởng của một nhóm vận hành nhỏ [1], [10].

Vai trò của nút phụ thuộc nền tảng. Nút đầy đủ thường lưu đủ dữ liệu cần thiết để tự kiểm tra lịch sử; nút nhẹ chỉ giữ một phần thông tin và dựa vào bằng chứng mật mã; nút đề xuất hoặc xác thực tham gia đồng thuận; một số nền tảng còn tách riêng nhiệm vụ sắp thứ tự, thực thi và lưu trữ [1], [8]. Vì vậy, các tên gọi như nút khai thác, nút xác thực hay nút lưu trữ không phải bộ phân loại áp dụng cho mọi Blockchain. Khi đánh giá kiến trúc, cần xác định rõ mỗi nút lưu gì, kiểm tra gì, thuộc tổ chức nào và có quyền tác động đến trạng thái ra sao.

1.3.2. Vòng đời giao dịch



Hình 1.3. Vòng đời giao dịch Blockchain ở mức khái quát

Vòng đời giao dịch bắt đầu khi chủ thể tạo yêu cầu thay đổi trạng thái và ký bằng khóa bí mật tương ứng. Nút tiếp nhận kiểm tra cấu trúc, chữ ký, số thứ tự chống phát lại, quyền thao tác và điều kiện trạng thái hiện tại [1]. Giao dịch hợp lệ được truyền tới các thành phần tham gia sắp thứ tự hoặc đồng thuận. Trong mạng công khai, giao dịch có thể được giữ trong vùng chờ trước khi nút đề xuất lựa chọn; mạng cấp quyền có thể sử dụng quy trình đề xuất, chứng thực và sắp thứ tự khác mà không có hoạt động khai thác [2], [8].

Sau khi đạt điều kiện chấp nhận của giao thức, giao dịch được đưa vào khối hoặc lô dữ liệu, truyền tới các nút và kiểm tra lần cuối trước khi cập nhật trạng thái. Ứng dụng chỉ nên thông báo thành công khi đã nhận bằng chứng giao dịch được cam kết hợp lệ, thay vì coi việc tạo mã giao dịch hay phát tán yêu cầu là hoàn tất. Mức độ chắc chắn của kết quả phụ thuộc cơ chế đồng thuận: một số mạng cần nhiều khối xác nhận, trong khi mạng dựa trên tấc số có thể đạt hoàn tất xác định dưới các giả định vận hành đã nêu [1], [6].

1.3.3. Sao chép trạng thái và thực thi xác định

Blockchain có thể được phân tích như một dạng sao chép máy trạng thái. Gọi S là trạng thái, T là giao dịch và δ là hàm chuyển trạng thái; cùng trạng thái đầu vào và cùng thứ tự giao dịch phải tạo cùng kết quả $S' = \delta(S, T)$ tại các nút trung thực [10]. Điều kiện xác định này đòi hỏi chương trình tránh phụ thuộc trực tiếp

vào đồng hồ cục bộ, số ngẫu nhiên không được thống nhất, phản hồi mạng bên ngoài hoặc cách làm tròn khác nhau giữa các máy.

Nhiều nền tảng tách nhật ký giao dịch nối tiếp khỏi phần biểu diễn trạng thái mới nhất để vừa bảo toàn lịch sử vừa truy vấn hiệu quả. Nhật ký cho phép kiểm toán và tái dựng; trạng thái hiện thời phục vụ nghiệp vụ đọc thường xuyên. Đây là mô hình kiến trúc phổ biến nhưng cách tổ chức dữ liệu, xử lý giao dịch không hợp lệ và phục hồi trạng thái thay đổi theo từng nền tảng [8], [10].

1.3.4. Hợp đồng thông minh và dữ liệu ngoài hệ thống

Hợp đồng thông minh là chương trình biểu diễn quy tắc chuyển trạng thái và được các nút thực thi theo cơ chế của nền tảng [1], [10]. Nó có thể chuẩn hóa điều kiện nghiệp vụ và tạo dấu vết nhất quán, nhưng không tự bảo đảm logic được viết đúng. Lỗi phân quyền, kiểm tra đầu vào, xử lý trạng thái hoặc cơ chế nâng cấp có thể gây hậu quả trên toàn mạng. Vì vậy, mã cần được kiểm thử, rà soát an toàn, quản lý phiên bản và triển khai theo quy trình phê duyệt phù hợp.

Môi trường thực thi xác định không thể tùy ý lấy dữ liệu từ Internet vì các nút có thể nhận kết quả khác nhau. Dữ liệu ngoài hệ thống thường được đưa vào qua nguồn cung cấp có thẩm quyền hoặc cơ chế oracle. Chữ ký của nguồn chỉ chứng minh dữ liệu đến từ khóa tương ứng và không bị sửa sau khi ký; nó không chứng minh nội dung phản ánh đúng sự thật ngoài đời. Thiết kế an toàn cần quy định nguồn tin cậy, thời hạn hiệu lực, cách xử lý sai lệch và cơ chế dừng an toàn khi dữ liệu ngoài không đáng tin [1].

1.4. Cơ chế đồng thuận

1.4.1. Bài toán đồng thuận và mô hình lỗi

Đồng thuận giúp các nút thống nhất thứ tự giao dịch và trạng thái được chấp nhận trong điều kiện thông điệp có thể đến không đồng thời hoặc một số nút gặp sự cố [1]. Hai thuộc tính thường được xem xét là tính an toàn, nghĩa là các nút trung thực không chấp nhận hai quyết định mâu thuẫn trong cùng ngữ cảnh, và tính sống, nghĩa là hệ thống có thể tiếp tục đưa ra quyết định khi các giả định về mạng và số nút lỗi còn được thỏa mãn [6], [7]. Không giao thức nào bảo đảm mọi thuộc tính trong mọi điều kiện; mỗi thiết kế phải nêu mô hình mạng, loại lỗi và ngưỡng đối phương mà nó chịu được.

Mô hình chịu lỗi dừng (Crash Fault Tolerance - CFT) giả định nút lỗi chỉ dừng hoặc mất kết nối, không chủ động tạo thông điệp sai. Mô hình chịu lỗi Byzantine (Byzantine Fault Tolerance - BFT) mạnh hơn vì cho phép nút bị chiếm quyền gửi thông điệp khác nhau tới các bên, thông đồng hoặc cố ý vi phạm giao

thức [6], [7]. Việc lựa chọn CFT hay BFT phải dựa trên ranh giới tin cậy: nút trong cùng miền quản trị và được bảo vệ tốt có thể chấp nhận giả định lỗi dừng, còn mạng liên tổ chức có nguy cơ hành vi tùy ý cần đánh giá BFT hoặc biện pháp kiểm soát bổ sung.

1.4.2. Proof of Work và Proof of Stake

Trong mạng không cấp quyền, một thực thể có thể tạo nhiều định danh với chi phí thấp để thao túng biểu quyết. Proof of Work (PoW) gắn quyền đề xuất khối với công việc tính toán, còn Proof of Stake (PoS) gắn quyền tham gia với lượng tài sản cam kết và các quy tắc khuyến khích hoặc xử phạt [1], [2]. Hai nhóm cơ chế này tạo chi phí cho tấn công Sybil mà không cần danh sách thành viên cố định.

PoW có lịch sử vận hành rõ trong Bitcoin nhưng tiêu thụ nhiều năng lượng và thường đạt hoàn tất theo xác suất [2]. PoS giảm nhu cầu tính toán lặp lại, song độ an toàn phụ thuộc cách chọn người đề xuất, cơ chế xử lý nhánh cạnh tranh, phân phối cổ phần và quản trị giao thức. Không phải mọi PoS đều có cùng mô hình hoàn tất hoặc hiệu năng. Đối với mạng gồm các tổ chức đã định danh, PoW và PoS thường không phải lựa chọn đầu tiên vì quyền tham gia có thể được kiểm soát trực tiếp bằng danh tính và chính sách [1], [8].

1.4.3. Đồng thuận CFT và BFT trong mạng định danh

Raft là thuật toán CFT sử dụng nút dẫn dắt để sắp thứ tự và sao chép nhật ký; một mục được cam kết khi đạt đa số theo quy tắc của giao thức [7]. PBFT cổ điển hướng tới mô hình Byzantine và sử dụng các pha trao đổi thông điệp để các bản sao thống nhất thứ tự yêu cầu ngay cả khi có một số nút hành xử tùy ý [6]. So với PoW hoặc PoS, các giao thức này dựa trên tập thành viên đã biết và cơ chế tức số, nên phù hợp hơn với nhiều mạng cấp quyền.

Các ngưỡng chịu lỗi phụ thuộc giả định và giao thức, không phải công thức phổ quát cho mọi thuật toán CFT/BFT. Với cấu hình đa số CFT và mô hình PBFT cổ điển, quan hệ số nút thường được viết:

$$n \geq 2f + 1 \text{ (CFT)}; n \geq 3f + 1 \text{ (BFT)} \quad (1.3)$$

Trong đó n là tổng số nút và f là số nút lỗi tối đa. Với $n=3f+1$, một tức số BFT thường có $2f+1$ phiếu; hai tức số như vậy giao nhau ít nhất $f+1$ nút, nên phần giao chứa ít nhất một nút trung thực khi có tối đa f nút Byzantine [6]. PBFT cổ điển còn có chi phí truyền thông bậc hai $O(n^2)$ ở các pha nhiều nút trao đổi chéo, do đó chi phí tăng nhanh theo quy mô mạng [6].

1.4.4. Tính hoàn tất và đánh đổi thiết kế

Tính hoàn tất (finality) biểu thị mức độ chắc chắn rằng một giao dịch đã được chấp nhận sẽ không bị đảo ngược trong điều kiện an toàn giả định. Trong cơ chế hoàn tất theo xác suất, khả năng tổ chức lại lịch sử giảm dần khi có thêm khối xác nhận nhưng không bằng không về mặt tuyệt đối [2]. Trong cơ chế hoàn tất xác định dựa trên tấc số, giao dịch được coi là hoàn tất sau khi đạt đủ phiếu theo giao thức; kết luận này vẫn phụ thuộc vào giả định số nút lỗi, an toàn khóa và quy tắc quản trị [6], [7].

So sánh khái quát các họ cơ chế đồng thuận

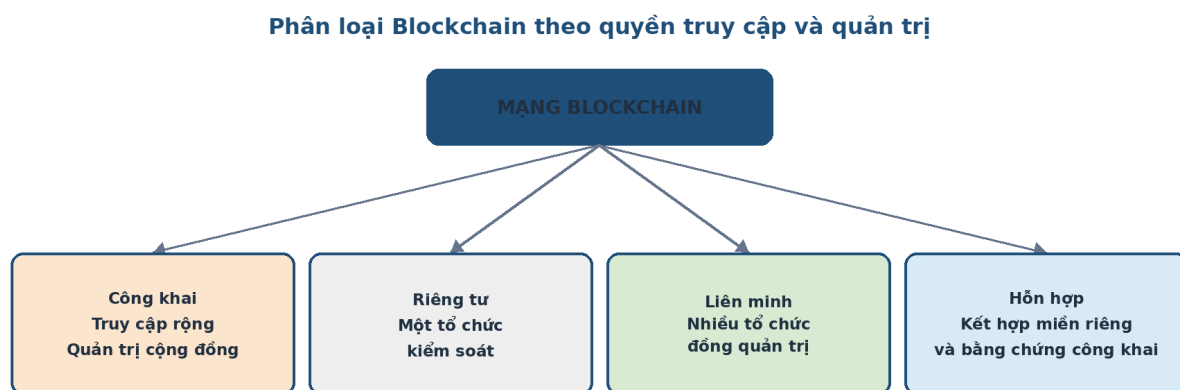
Tiêu chí	Proof of Work	Proof of Stake	Mạng định danh
Căn cứ	Tính toán	Tài sản đặt cọc	Danh tính, biểu quyết
Ưu điểm	Mở, đã kiểm chứng	Ít năng lượng hơn	Độ trễ thấp
Đánh đổi	Tốn năng lượng	Phức tạp kinh tế	Cần quản trị thành viên
Kết thúc	Thường xác suất	Xác suất/xác định	Thường xác định

Hình 1.4. So sánh khái quát các nhóm cơ chế đồng thuận

Lựa chọn đồng thuận là bài toán đánh đổi giữa mức phân tán quyền quyết định, khả năng chịu lỗi, độ trễ, thông lượng và chi phí vận hành [10]. “Bộ ba phân quyền – an toàn – khả năng mở rộng” có thể dùng như một khung trực giác, không phải định luật buộc mọi hệ thống chỉ được chọn hai thuộc tính. Cơ chế phù hợp phải được đánh giá trên tải thực tế, số tổ chức vận hành, loại hành vi đối phương và yêu cầu phục hồi, thay vì dựa trên tên gọi hoặc tuyên bố hiệu năng của nền tảng.

1.5. Phân loại Blockchain và điều kiện lựa chọn

1.5.1. Các trục phân loại



Cần xét riêng quyền đọc, gửi giao dịch, xác thực và quản trị thay vì chỉ nhìn một nhãn.

Hình 1.5. Phân loại Blockchain theo phạm vi tham gia và quyền truy cập

Blockchain có thể được phân loại theo nhiều trục độc lập. Trục quyền tham gia phân biệt mạng không cấp quyền, nơi chủ thể có thể tham gia theo quy tắc mở, với mạng cấp quyền, nơi danh tính và vai trò được phê duyệt trước [1]. Trục phạm vi quản trị phân biệt mạng công khai, mạng riêng do một miền quản trị chi phối và mạng liên minh do nhiều tổ chức cùng quản trị. Quyền đọc dữ liệu lại là một quyết định khác: một mạng cấp quyền có thể cho công chúng đọc bằng chứng tối thiểu, trong khi một mạng công khai vẫn có thể sử dụng cơ chế bảo vệ nội dung riêng tư. Vì vậy, không nên đồng nhất “công khai” với “không cấp quyền”, hoặc “riêng tư” với “cấp quyền” [1], [8].

1.5.2. So sánh các mô hình triển khai

Các đặc điểm chính của ba mô hình triển khai được đối chiếu trong Bảng 1.1.

Bảng 1.1. So sánh khái quát các mô hình triển khai Blockchain

Tiêu chí	Công khai, không cấp quyền	Riêng, cấp quyền	Liên minh, cấp quyền
Quản trị	Cộng đồng và quy tắc giao thức	Một tổ chức hoặc một miền quản trị	Nhiều tổ chức theo chính sách chung
Thành viên xác thực	Mở theo điều kiện chống Sybil	Danh tính được phê duyệt nội bộ	Danh tính thuộc các tổ chức thành viên

Tiêu chí	Công khai, không cấp quyền	Riêng, cấp quyền	Liên minh, cấp quyền
Khả năng kiểm chứng	Rộng, thường công khai	Chủ yếu trong nội bộ	Giữa các thành viên và bên được cấp quyền
Quyền riêng tư	Khó kiểm soát phạm vi phổ biến dữ liệu	Dễ giới hạn hơn theo chính sách nội bộ	Có thể phân vùng theo tổ chức và nghiệp vụ
Hiệu năng	Phụ thuộc mạng mở và cơ chế kinh tế	Có thể tối ưu trong phạm vi nhỏ	Phụ thuộc số tổ chức, tức số và hạ tầng
Bài toán phù hợp	Hệ sinh thái mở cần kiểm chứng rộng	Nhật ký và quy trình nội bộ	Phối hợp và kiểm toán liên tổ chức

Mạng mở tạo khả năng kiểm chứng rộng và chống Sybil bằng nguồn lực kinh tế, nhưng thường phát sinh phí, độ trễ và khó kiểm soát phạm vi phổ biến dữ liệu. Mạng riêng dễ quản trị và tối ưu hiệu năng, song giá trị phân tán niềm tin bị hạn chế nếu một tổ chức vẫn quyết định toàn bộ lịch sử. Mạng liên minh phân bổ quyền giữa nhiều tổ chức đã định danh, thuận lợi cho kiểm toán liên tổ chức và chính sách dữ liệu có kiểm soát, nhưng đòi hỏi thỏa thuận quản trị, vận hành nút và xử lý tranh chấp [1], [8], [10].

Không mô hình nào mặc nhiên tốt hơn. Hiệu năng thực tế phụ thuộc thuật toán đồng thuận, số nút, chính sách xác nhận, hạ tầng mạng, kích thước giao dịch và cách tổ chức dữ liệu. Vì vậy, các con số thông lượng hoặc độ trễ chỉ có ý nghĩa khi được đo trên cấu hình và tải xác định [10].

1.5.3. Khi nên và không nên sử dụng Blockchain

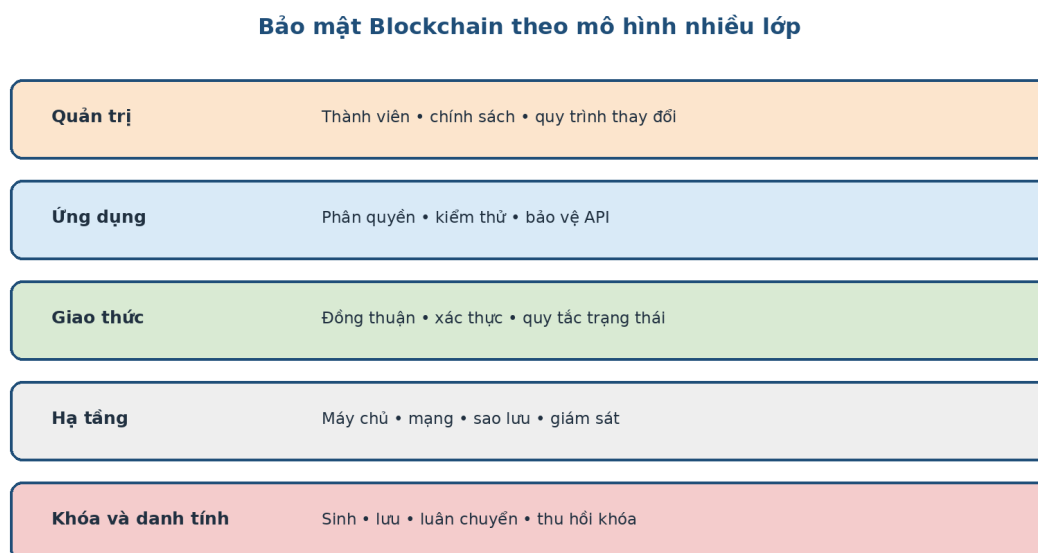
Nên xem xét Blockchain khi nhiều tổ chức cần cùng cập nhật trạng thái, lịch sử phải được kiểm chứng độc lập và không bên nào được chấp nhận làm chủ thể duy nhất có quyền sửa dữ liệu. Quy trình cũng phải đủ rõ để biểu diễn thành giao dịch, các bên có động lực vận hành chung và dữ liệu có thể được tối thiểu hóa phù hợp với yêu cầu riêng tư [1], [10].

Không nên sử dụng Blockchain nếu chỉ một tổ chức chịu trách nhiệm và được các bên tin cậy, nếu dữ liệu phải sửa hoặc xóa thường xuyên, hoặc nếu tải và độ trễ vượt khả năng của cơ chế đồng thuận. Một cơ sở dữ liệu truyền thống có

nhật ký kiểm toán và chữ ký số có thể đơn giản, rẻ và dễ quản trị hơn trong các trường hợp đó. Blockchain cũng không sửa được một quy trình nghiệp vụ mơ hồ, dữ liệu đầu vào kém chất lượng hay tranh chấp trách nhiệm giữa các tổ chức.

1.6. An toàn, quyền riêng tư và khả năng mở rộng

1.6.1. Mô hình đe dọa và bảo vệ nhiều lớp



An toàn là thuộc tính của toàn hệ thống, không chỉ của chuỗi khối.

Hình 1.6. Các lớp bảo vệ trong một hệ thống Blockchain

An toàn Blockchain không chỉ phụ thuộc hàm băm hoặc đồng thuận mà phải được xem xét theo nhiều lớp: mật mã, mạng, đồng thuận, hợp đồng thông minh, ứng dụng, lưu trữ ngoài chuỗi và quản trị vận hành [1], [10]. Tài sản cần bảo vệ gồm khóa bí mật, danh tính thành viên, trạng thái sổ cái, dữ liệu nghiệp vụ, cấu hình nút và nhật ký. Đối phương có thể là người ngoài tấn công hạ tầng, người dùng bị chiếm tài khoản, quản trị viên lạm quyền hoặc một nhóm nút thông đồng.

Biện pháp bảo vệ phải bám vào mô hình đe dọa. Kênh truyền cần được mã hóa và xác thực; nút cần vá lỗi, phân đoạn mạng, giám sát và sao lưu; chính sách đồng thuận phải giới hạn quyền của một tổ chức; ứng dụng phải kiểm soát đầu vào, phiên đăng nhập và phân quyền. Việc nhiều nút lưu sổ cái không thay thế kế hoạch ứng phó sự cố: nếu các nút cùng phụ thuộc một hạ tầng, cùng dùng cấu hình sai hoặc cùng bị lộ khóa, lỗi có thể lan rộng trên toàn hệ thống.

1.6.2. Quản lý khóa, hợp đồng thông minh và nguồn dữ liệu ngoài

Khóa bí mật đại diện cho quyền ký giao dịch nên cần được quản lý suốt vòng đời: sinh bằng nguồn ngẫu nhiên an toàn, lưu trong kho được bảo vệ, giới hạn mục đích sử dụng, luân chuyển, sao lưu có kiểm soát và thu hồi khi nghi ngờ lộ

[5]. Mô-đun bảo mật phần cứng, chữ ký nhiều bên hoặc mật mã ngưỡng có thể giảm rủi ro một khóa hay một cá nhân trở thành điểm lỗi đơn, nhưng không loại bỏ nhu cầu quản trị và phục hồi.

Hợp đồng thông minh cần kiểm tra quyền, dữ liệu đầu vào, chuyển trạng thái và các trường hợp đồng thời; phiên bản mới phải có quy trình rà soát, kiểm thử và phê duyệt [10]. Nguồn dữ liệu ngoài phải được xác định thẩm quyền, ký số khi phù hợp và có thời hạn hiệu lực. Dùng nhiều nguồn chỉ hữu ích khi các nguồn đủ độc lập; nếu tất cả cùng lấy dữ liệu từ một hệ thống gốc, biểu quyết không làm dữ liệu đáng tin hơn.

1.6.3. Quyền riêng tư và dữ liệu cá nhân

Sở cái được nhân bản và khó sửa đặt ra xung đột với yêu cầu hạn chế mục đích sử dụng, chỉnh sửa hoặc xóa dữ liệu cá nhân. Đưa trực tiếp họ tên, ngày sinh, điểm số hay tệp văn bản lên chuỗi làm tăng phạm vi sao chép và khó kiểm soát thời hạn lưu giữ. Mã hóa không giải quyết hoàn toàn vấn đề vì bản mã tồn tại lâu dài và khóa có thể bị lộ trong tương lai.

Cách tiếp cận phù hợp là tối thiểu hóa dữ liệu trên chuỗi, giữ hồ sơ chi tiết ở kho ngoài chuỗi có phân quyền và chỉ ghi bằng chứng mật mã hoặc trạng thái cần thiết cho kiểm chứng [1], [9]. Tuy nhiên, giá trị băm không mặc nhiên ẩn danh: nếu dữ liệu gốc dễ đoán, kẻ tấn công có thể thử các khả năng và so sánh kết quả; bản ghi cũng có thể liên kết với sự kiện hoặc định danh khác. Việc xóa dữ liệu ngoài chuỗi không xóa bằng chứng đã ghi, vì vậy thiết kế phải đánh giá khả năng liên kết, căn cứ xử lý và trách nhiệm của từng bên.

1.6.4. Khả năng mở rộng và khả năng tương tác

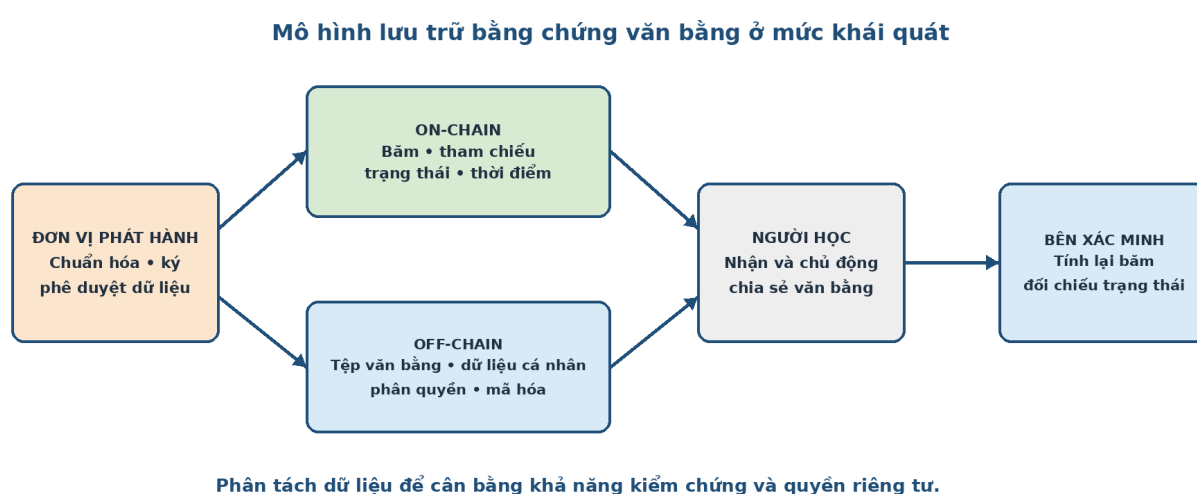
Khi nhiều nút cùng lưu và kiểm tra giao dịch, thông lượng, độ trễ, băng thông và dung lượng trở thành các giới hạn cần đo [10]. Có thể cải thiện bằng cách xử lý theo lô, tối ưu chính sách xác nhận, phân vùng dữ liệu hoặc đưa một phần xử lý ra ngoài chuỗi; mỗi cách tạo thêm giả định tin cậy và độ phức tạp. Các giải pháp lớp 2 chỉ nên được xem xét khi thật sự kế thừa an toàn từ chuỗi cơ sở, không nên đồng nhất mọi sidechain hoặc cầu nối với lớp 2.

Khả năng tương tác không chỉ là truyền thông điệp giữa hai mạng mà còn là thống nhất ý nghĩa dữ liệu, định danh tổ chức, phiên bản lược đồ và trạng thái nghiệp vụ. Cầu nối hoặc dịch vụ chuyển tiếp bổ sung điểm tin cậy mới và từng là mục tiêu của nhiều sự cố. Các chuẩn dữ liệu chứng thực như W3C Verifiable Credentials giúp thống nhất cách biểu diễn vai trò phát hành, chủ thể và bên xác minh, nhưng không tự giải quyết quản trị khóa, quyền truy cập hay tranh chấp pháp lý [9].

1.7. Định hướng ứng dụng trong quản lý văn bằng, chứng chỉ

1.7.1. Bài toán, tác nhân và yêu cầu kiểm chứng

Quản lý văn bằng, chứng chỉ liên quan ba vai trò chính: cơ sở đào tạo phát hành, người học nắm giữ hoặc chia sẻ và tổ chức bên ngoài xác minh [9]. Quy trình truyền thống thường dựa vào bản giấy, bản sao chứng thực hoặc yêu cầu xác nhận thủ công; khi dữ liệu nằm ở nhiều hệ thống, việc đối soát có thể chậm và khó phát hiện tài liệu đã bị chỉnh sửa. Một sổ bằng chứng dùng chung có thể hỗ trợ bên xác minh kiểm tra nguồn phát hành, tính toàn vẹn và trạng thái hiện thời mà giảm số bước liên hệ thủ công.



Hình 1.7. Mô hình khái niệm ứng dụng Blockchain trong quản lý văn bằng

Yêu cầu kiểm chứng không chỉ là so sánh mã băm. Hệ thống phải xác định đơn vị nào có thẩm quyền phát hành, khóa nào hợp lệ tại thời điểm ký, dữ liệu nào tạo thành nội dung văn bằng và trạng thái nào cho phép sử dụng. Kết quả không tìm thấy bằng chứng cũng không đủ để kết luận văn bằng giả, bởi hồ sơ cũ có thể chưa được số hóa hoặc nằm ngoài phạm vi hệ thống.

1.7.2. Mô hình dữ liệu khái niệm trên chuỗi và ngoài chuỗi

Ở mức khái niệm, sổ cái chỉ nên giữ lượng dữ liệu tối thiểu phục vụ kiểm chứng, chẳng hạn cam kết mật mã, tham chiếu đơn vị phát hành và trạng thái hiệu lực. Hồ sơ đầy đủ cùng dữ liệu cá nhân được quản lý ngoài chuỗi trong hệ thống có thẩm quyền, với mã hóa, phân quyền, nhật ký truy cập và sao lưu. Hai miền được liên kết bằng quy tắc tạo bằng chứng xác định để tài liệu bị thay đổi tạo kết quả kiểm tra khác [1], [9].

Cách phân tách này giảm việc sao chép dữ liệu nhạy cảm nhưng không làm rủi ro riêng tư biến mất. Bằng chứng trên chuỗi vẫn có thể bị liên kết hoặc dò đoán;

kho ngoài chuỗi vẫn có thể bị mất, sửa hoặc ngừng phục vụ. Vì vậy, Chương II cần xác định mô hình dữ liệu, quy tắc chuẩn hóa, quyền truy cập và cơ chế xử lý khi dữ liệu nguồn sai mà không giả định Blockchain tự giải quyết các vấn đề đó.

1.7.3. Giá trị kỳ vọng, giới hạn và ranh giới trách nhiệm

Giá trị kỳ vọng là rút ngắn thời gian xác minh, tăng khả năng phát hiện tài liệu bị sửa và tạo lịch sử trạng thái dùng chung giữa các tổ chức. Công nghệ có thể hỗ trợ ghi nhận phát hành hoặc thay đổi hiệu lực, nhưng không tự quyết định người học đủ điều kiện tốt nghiệp, không chứng minh người xuất trình là chủ văn bằng và không đánh giá chất lượng đào tạo.

Cơ sở đào tạo vẫn chịu trách nhiệm về dữ liệu đầu vào và thẩm quyền phát hành; người vận hành chịu trách nhiệm bảo vệ khóa và hạ tầng; bên xác minh phải diễn giải kết quả đúng phạm vi. Nếu khóa phát hành bị lộ, bản ghi kỹ thuật hợp lệ vẫn có thể được tạo cho đến khi quyền bị thu hồi. Do đó, Blockchain phải được xem là công cụ hỗ trợ kiểm chứng, không phải nguồn phán quyết duy nhất.

1.7.4. Mô hình Blockcerts V3

Blockcerts tổ chức chứng thư dưới dạng tài liệu JSON-LD có định danh, thông tin người nhận, nội dung thành tích, đơn vị cấp và bằng chứng mật mã. JSON-LD giúp dữ liệu có ngữ cảnh ngữ nghĩa thay vì chỉ là tập cặp khóa–giá trị. Trong phiên bản V3, tài liệu sử dụng cấu trúc tương thích với hệ sinh thái Verifiable Credentials và bộ schema của Blockcerts [9], [11], [12].

Quy trình phát hành bắt đầu bằng tài liệu chưa có proof. Hệ thống chuẩn hóa phần dữ liệu cần bảo vệ, tính target hash cho từng chứng thư và đưa các hash vào cây Merkle. Merkle root đại diện cho toàn bộ lô. Sau khi root được neo lên Bitcoin, mỗi chứng thư được bổ sung đường dẫn Merkle từ leaf của nó tới root và thông tin receipt để verifier tìm giao dịch. Một chứng thư vì vậy có thể tự mang bằng chứng cần thiết mà không buộc cơ sở cấp công bố toàn bộ lô.

Bằng chứng MerkleProof2019 liên kết nội dung JSON-LD với Merkle root [14]. Verifier tính lại hash từ nội dung nhận được, đi qua các nút proof theo đúng thứ tự và so sánh root thu được với dữ liệu neo trên chuỗi. Nếu một trường được bảo vệ bị thay đổi, target hash thay đổi và proof không còn dẫn tới root ban đầu. Cơ chế phát hiện sửa đổi không phụ thuộc việc tệp được nhận từ cổng Holder, thư điện tử hay kho lưu trữ cá nhân.

Issuer Profile tạo liên kết giữa khóa công khai và danh tính đơn vị cấp. Verifier không chỉ kiểm tra một transaction tồn tại mà còn đối chiếu địa chỉ phát hành, profile và khoảng thời gian khóa có hiệu lực. Khóa riêng không được đưa vào profile. Việc xác minh danh tính tổ chức ở ngoài chuỗi vẫn là một bước quản

trị quan trọng; blockchain không tự chứng minh rằng một profile công khai thực sự thuộc cơ sở đào tạo nếu khâu thiết lập ban đầu sai.

Cơ chế thu hồi bổ sung chiều thời gian cho bằng chứng. Tính toàn vẹn cho biết tài liệu không bị sửa so với lúc phát hành, còn danh sách thu hồi phản ánh việc đơn vị cấp tiếp tục hoặc chấm dứt công nhận tài liệu tại thời điểm xác minh. Hai kiểm tra này phải tồn tại song song. Một chứng thư đã thu hồi vẫn có thể có proof và anchor hoàn toàn hợp lệ, nhưng kết quả nghiệp vụ phải là REVOKED.

1.7.5. Bitcoin OP_RETURN và môi trường regtest

Bitcoin cho phép lưu một lượng dữ liệu nhỏ trong output không thể chi tiêu bằng OP_RETURN. Blockcerts sử dụng vị trí này để neo giá trị đại diện cho lô thay vì ghi toàn bộ hồ sơ người học lên chuỗi. Cách tiếp cận giảm dung lượng, tránh công bố dữ liệu cá nhân và tận dụng tính khó sửa đổi của lịch sử giao dịch. Dữ liệu ngoài chuỗi vẫn cần được lưu, sao lưu và cung cấp cho Holder.

Một giao dịch neo có thể chứa Merkle root của nhiều chứng thư. Chi phí anchor vì thế chủ yếu tăng theo số transaction chứ không tuyến tính theo số chứng thư trong cùng lô. Tuy nhiên, lô quá lớn làm tăng thời gian chuẩn bị, kích thước artifact, thời gian xử lý lỗi và phạm vi ảnh hưởng nếu job thất bại. Kích thước batch cần được chọn từ số liệu thực nghiệm thay vì giả định batching luôn tốt hơn ở mọi quy mô.

Regtest là chế độ mạng riêng do người thử nghiệm kiểm soát [18]. Khởi có thể được tạo theo yêu cầu, coin không có giá trị và kết quả dễ tái lập. Đây là lựa chọn phù hợp để kiểm tra logic phát hành, xác minh, confirmation và thu hồi mà không phát sinh chi phí thật. Đổi lại, regtest không tái hiện độ trễ mempool, biến động phí, cạnh tranh khai thác, độ phân tán nút hoặc rủi ro tổ chức lại chuỗi của mainnet.

Báo cáo do đó chỉ dùng regtest để chứng minh tính đúng của nguyên mẫu và đo hành vi trong môi trường kiểm soát. Các số liệu thời gian không được diễn giải thành cam kết hiệu năng cho Bitcoin mainnet. Việc triển khai thực tế còn cần chính sách số confirmation, quản lý phí, giám sát transaction, dự phòng RPC và bảo vệ khóa bằng hạ tầng chuyên dụng.

1.7.6. So sánh các hướng quản lý văn bằng số

Các đặc điểm cần cân nhắc giữa bốn hướng tiếp cận được tổng hợp trong Bảng 1.2.

Bảng 1.2. So sánh các hướng quản lý và xác minh văn bằng

Hướng tiếp cận	Nguồn tin cậy chính	Khả năng xác minh độc lập	Thu hồi	Rủi ro nổi bật
Cơ sở dữ liệu tập trung	Cổng trực tuyến của cơ sở cấp	Phụ thuộc cổng còn hoạt động	Cập nhật trực tiếp trong DB	Điểm lỗi tập trung và quyền quản trị lớn
Tập ký số truyền thống	Chứng thư khóa công khai và chữ ký	Có thể kiểm tra ngoại tuyến nếu đủ chuỗi tin cậy	CRL hoặc OCSP	Quản trị PKI và vòng đời chứng thư số
Verifiable Credentials	Issuer, proof và cơ chế trạng thái	Cao nếu profile và proof truy cập được	Status list hoặc cơ chế tương đương	Khả năng tương tác giữa implementation
Blockcerts trên Bitcoin	Issuer Profile, Merkle proof và anchor	Cao đối với tính toàn vẹn và thời điểm neo	Danh sách thu hồi của issuer	Phụ thuộc profile ngoài chuỗi và quản lý khóa

Không có phương án nào loại bỏ hoàn toàn niềm tin vào cơ sở cấp. Blockchain thay đổi cách kiểm chứng bằng cách đưa bằng chứng tối thiểu lên một sổ cái khó sửa đổi và cho phép Holder mang theo chứng thư. Cơ sở cấp vẫn chịu trách nhiệm về tính đúng của dữ liệu ban đầu, danh tính issuer, quyền phê duyệt, bảo vệ khóa và quyết định thu hồi.

Đồ án lựa chọn Blockcerts V3 vì phù hợp mục tiêu chứng minh Merkle batching, xác minh công khai và mô hình Holder. PostgreSQL vẫn giữ vai trò quản lý nghiệp vụ và audit; Bitcoin chỉ lưu bằng chứng neo. Đây là kiến trúc kết hợp thay vì thay thế toàn bộ hệ thống quản lý đào tạo bằng blockchain.

1.8. Tổng kết Chương I

1.8.1. Kết quả lý luận chính

Chương I đã trình bày Blockchain như một kiến trúc sổ cái phân tán kết hợp cấu trúc liên kết băm, chữ ký số, mạng ngang hàng và đồng thuận. Mức bảo đảm của hệ thống phụ thuộc đồng thời vào giả định mật mã, số nút lỗi, quản trị khóa, chất lượng ứng dụng và dữ liệu ngoài chuỗi; vì vậy không thể đồng nhất Blockchain với tính bất biến hay an toàn tuyệt đối [1], [10]. Phân tích cũng cho

thấy việc lựa chọn mô hình công khai, riêng, cấp quyền hoặc không cấp quyền phải xuất phát từ ranh giới tin cậy và yêu cầu nghiệp vụ.

1.8.2. Cơ sở chuyển sang Chương II

Đối với quản lý văn bản, hướng tiếp cận phù hợp ở mức khái niệm là giữ hồ sơ cá nhân ngoài chuỗi và chỉ sử dụng sổ cái cho bằng chứng tối thiểu cùng lịch sử trạng thái. Chương II sẽ chuyển các nguyên tắc này thành yêu cầu chức năng, yêu cầu an toàn, mô hình dữ liệu và luồng nghiệp vụ; đồng thời so sánh các phương án trước khi lựa chọn nền tảng, cấu trúc mạng và cơ chế triển khai cụ thể.

CHƯƠNG II. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích bài toán

2.1.1. Bối cảnh nghiệp vụ

Quy trình văn bằng truyền thống có ba nhóm rủi ro chính. Thứ nhất, dữ liệu có thể bị nhập sai hoặc sửa trái phép trong hệ thống tập trung. Thứ hai, bên tuyên dụng thường phải liên hệ cơ sở đào tạo để xác minh, làm tăng thời gian và chi phí. Thứ ba, nếu một người có thể vừa lập vừa duyệt yêu cầu phát hành thì nguy cơ lạm quyền khó được kiểm soát. Blockchain chỉ hỗ trợ phát hiện thay đổi sau khi dữ liệu đã được chấp nhận; nó không tự bảo đảm dữ liệu đầu vào đúng [1], [10]. Vì vậy bài toán phải kết hợp kiểm soát quy trình, quản trị khóa và bằng chứng blockchain.

Hệ thống được thiết kế cho một cơ sở đào tạo phát hành chứng thư Blockcerts V3. Thông tin nghiệp vụ và tài khoản lưu trong PostgreSQL; chứng thư hoàn chỉnh lưu ngoài chuỗi; Bitcoin chỉ nhận Merkle root của lô hoặc dấu vết thu hồi qua OP_RETURN. Mô hình này hạn chế đưa dữ liệu cá nhân trực tiếp lên blockchain, đồng thời cho phép phát hiện chứng thư bị sửa [2], [3].

2.1.2. Phạm vi

Phạm vi đề án gồm đăng nhập, quản lý tài khoản, lập–duyet yêu cầu, phát hành đơn/lô tối đa 500 chứng thư, holder portal, xác minh công khai, thu hồi, audit và triển khai pilot. Bitcoin chạy regtest; đây là mạng phát triển cho phép tạo block chủ động và không dùng tiền thật [18]. Hệ thống không tuyên bố đã sẵn sàng cho văn bằng pháp lý trên mainnet, không thay thế nghiệp vụ xét tốt nghiệp và không xác thực danh tính ngoài các tài khoản do đơn vị tạo.

2.2. Tác nhân và yêu cầu chức năng

2.2.1. Tác nhân

Hệ thống có ba vai trò xác thực và một tác nhân công khai:

Maker: lập yêu cầu đơn hoặc lô; xem trạng thái và audit; không được duyệt hoặc thu hồi.

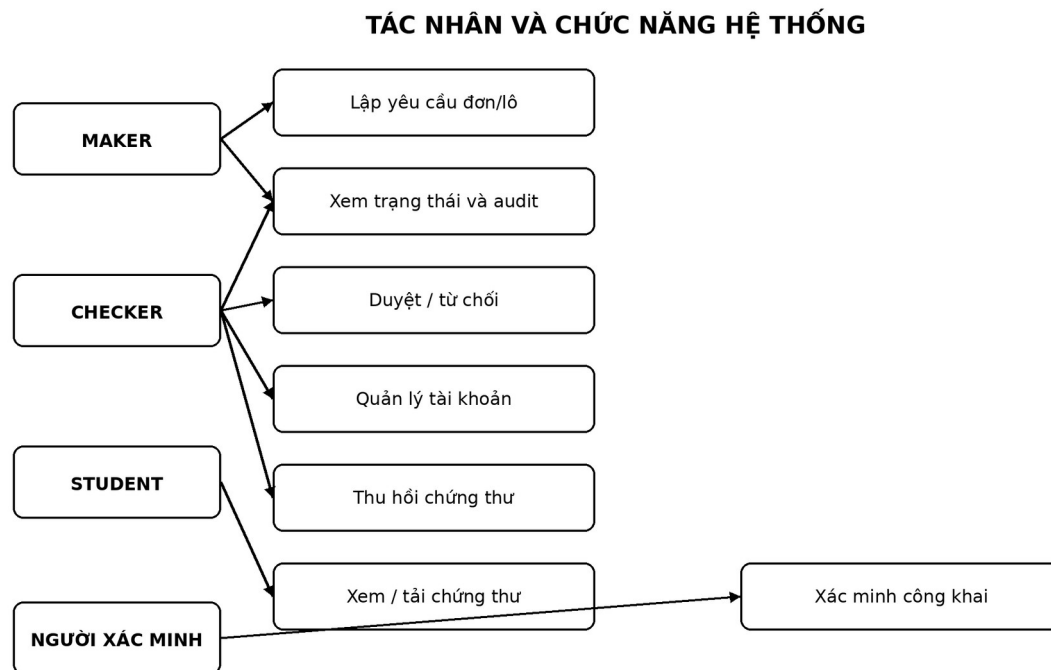
Checker: duyệt/từ chối yêu cầu, quản lý tài khoản, xem audit và thu hồi chứng thư đã phát hành.

Student: đăng nhập cổng Holder để xem/tải chứng thư được liên kết với mình; không được vào cổng quản trị.

Người xác minh: không cần đăng nhập; tải hoặc dán JSON chứng thư để nhận kết luận.

Không có vai trò Admin độc lập trong phiên bản mã nguồn được báo cáo. Tên “cổng Admin” chỉ mô tả cổng nghiệp vụ phía cơ sở đào tạo dành cho Maker và Checker, không phải một role thứ tư.

Hình 2.1 khái quát các tác nhân và nhóm chức năng tương ứng trong phạm vi hệ thống.



Hình 2.1. Tác nhân và nhóm chức năng của hệ thống

2.2.2. Yêu cầu chức năng

Các yêu cầu chức năng của hệ thống được tổng hợp trong Bảng 2.1.

Bảng 2.1. Danh sách yêu cầu chức năng

Mã	Yêu cầu	Tác nhân
FR-01	Đăng nhập và nhận JWT	Maker, Checker, Student
FR-02	Tạo/list tài khoản, không trả password hash	Checker
FR-03	Tạo một yêu cầu phát hành	Maker
FR-04	Tạo lô 1–500 yêu cầu	Maker
FR-05	Duyệt hoặc từ chối yêu cầu	Checker

Mã	Yêu cầu	Tác nhân
FR-06	Phát hành Blockcerts V3 và neo Bitcoin	Worker sau khi Checker duyệt
FR-07	Xem danh sách, chi tiết và trạng thái	Maker, Checker
FR-08	Xem/tải chứng thư của mình	Student
FR-09	Xác minh chứng thư JSON	Công khai
FR-10	Thu hồi chứng thư đã phát hành	Checker
FR-11	Công bố Issuer Profile và Revocation List	Công khai
FR-12	Ghi và đọc audit nghiệp vụ	Hệ thống; Maker/Checker đọc
FR-13	Kiểm tra sức khỏe dịch vụ	Công khai có giới hạn thông tin

2.2.3. Quy tắc nghiệp vụ

Yêu cầu mới có trạng thái pending_approval. Maker không được gọi API duyệt; Checker lấy danh tính từ JWT thay vì tin approvedBy do client gửi. Khi duyệt, yêu cầu chuyển queued và được đưa vào BullMQ. Worker chuyển lần lượt sang processing, issued hoặc failed. Checker chỉ thu hồi bản ghi issued; chứng thư thu hồi chuyển revoked và không bị xóa.

Phát hành theo lô yêu cầu tất cả bản ghi thuộc trạng thái cho phép. Mô hình tách requestBatchId (nhóm phiếu Maker tạo) khỏi issuanceBatchId (một lần Checker duyệt/phát hành). Vì vậy một nhóm yêu cầu có thể được duyệt từng phần thành nhiều lô phát hành mà không xung đột khóa; mỗi phiếu chỉ chuyển khỏi pending_approval một lần. Các chứng thư trong cùng issuanceBatchId dùng chung Merkle root và transaction id nhưng có certUid riêng. Cột txid không được đặt duy nhất vì Merkle batching chủ ý cho phép nhiều chứng thư cùng neo vào một transaction [3], [14].

2.3. Yêu cầu phi chức năng và mô hình đe dọa

2.3.1. Yêu cầu phi chức năng

An toàn: băm mật khẩu; JWT có thời hạn; RBAC ở backend; bí mật ngoài Git; giới hạn tần suất và kích thước yêu cầu.

Toàn vẹn: sửa payload, proof hoặc transaction id phải không còn kết luận VALID.

Truy vết: lưu người lập, người duyệt, người thu hồi, transaction và audit.

Hiệu năng: tối đa 500 chứng thư/lô trong phạm vi nguyên mẫu; một transaction neo cho mỗi lô.

Khả dụng: tách request HTTP khỏi tác vụ phát hành dài; có queue, health check và chính sách khởi động lại.

Tái lập: dùng Compose, migration, scripts, freeze và checksum artifact.

Riêng tư: không ghi toàn bộ hồ sơ cá nhân lên Bitcoin; không công bố credential hoặc khóa trong repository.

2.3.2. Tài sản và đối thủ

Tài sản gồm WIF phát hành, JWT secret, RPC/database credential, tài khoản, chứng thư, trạng thái phát hành–thu hồi, audit và cấu hình triển khai. Đối thủ có thể là người ngoài gửi chứng thư giả, tài khoản nội bộ vượt quyền, kẻ lấy được khóa, hoặc người vận hành sửa DB. Các bề mặt chính là API công khai, cổng đăng nhập, reverse proxy, queue, RPC Bitcoin, tệp workspace và chuỗi cung ứng phần mềm.

Các đe dọa chính và biện pháp kiểm soát tương ứng được trình bày trong Bảng 2.2.

Bảng 2.2. Đe dọa và kiểm soát

Đe dọa	Hậu quả	Kiểm soát thiết kế	Giới hạn còn lại
Maker tự duyệt	Phát hành không có kiểm soát chéo	RBAC, Checker-only approve	Tài khoản Checker bị chiếm vẫn nguy hiểm
Sửa chứng thư	Văn bằng giả	Hash, proof và anchor	Không phát hiện dữ liệu sai từ đầu vào hợp lệ

Đe dọa	Hậu quả	Kiểm soát thiết kế	Giới hạn còn lại
Chiếm mật khẩu	Mạo danh	bencrypt, JWT, throttling	Chưa có MFA
Lộ WIF	Neo bằng chứng giả	Bí mật ngoài Git, RPC cục bộ	Chưa dùng HSM/KMS
Sửa DB thu hồi	Kết luận trạng thái sai	OP_RETURN thu hồi, audit	Chưa có indexer tái tạo từ chuỗi
DoS Verify	Cạn tài nguyên	body limit, throttling, Nginx	Chưa thử tải tấn công dài hạn
Lộ dữ liệu runtime	Rò rỉ thông tin	workspace ngoài Git, ignore	Cần chính sách xóa/lưu giữ

OWASP khuyến nghị từ chối mặc định, kiểm tra quyền trên mọi yêu cầu và kiểm thử logic phân quyền [16]. Vì vậy frontend chỉ ẩn chức năng để cải thiện trải nghiệm; backend mới là lớp quyết định quyền.

2.4. Lựa chọn công nghệ và kiến trúc

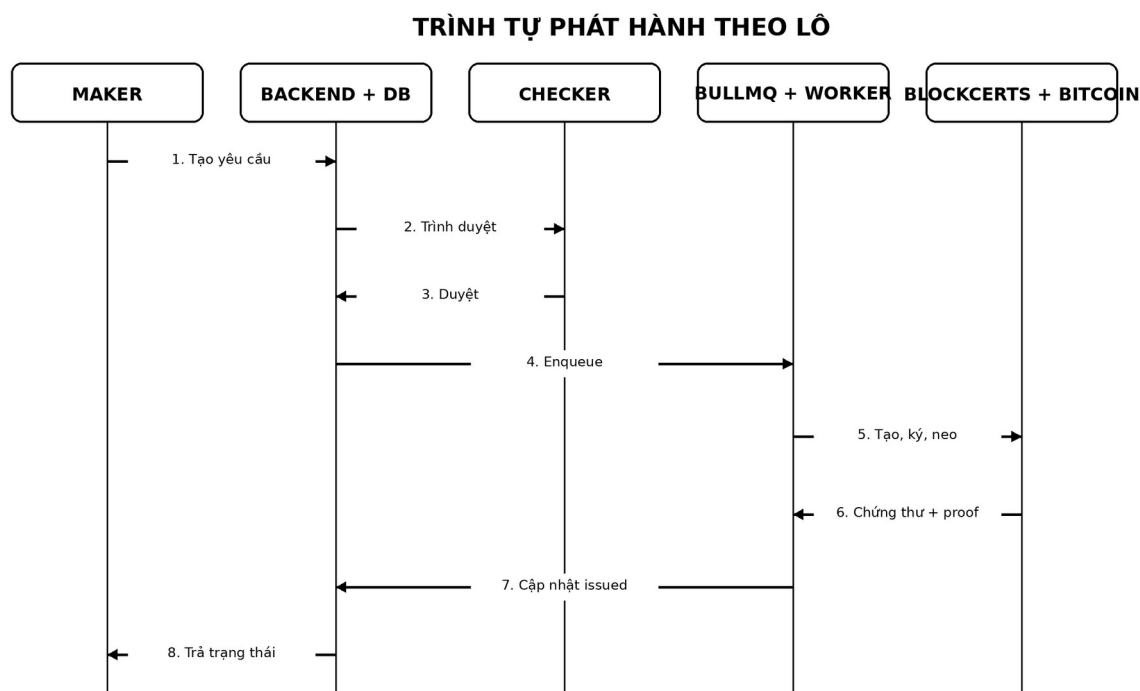
2.4.1. Lựa chọn mô hình Blockcerts–Bitcoin

Blockcerts biểu diễn chứng thư dưới dạng JSON-LD/Verifiable Credential và gắn bằng chứng Merkle. Mô hình VC phân biệt issuer, holder và verifier [9]; JSON-LD cung cấp ngữ nghĩa liên kết [12]. Schema và context Blockcerts được dùng làm nguồn đối chiếu cấu trúc [11]. Trong chứng thư thực tế của hệ thống, proof.type là DataIntegrityProof, cryptosuite là merkle-proof-2019; receipt nằm trong proofValue và được giải mã bằng bộ LDMerkleProof2019 [13], [14].

Bitcoin regtest được chọn thay vì hợp đồng thông minh. Giá trị on-chain chỉ là Merkle root hoặc chuỗi thu hồi được mã hóa trong OP_RETURN; toàn bộ workflow vẫn ở ứng dụng và PostgreSQL. Thiết kế này bám sát toolchain Blockcerts hiện có, giảm phạm vi dữ liệu công khai và cho phép một giao dịch đại diện nhiều chứng thư. Nhược điểm là cần dịch vụ ngoài chuỗi cho issuer profile, revocation list và quản lý nghiệp vụ.

2.4.2. Kiến trúc thành phần

Hình 2.2 thể hiện kiến trúc thành phần và ranh giới giữa các cổng giao diện, backend, hàng đợi, cơ sở dữ liệu và Bitcoin.



Hình 2.2. Kiến trúc thành phần của hệ thống

Hệ thống gồm ba frontend React; Nginx reverse proxy; backend NestJS; PostgreSQL; Redis/BullMQ; worker; container cert-tools; container cert-issuer; verifier service; Bitcoin Core regtest. Các thành phần dữ liệu và RPC chỉ bind loopback. Cổng quản trị, Holder và Verify được tách để giảm nhầm lẫn quyền và cho phép triển khai độc lập.

NestJS cung cấp module, guard và validation [19]. PostgreSQL cung cấp transaction và ràng buộc dữ liệu [20]. TypeORM quản lý entity/migration [22]. BullMQ dùng Redis để chuyển tác vụ dài khỏi request HTTP [21]. Việc chọn công nghệ không có nghĩa framework tự bảo đảm an toàn; cấu hình, kiểm thử và quy trình vận hành vẫn quyết định kết quả.

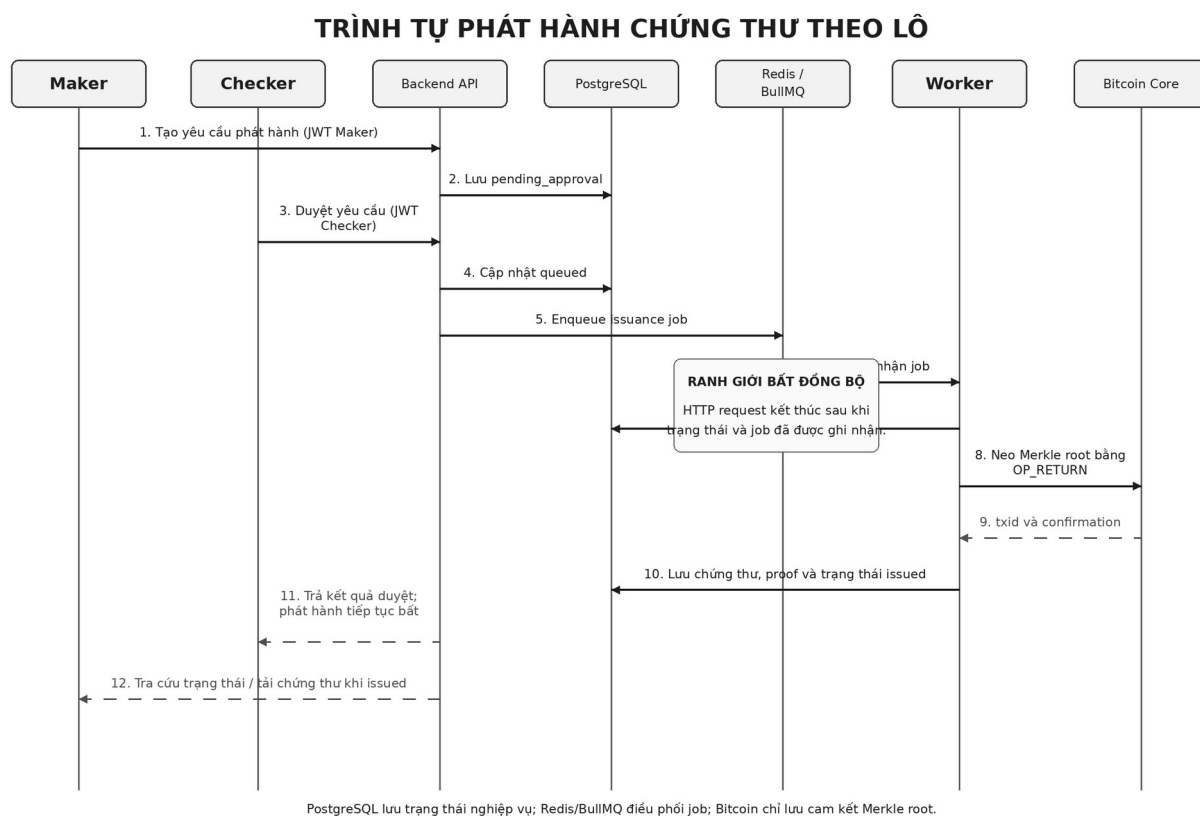
2.4.3. Luồng dữ liệu

Maker gửi thông tin người nhận và khóa công khai tới backend. Backend lưu yêu cầu; sau khi Checker duyệt, worker tạo roster và gọi toolchain phát hành. Chứng thư đã ký được lưu ngoài chuỗi; certUid, txid, merkleRoot, trạng thái và actor được lưu DB. Bitcoin chỉ nhận root. Student tải JSON qua Holder API. Verify portal gửi JSON tới backend, backend phối hợp verifier service, Bitcoin RPC và DB thu hồi để kết luận.

2.5. Thiết kế Maker–Checker và máy trạng thái

2.5.1. Luồng phát hành

Hình 2.3 mô tả trình tự phát hành theo lô từ lúc Maker tạo yêu cầu đến khi worker hoàn tất chứng thư.



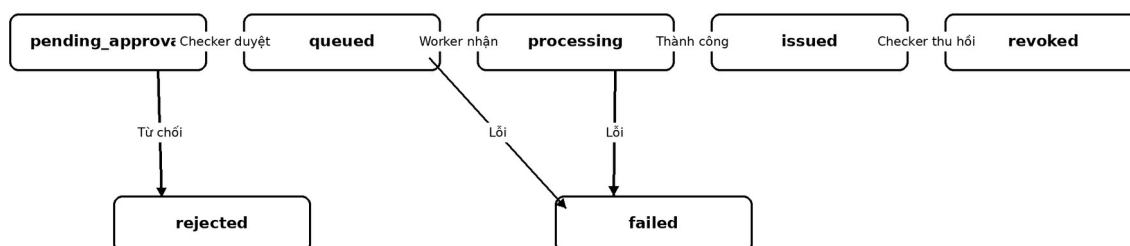
Hình 2.3. Trình tự phát hành chứng thư theo lô

Maker đăng nhập và gửi một yêu cầu hoặc mảng tối đa 500 phần tử. Backend lấy actor từ JWT, kiểm tra DTO và lưu pending_approval. Checker xem danh sách, duyệt hoặc từ chối. Khi duyệt, backend đặt queued và enqueue job. Worker đặt processing, tạo chứng thư unsigned theo các chunk riêng. Các output được gom để cert-issuer tạo một cây Merkle và một anchor. Regtest tạo một block; worker giải mã proof, ánh xạ bằng nonce và cập nhật issued. Holder tải chứng thư; bên thứ ba có thể Verify.

2.5.2. Trạng thái

Hình 2.4 biểu diễn các trạng thái và điều kiện chuyển trạng thái của yêu cầu phát hành.

MÁY TRẠNG THÁI YÊU CẦU PHÁT HÀNH



Hình 2.4. Máy trạng thái của yêu cầu phát hành

Luồng chính: pending_approval → queued → processing → issued → revoked. Hai nhánh thay thế là pending_approval → rejected và queued/processing → failed. Không cho chuyển ngược tự do, vì điều đó làm mất ý nghĩa audit. Yêu cầu thất bại chỉ được xử lý lại bằng quy trình vận hành có kiểm soát.

Ý nghĩa các trạng thái nghiệp vụ được tổng hợp trong Bảng 2.3.

Bảng 2.3. Ý nghĩa trạng thái

Trạng thái	Ý nghĩa	Tác nhân chuyển
pending_approval	Chờ Checker xem xét	Maker tạo
rejected	Bị từ chối, có lý do	Checker
queued	Đã duyệt và chờ worker	Checker/service
processing	Toolchain đang xử lý	Worker
issued	Có chứng thư và anchor	Worker
failed	Worker kết thúc lỗi	Worker
revoked	Chứng thư mất hiệu lực	Checker

2.6. Thiết kế phát hành Blockcerts V3

2.6.1. Cấu trúc chứng thư

Chứng thư gồm @context, mảng type chứa VerifiableCredential và BlockcertsCredential, URI issuer, issuanceDate, id, credentialSubject, nội dung hiển thị và proof. credentialSubject.id phải là URI định danh, không dùng địa chỉ Bitcoin trần. Khóa công khai của người nhận được cung cấp ở đầu vào phát hành; nonce ngẫu nhiên phục vụ ánh xạ nội bộ giữa roster và output.

DataIntegrityProof gồm thời điểm tạo, mục đích assertionMethod, verification method trỏ tới Issuer Profile, cryptosuite merkle-proof-2019 và proofValue. proofValue bao gói receipt Merkle, trong đó có target hash, proof và anchor Bitcoin. Cấu trúc được tạo bởi toolchain Blockcerts, không phải hợp đồng thông minh [11], [14].

2.6.2. Batching và ánh xạ kết quả

Phương án chạy toàn bộ toolchain cho từng hồ sơ gây chi phí khởi động container và tăng transaction. Thiết kế cuối chia bước tạo unsigned certificate thành chunk 10 hồ sơ, tối đa 16 chunk đồng thời trên máy thử nghiệm. Mỗi chunk có workspace riêng; sau đó toàn bộ output được gom vào một thư mục để cert-issuer tạo một root và một transaction.

Tên người nhận có thể trùng và tên file do tool tạo không nên dùng làm khóa. Mỗi dòng roster nhận một nonce ngẫu nhiên; worker đọc nonce trong output để tìm đúng bản ghi DB. batchId dùng cho truy vấn và thực nghiệm, còn certUid dùng định danh chứng thư. Nếu mở giới hạn trên 500, cần chia thành nhiều anchor job thay vì ngấm tạo một lô vượt khả năng đã kiểm thử.

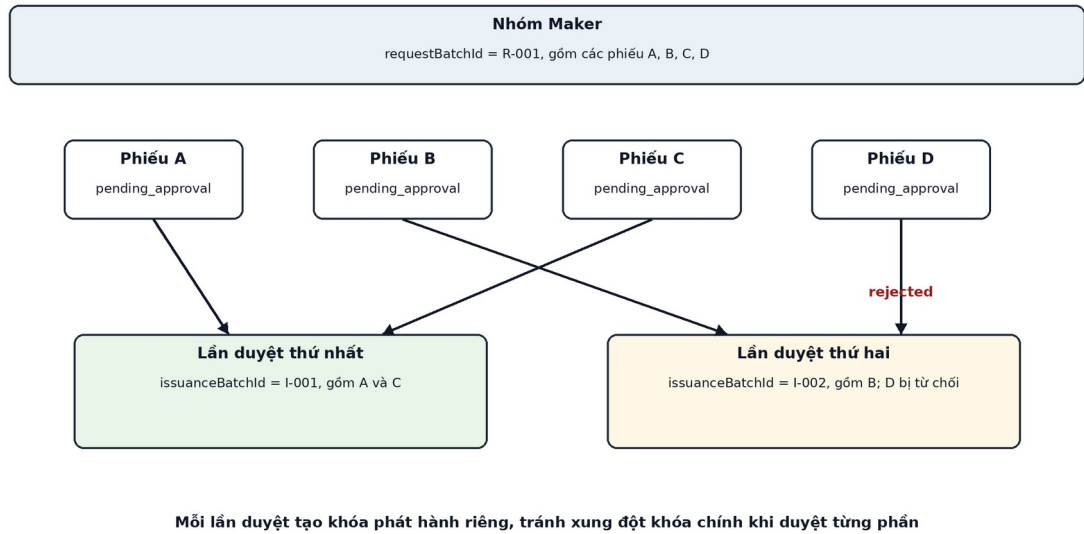
2.6.3. Tách nhóm yêu cầu và lô phát hành

Trong phiên bản đầu của nguyên mẫu, một định danh batch đồng thời được dùng để biểu diễn nhóm phiếu do Maker tạo và lần phát hành do Checker phê duyệt. Mô hình này chỉ phù hợp khi toàn bộ phiếu trong nhóm được duyệt một lần. Khi Checker chọn một phần, lần duyệt tiếp theo có thể tạo bản ghi lô trùng khóa chính. Thiết kế hiện hành tách hai khái niệm có vòng đời khác nhau thành requestBatchId và issuanceBatchId.

requestBatchId được sinh khi Maker gửi nhóm phiếu. Giá trị này ổn định trong toàn bộ vòng đời và cho phép truy ngược các phiếu có cùng nguồn nghiệp vụ. issuanceBatchId chỉ được sinh khi một tập phiếu cụ thể thực sự được duyệt. Mỗi lần duyệt từng phần tạo một giá trị mới. Vì vậy một requestBatchId có thể liên kết với nhiều issuanceBatchId, còn mỗi chứng thư sau khi phát hành chỉ thuộc một lô phát hành xác định.

Quan hệ một–nhiều giữa nhóm yêu cầu và các lần phát hành được thể hiện trên Hình 2.5.

QUAN HỆ GIỮA NHÓM YÊU CẦU VÀ LẦN PHÁT HÀNH



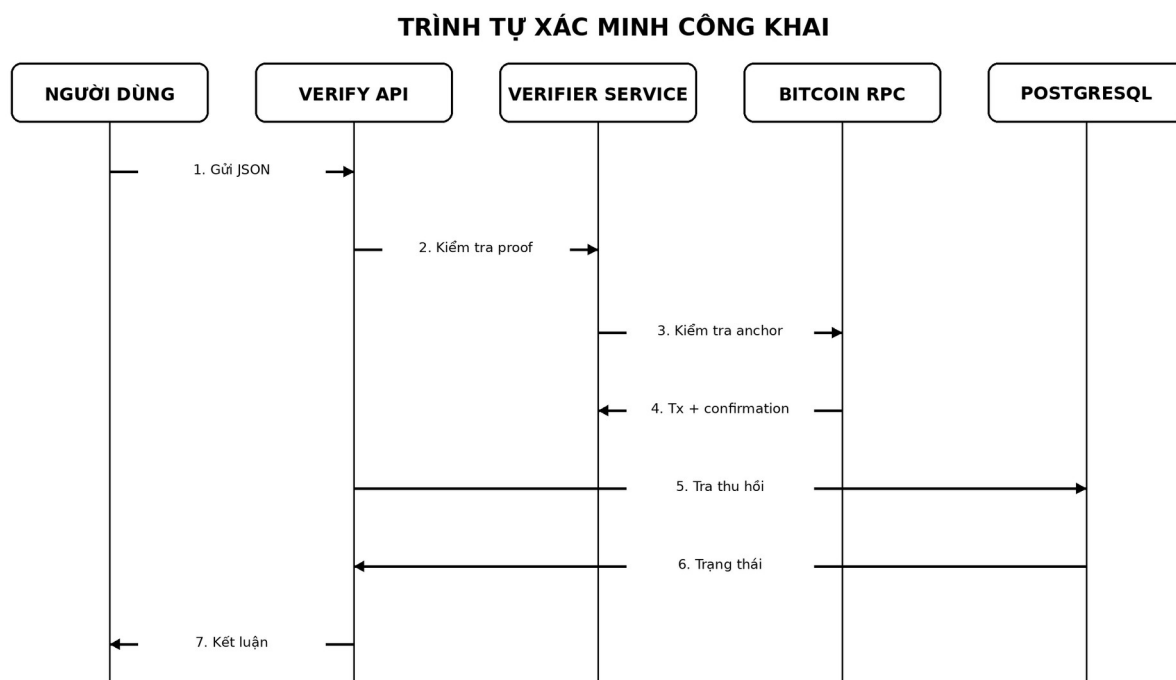
Hình 2.5. Quan hệ giữa nhóm yêu cầu và lần phát hành

Thiết kế trên giải quyết đồng thời ba yêu cầu. Thứ nhất, khóa chính của bảng lô phát hành không còn bị sử dụng lại. Thứ hai, audit có thể ghi nhận chính xác Checker nào đã duyệt tập phiếu nào ở từng thời điểm. Thứ ba, worker nhận đầu vào bất biến theo issuanceBatchId, nên lần chạy lại không vô tình kéo thêm phiếu vừa được duyệt ở thời điểm khác. Migration 1790300000000-SeparateRequestAndIssuanceBatch.ts chuyển dữ liệu cũ sang mô hình mới và là migration thứ mười trong chuỗi migration hiện hành.

2.7. Thiết kế xác minh và thu hồi

2.7.1. Xác minh

Hình 2.6 trình bày trình tự xác minh công khai và cách tổng hợp các kết quả kiểm tra.



Hình 2.6. Trình tự xác minh chứng thư công khai

Verify API tiếp nhận toàn bộ JSON. Backend kiểm tra semantic tối thiểu, sau đó gửi chứng thư đến verifier service. Verifier giải mã proofValue bằng LDMerkleProof2019, kiểm tra cấu trúc/chứng thư; adapter regtest lấy transaction theo txid, kiểm tra đầu vào gắn với issuer, tìm OP_RETURN, so Merkle root và yêu cầu ít nhất một confirmation. Nếu lớp mật mã/anchor đạt, backend lấy certUid để tra thu hồi. Kết luận gồm VALID, INVALID, REVOKED hoặc INDETERMINATE.

INVALID dùng khi nội dung không khớp bằng chứng; REVOKED dùng khi chứng thư vốn hợp lệ nhưng đã bị thu hồi; INDETERMINATE dùng khi không đủ bằng chứng để kết luận, ví dụ transaction/proof không thể kiểm tra. Phân biệt này tránh biến lỗi kết nối thành tuyên bố chứng thư giả.

2.7.2. Thu hồi

Checker chọn bản ghi issued và nhập lý do. Backend tạo payload REVOKE:<certUid>, mã hóa hex, tạo raw transaction có OP_RETURN, ký bằng khóa phát hành, broadcast và tạo một block regtest. Sau đó DB lưu revocationTxid, revokedBy, revokeReason, revokedAt và trạng thái revoked; Issuer Revocation List đọc dữ liệu này để công bố.

Thu hồi có hai dấu vết: transaction Bitcoin và chỉ mục DB/Revocation List. Phiên bản hiện tại kiểm tra nhanh từ DB sau khi đã kiểm tra chứng thư/anchor; nó

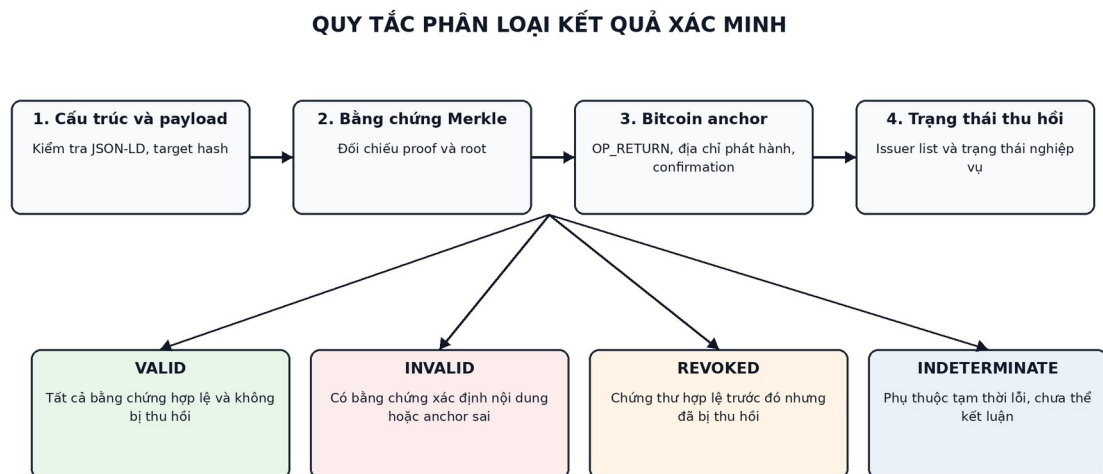
chưa quét blockchain để tự xây lại trạng thái. Vì vậy backup DB và một indexer phục hồi là yêu cầu cho bản sản xuất.

2.7.3. Quy tắc hợp nhất kết quả xác minh

Dịch vụ xác minh không chỉ trả một giá trị đúng hoặc sai. Mỗi nhóm kiểm tra có mức chắc chắn khác nhau và phải được hợp nhất theo quy tắc bảo toàn ý nghĩa. Sai cấu trúc chứng thư, target hash không khớp, Merkle proof sai hoặc anchor không chứa Merkle root là bằng chứng xác định để kết luận INVALID. Chứng thư có bằng chứng mật mã hợp lệ nhưng xuất hiện trong danh sách thu hồi được kết luận REVOKED.

Lỗi mạng, timeout, dịch vụ phụ thuộc tạm thời không phản hồi hoặc dữ liệu xác nhận chưa đủ không chứng minh chứng thư sai. Các trường hợp này được gán INDETERMINATE, hiển thị bằng cụm từ “Chưa thể kết luận” và cho phép người dùng thực hiện lại phép kiểm tra. Quy tắc này tránh tạo kết luận âm tính giả khi hạ tầng ngoài hệ thống gián đoạn.

Bốn nhánh kết quả và điều kiện tương ứng được khái quát trên Hình 2.7.



INDETERMINATE không được ánh xạ thành INVALID và giao diện cung cấp thao tác thử lại

Hình 2.7. Quy tắc phân loại kết quả xác minh

Quy tắc hợp nhất và cách thể hiện trên giao diện được tổng hợp trong Bảng 2.4.

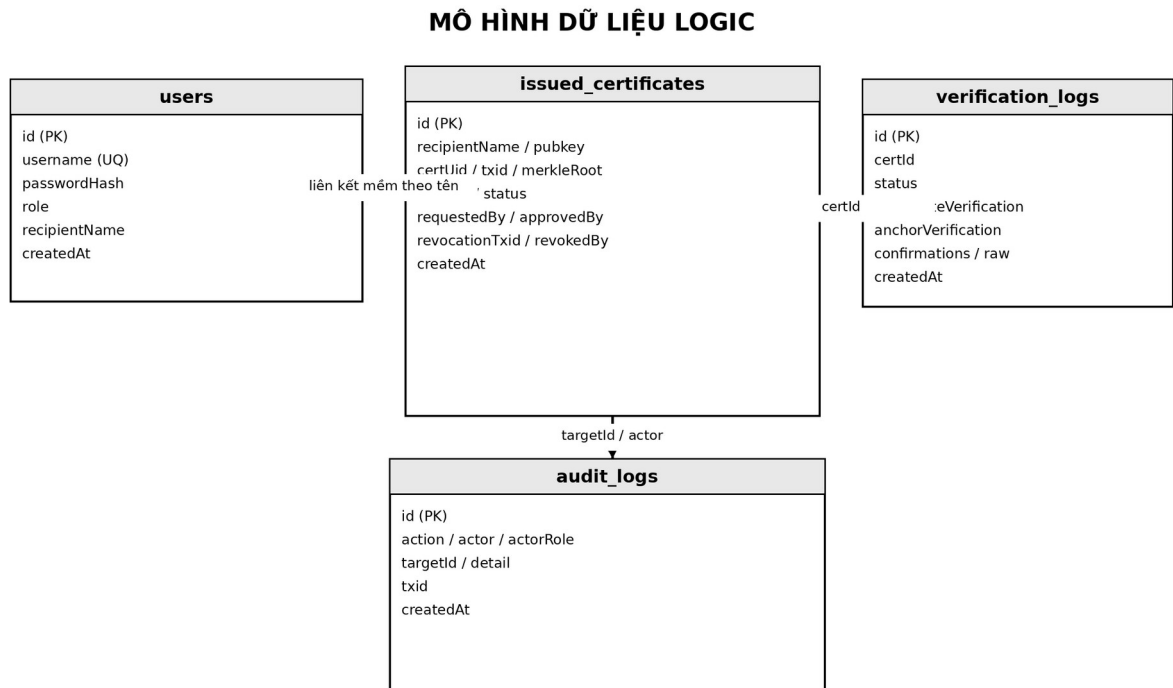
Bảng 2.4. Ma trận hợp nhất kết quả xác minh

Điều kiện quan sát	Kết quả	Ý nghĩa nghiệp vụ	Hành động giao diện
Payload, proof, anchor và trạng thái đều hợp lệ	VALID	Chứng thư nguyên vẹn và chưa bị thu hồi	Hiển thị kết quả hợp lệ
Có bằng chứng xác định dữ liệu hoặc anchor sai	INVALID	Chứng thư không vượt qua kiểm tra toàn vẹn	Nêu bước kiểm tra thất bại
Bằng chứng hợp lệ nhưng chứng thư đã bị thu hồi	REVOKED	Chứng thư không còn hiệu lực sử dụng	Hiển thị lý do và thời điểm thu hồi
Phụ thuộc tạm thời lỗi hoặc chưa đủ dữ liệu	INDETERMINATE	Chưa thể đưa ra kết luận chắc chắn	Hiển thị trạng thái riêng và nút thử lại

Việc phân loại được kiểm thử độc lập tại frontend Verify và dịch vụ xác minh. Sáu ca kiểm thử trạng thái bảo đảm nhân hiển thị không làm thay đổi ý nghĩa từ backend. Mười ca SSRF kiểm tra các biến thể địa chỉ nội bộ, hostname cục bộ, URL chuyển hướng và địa chỉ không được phép trước khi dịch vụ tải tài nguyên từ chứng thư.

2.8. Thiết kế dữ liệu ngoài chuỗi

Hình 2.8 khái quát các thực thể dữ liệu chính và quan hệ giữa chúng.



Hình 2.8. Mô hình dữ liệu của hệ thống

2.8.1. Bảng users

Bảng lưu id, username duy nhất, passwordHash, role, studentCode và recipientName tùy chọn cùng createdAt. studentCode là định danh nghiệp vụ duy nhất; chứng thư liên kết tới tài khoản Student bằng khóa ngoại studentId, còn recipientName chỉ là dữ liệu hiển thị, không dùng để quyết định quyền sở hữu.

2.8.2. Bảng issued_certificates

Mỗi dòng vừa là yêu cầu nghiệp vụ vừa là chứng thư sau phát hành. Nhóm đầu vào gồm dữ liệu văn bản, khóa ngoại studentId và requestBatchId; nhóm blockchain gồm certUid, txid, merkleRoot và issuanceBatchId; nhóm Maker–Checker gồm requestedBy, approvedBy, rejectedBy, rejectReason; nhóm trạng thái gồm status, errorMessage; nhóm thu hồi gồm revocationTxid, raw transaction/txid checkpoint, số lần thử, người thực hiện, lý do và thời điểm.

Thiết kế một bảng làm giảm số phép nối và phù hợp nguyên mẫu nhưng vẫn trộn dữ liệu yêu cầu với artifact phát hành. Hệ thống hiện đã liên kết Student bằng khóa ngoại và dùng studentCode duy nhất toàn hệ thống; khi mở rộng vẫn nên tách issuance_requests, batches, certificates và revocation_requests.

2.8.3. *verification_logs* và *audit_logs*

verification_logs lưu *certId*, trạng thái, kết quả verifier, kết quả adapter, confirmation, phản hồi thô và thời gian. *audit_logs* lưu action, actor, role, target, detail, txid và thời gian; mỗi dòng còn có *previousHash* và *entryHash* để tạo chuỗi hash SHA-256, cho phép phát hiện sửa/xóa ở giữa chuỗi qua endpoint kiểm tra toàn vẹn. Cơ chế này chỉ mang tính tamper-evident: DBA vẫn có thể viết lại toàn bộ chuỗi hoặc xóa phần đuôi nếu không đối chiếu head hash với kho log ngoài hệ thống. Vì vậy vẫn cần kiểm soát truy cập, backup và hệ thống log append-only độc lập khi triển khai production.

Các thực thể dữ liệu chính được khái quát trong Bảng 2.5.

Bảng 2.5. Mô hình dữ liệu chính

Bảng	Khóa/định danh	Chức năng
users	UUID, username unique	Tài khoản và vai trò
issued_certificates	UUID, certUid, requestBatchId, issuanceBatchId	Vòng đời yêu cầu/chứng thư
verification_logs	UUID, certId	Lịch sử xác minh
audit_logs	UUID, targetId	Nhật ký hành động

2.9. Thiết kế API và phân quyền

Các nhóm API và quyền truy cập tương ứng được tổng hợp trong Bảng 2.6.

Bảng 2.6. API chính và quyền

Phương thức	Endpoint	Quyền	Chức năng
POST	/api/auth/login	Công khai	Đăng nhập
POST	/api/auth/register	Checker	Tạo tài khoản
GET/POST	/api/admin/users	Checker	List/tạo tài khoản
GET	/api/audit	Maker, Checker	Xem audit mới nhất
POST	/api/issue/request	Maker	Lập một yêu cầu
POST	/api/issue/batch/request	Maker	Lập 1–500 yêu cầu

Phương thức	Endpoint	Quyền	Chức năng
POST	/api/issue/:id/ approve	Checker	Duyệt một yêu cầu
POST	/api/issue/batch/ approve	Checker	Duyệt theo lô
POST	/api/issue/:id/reject	Checker	Từ chối
GET	/api/issue, /api/issue/:id	Maker, Checker	Danh sách/chi tiết
GET	/api/issue/holder/ certificates	Student	Chứng thư của holder
POST	/api/revoke/:id	Checker	Thu hồi
POST	/api/revoke/check	Công khai	Kiểm tra nhanh thu hồi
POST	/api/verify	Công khai	Xác minh JSON
GET	/api/blockcerts/ issuers/kma/ profile.json	Công khai	Issuer Profile
GET	/api/blockcerts/ issuers/kma/ revocation-list.json	Công khai	Revocation List
GET	/health	Công khai	Health check

Route tĩnh batch/request, batch/approve, holder/certificates và check phải được khai báo trước route :id; nếu không router có thể hiểu chuỗi tĩnh là ID. ValidationPipe loại hoặc từ chối trường không được khai báo. Actor của approve/revoke lấy từ JWT [15], không lấy từ body.

2.10. Thiết kế ba cổng giao diện

Cổng quản trị phục vụ Maker và Checker. Maker có màn hình lập phiếu đơn/lô, danh sách và audit; Checker có màn hình duyệt, văn bằng, thu hồi, người dùng và audit. Menu thay đổi theo role nhưng backend vẫn kiểm tra quyền. Cổng Holder chỉ nhận tài khoản Student, hiển thị cả chứng thư còn hiệu lực và đã thu hồi

để giữ lịch sử. Công Verify không yêu cầu tài khoản, nhận file/dữ liệu JSON và hiển thị trạng thái cùng chi tiết kiểm tra.

Ba frontend dùng chung API nhưng build độc lập. Khi triển khai sau Nginx, mỗi ứng dụng dùng base path riêng. Giao diện không tự tính hợp lệ ở client; nó gửi chứng thư tới backend để logic nhất quán và tránh phụ thuộc khả năng trình duyệt.

2.11. Thiết kế an toàn và triển khai

Mật khẩu được băm bcrypt. OWASP yêu cầu work factor đủ lớn và ưu tiên Argon2id cho hệ thống mới; bcrypt vẫn có thể dùng cho hệ thống cũ khi cấu hình phù hợp [17]. JWT được ký và có thời hạn theo mô hình RFC 7519 [15]. AuthGuard chạy trước RolesGuard trên route bảo vệ để req.user tồn tại trước khi xét role.

Helmet thêm security header; throttling giới hạn yêu cầu; CORS dùng allowlist; body có giới hạn; Swagger có thể cấu hình. PostgreSQL, Redis và Bitcoin RPC chỉ bind loopback. Nginx là điểm vào, backend/worker/Nginx được quản lý bởi systemd user service, còn các container có restart policy. .env, WIF, RPC credential, tài khoản thực nghiệm và workspace bị loại khỏi Git.

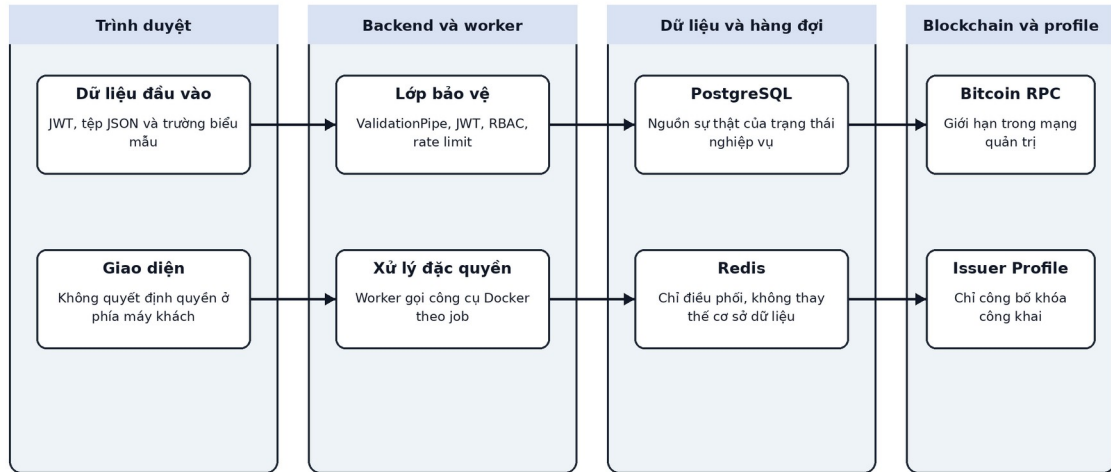
Thiết kế pilot dùng HTTP bên trong mạng riêng. HTTPS công khai, HSTS, secret manager, HSM/KMS, luân chuyển khóa, backup/restore, HA, giám sát tập trung và kiểm thử xâm nhập nằm ngoài phạm vi phiên bản này, nhưng là điều kiện trước production.

2.11.1. Ranh giới tin cậy

Hệ thống được chia thành bốn vùng tin cậy. Trình duyệt là vùng không tin cậy, vì người dùng có thể sửa request, lưu trữ cục bộ hoặc mã giao diện. Backend và worker là vùng xử lý đặc quyền, nhưng vẫn phải xác thực mọi đầu vào và không được dựa vào điều kiện ẩn trên giao diện. PostgreSQL là nguồn sự thật của trạng thái nghiệp vụ; Redis chỉ cung cấp cơ chế điều phối hàng đợi. Bitcoin RPC và kho cấu hình Blockcerts thuộc vùng quản trị, không được công khai trực tiếp ra Internet.

Các vùng tin cậy và điểm kiểm soát tại biên được mô tả trên Hình 2.9.

RANH GIỚI TIN CẬY VÀ KIỂM SOÁT AN TOÀN



Credential thật không xuất hiện trong frontend, Git, báo cáo hoặc artifact công khai

Hình 2.9. Ranh giới tin cậy và kiểm soát an toàn

Tại biên trình duyệt–backend, ValidationPipe, DTO và giới hạn kích thước request kiểm soát dữ liệu đầu vào. JWT xác định danh tính, còn RolesGuard quyết định quyền dựa trên vai trò tại server. Tại biên backend–worker, job chỉ mang định danh cần thiết; worker đọc lại dữ liệu từ PostgreSQL thay vì tin hoàn toàn vào payload trong Redis. Tại biên worker–Bitcoin, RPC credential chỉ tồn tại trong cấu hình runtime và không xuất hiện trong chứng thư, frontend hoặc artifact công khai.

2.11.2. Quản lý bí mật và dữ liệu nhạy cảm

Nhóm bí mật gồm JWT secret, mật khẩu cơ sở dữ liệu, Redis password, Bitcoin RPC password, WIF phát hành và token dùng cho các lượt kiểm thử. Các giá trị này phải được sinh riêng theo môi trường, truyền qua biến môi trường hoặc tệp runtime bị loại khỏi Git. Tệp .env.example chỉ mô tả tên biến và giá trị minh họa không có khả năng sử dụng như credential thật.

Báo cáo, log và bộ evidence chỉ giữ định danh kỹ thuật cần cho tái lập. Bất kỳ giá trị nhạy cảm nào cần minh họa đều được thay bằng [REDACTED]. Chứng thư công khai chỉ chứa dữ liệu cần cho mục đích xác minh và không chứa khóa riêng. Issuer Profile công bố khóa công khai, định danh đơn vị cấp và endpoint trạng thái; WIF không thuộc cấu trúc profile.

2.11.3. Kiểm soát SSRF trong dịch vụ xác minh

Chứng thư Blockcerts có thể tham chiếu tài nguyên ngoài như Issuer Profile hoặc danh sách thu hồi. Nếu backend tải trực tiếp mọi URL do chứng thư cung cấp, kẻ tấn công có thể lợi dụng máy chủ làm điểm truy cập mạng nội bộ. Thiết kế

do đó phân tích URL trước khi gửi request, chỉ chấp nhận giao thức được hỗ trợ, từ chối thông tin xác thực nhúng trong URL và chặn các miền hoặc dải địa chỉ không được phép.

Kiểm soát phải được áp dụng lại sau mỗi lần chuyển hướng, vì một URL công khai có thể trả redirect về địa chỉ nội bộ. Phân giải DNS và kết nối mạng được ràng buộc bởi timeout. Kết quả do phụ thuộc mạng thất bại được phân loại là INDETERMINATE, không được dùng để kết luận chứng thư INVALID.

2.11.4. Bảo vệ trạng thái và chống cạnh tranh

Các thao tác thay đổi trạng thái dùng điều kiện ngay trong câu lệnh cập nhật thay vì chỉ đọc trạng thái rồi ghi lại. Chẳng hạn, thao tác từ chối chỉ cập nhật bản ghi đang ở pending_approval; nếu số hàng bị ảnh hưởng bằng không, dịch vụ dừng và báo xung đột. Cách làm này ngăn hai request duyệt và từ chối cùng thắng trong khoảng thời gian giữa bước đọc và bước ghi.

Với thu hồi, biến revocationStarted phân biệt lỗi tiền điều kiện xảy ra trước khi quy trình bắt đầu với lỗi phát sinh sau khi đã có tác động bên ngoài. Lỗi kiểm tra quyền, trạng thái hoặc dữ liệu không được đổi chứng thư sang trạng thái lỗi và không được gọi Bitcoin RPC. Sau thời điểm broadcast, checkpoint được lưu để lần phục hồi sử dụng lại transaction ID, tránh phát giao dịch thứ hai.

2.12. Tiêu chí chấp nhận và kế hoạch kiểm thử

Các tiêu chí chấp nhận của nguyên mẫu được liệt kê trong Bảng 2.7.

Bảng 2.7. Tiêu chí chấp nhận

Mã	Tiêu chí	Bảng chứng dự kiến
AC-01	Chứng thư gốc VALID	Kịch bản 1
AC-02	Sửa nội dung/proof/txid không còn VALID	Kịch bản 1
AC-03	Maker tự duyệt bị 403	Kịch bản 1
AC-04	10, 100, 500 chứng thư/lô đều phát hành đủ	Kịch bản 2
AC-05	Mỗi lô dùng đúng một transaction	Artifact audit
AC-06	Student/Maker không thu hồi được	Kịch bản 3

Mã	Tiêu chí	Bảng chứng dự kiến
AC-07	Checker thu hồi và Verify trả REVOKED	Kịch bản 3
AC-08	Revocation List và audit nhất quán	Kịch bản 3
AC-09	Build, test, dependency audit và health đạt	B12 preflight/postflight

2.12.1. Ma trận truy vết

Mỗi yêu cầu được nối với bốn lớp bằng chứng gồm thiết kế, thành phần hiện thực, phép kiểm thử và artifact. Cách tổ chức này làm giảm nguy cơ báo cáo chỉ mô tả tính năng mà không chứng minh tính năng đã chạy. Một hàng chỉ được đánh dấu đạt khi tồn tại kết quả thực thi phù hợp với phạm vi của yêu cầu.

Mỗi liên hệ giữa yêu cầu, thành phần và bằng chứng được trình bày trong Bảng 2.8.

Bảng 2.8. Ma trận truy vết thiết kế và kiểm thử

Nhóm yêu cầu	Thành phần hiện thực	Kiểm thử chính	Bằng chứng
Xác thực và RBAC	auth, guards và decorator vai trò	Backend Jest, functional E2E	Log test và response API
Maker–Checker	issuance.service.ts và cập nhật có điều kiện	Ca duyệt, từ chối và cạnh tranh	Trạng thái DB và audit log
Duyệt từng phần	requestBatchId, issuanceBatchId và migration 10	Functional E2E nhiều lần duyệt	Hai lô phát hành không trùng khóa
Phát hành Blockcerts	Worker, cert-tools và cert-issuer	E2E, benchmark	Chứng thư JSON và transaction ID
Phục hồi sau lỗi	Checkpoint của worker	Recovery E2E	Giữ nguyên transaction ID và một block

Nhóm yêu cầu	Thành phần hiện thực	Kiểm thử chính	Bằng chứng
Xác minh công khai	Verify API và frontend Verify	16 ca kiểm thử verifier	10 ca SSRF và 6 ca trạng thái
Thu hồi	Revocation service và danh sách thu hồi	Backend Jest và E2E	Trạng thái REVOKED, audit và txid
Quyền Holder	Endpoint theo subject JWT	Functional E2E	Student chỉ thấy chứng thư của mình
Tái lập kết quả	Script, manifest và SHA-256	Kiểm toán artifact	35 payload, 37 checksum và 1.830 chứng thư

2.12.2. Nguyên tắc diễn giải bằng chứng

Unit test chứng minh hành vi của đơn vị mã được cô lập theo các dependency mô phỏng; kết quả này không được gọi là kiểm thử toàn hệ thống. Functional E2E chứng minh một chuỗi API và hạ tầng phối hợp trong môi trường tái lập. Recovery E2E tập trung vào điểm lỗi sau broadcast. Benchmark đo thời gian trong cấu hình regtest cụ thể. Full verifier kiểm tra độc lập toàn bộ 1.830 chứng thư đã phát hành.

Các lớp bằng chứng bổ sung cho nhau nhưng không thay thế nhau. Việc build thành công không chứng minh nghiệp vụ đúng. Unit test đạt không chứng minh Bitcoin RPC hoạt động. E2E đạt trên regtest không chứng minh đặc tính của mainnet. Báo cáo giữ nguyên ranh giới này trong phần kết luận để tránh suy rộng quá mức.

2.13. Kết luận Chương 2

Chương 2 đã cụ thể hóa cơ sở lý thuyết thành kiến trúc ba công, RBAC, quy trình Maker–Checker, Merkle batching, xác minh và thu hồi trên Bitcoin regtest. Thiết kế đặt dữ liệu cá nhân ngoài chuỗi và xác định rõ phạm vi đánh giá cho Chương 3.

CHƯƠNG III. XÂY DỰNG, KIỂM THỬ VÀ ĐÁNH GIÁ

3.1. Môi trường và công nghệ triển khai

3.1.1. Môi trường thực nghiệm

Đợt tái kiểm chứng ngày 30/09/2026 chạy trên Ubuntu Server 22.04.5 LTS, 4 vCPU Intel Xeon Gold 6130 và 19 GiB RAM. Stack được dựng tách biệt trên Bitcoin regtest, PostgreSQL và Redis riêng. Mã nguồn được nhận diện bằng commit 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1, trạng thái sourceDirty=false và fingerprint cây nguồn 6e0cd862daf5641b26995575d8e7ff9d755daf43396191dfc838fc6ffb0997cc; các số liệu dưới đây lấy từ artifact của đúng lần chạy này. Regtest cho phép chủ động tạo block mà không dùng tiền thật [18].

Các thành phần phần mềm của môi trường thực nghiệm được trình bày trong Bảng 3.1.

Bảng 3.1. Thành phần phần mềm

Thành phần	Phiên bản/dòng phiên bản	Vai trò
Node.js	22	Runtime backend/frontend build
NestJS	11	REST API, JWT, RBAC và nghiệp vụ
PostgreSQL	17-bookworm	Dữ liệu nghiệp vụ và log
Redis	7.4-bookworm	Nền lưu trữ BullMQ
BullMQ	6 (^6.3.1)	Hàng đợi phát hành
Bitcoin Core	31.1	Sổ cái regtest và RPC
cert-tools	3.0.2	Sinh chứng thư unsigned từ roster
cert-issuer	3.13.1	Tạo Merkle proof và neo Bitcoin
React/Vite	19/7	Ba giao diện web

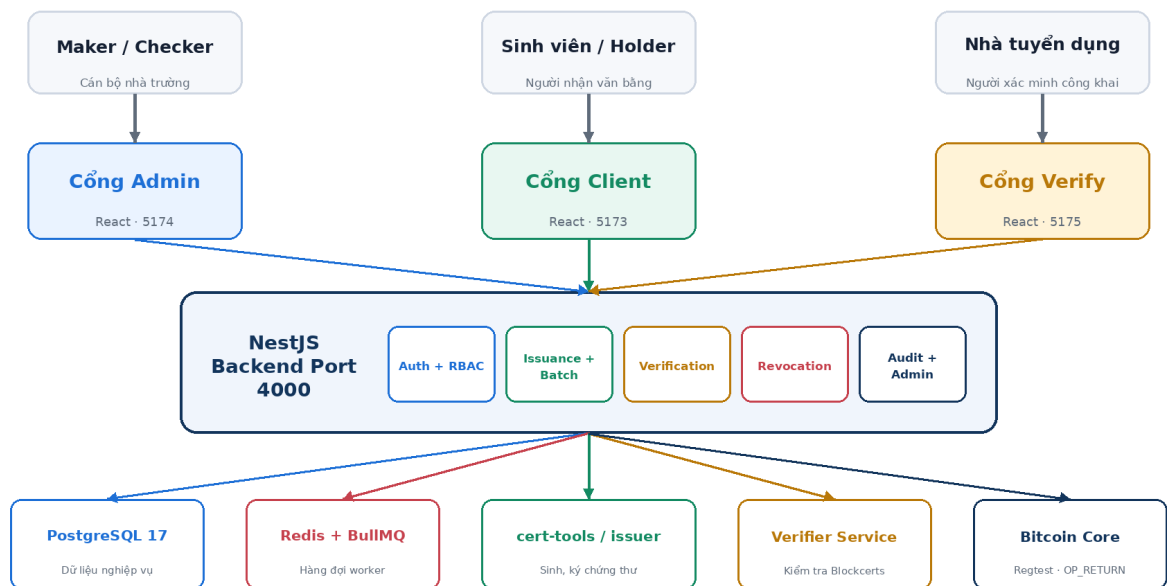
Thành phần	Phiên bản/dòng phiên bản	Vai trò
Nginx	Bản hệ thống	Reverse proxy và base path

Repository cung cấp Compose, migration, .env.example, script triển khai/health check và script thực nghiệm. PostgreSQL, Redis và Bitcoin RPC chỉ bind loopback. WIF, mật khẩu, JWT secret, RPC credential và tài khoản thử nghiệm nằm ngoài Git.

Hình 3.1 thể hiện kiến trúc triển khai của môi trường thực nghiệm.

KIẾN TRÚC HỆ THỐNG HIỆN TẠI

Ba cổng web độc lập, backend mô-đun và hạ tầng Blockcerts



Hình 3.1. Kiến trúc triển khai môi trường thực nghiệm

3.1.2. Tổ chức triển khai

PostgreSQL, Redis và Bitcoin Core chạy bằng Docker với restart policy. Backend, worker và Nginx được quản lý bằng systemd user service. Nginx phục vụ ba base path và chuyển tiếp API. Health check kiểm tra backend, DB, Redis và Bitcoin tùy mức ready. Kiến trúc này phù hợp pilot một máy; chưa phải kiến trúc HA nhiều node.

3.2. Hiện thực backend và cơ sở dữ liệu

3.2.1. Các module

Backend gồm AuthModule, AdminModule, IssuanceModule, VerificationModule, RevocationModule, AuditModule, IssuerModule và health controller. Controller gắn AuthGuard rồi RolesGuard tại route cần bảo vệ; service thực thi nghiệp vụ; entity mô tả dữ liệu; processor nhận job BullMQ. NestJS được chọn vì hỗ trợ module, dependency injection, guard và validation rõ ràng [19].

AuthModule đăng nhập, so khớp bcrypt, ký JWT và cung cấp một hàm đăng ký dùng chung. Cả endpoint đăng ký được bảo vệ bằng JWT/RBAC (chỉ Checker) và endpoint quản trị do Checker sử dụng đều gọi cùng hàm này, nên việc chuẩn hóa studentCode, kiểm tra trùng và lưu đầy đủ trường Student không còn bị tách thành hai luồng. AdminModule vẫn list/tạo tài khoản nhưng không trả passwordHash. IssuanceModule quản lý phiếu và worker. VerificationModule điều phối verifier service và lưu log. RevocationModule tạo transaction thu hồi. IssuerModule công bố profile cùng revocation list.

3.2.2. Migration và transaction

TypeORM chạy synchronize=false; schema thay đổi qua migration [22]. Bốn bảng chính là users, issued_certificates, verification_logs và audit_logs. PostgreSQL transaction bảo đảm nhóm thay đổi DB được commit hoặc rollback như một đơn vị [20]. txid không có unique constraint vì nhiều chứng thư trong một batch dùng chung transaction.

Bảng issued_certificates lưu cả yêu cầu và kết quả. Cách này làm workflow đơn giản nhưng tạo hạn chế đã nêu ở Chương 2. Migration thứ 10 sao chép định danh nhóm cũ sang requestBatchId, đổi cột phát hành thành issuanceBatchId, để trống định danh phát hành cho phiếu chưa duyệt và tạo chỉ mục riêng cho hai mục đích. Worker cập nhật DB theo chunk 100 bản ghi nhằm tránh một câu lệnh quá lớn.

3.2.3. Audit và verification log

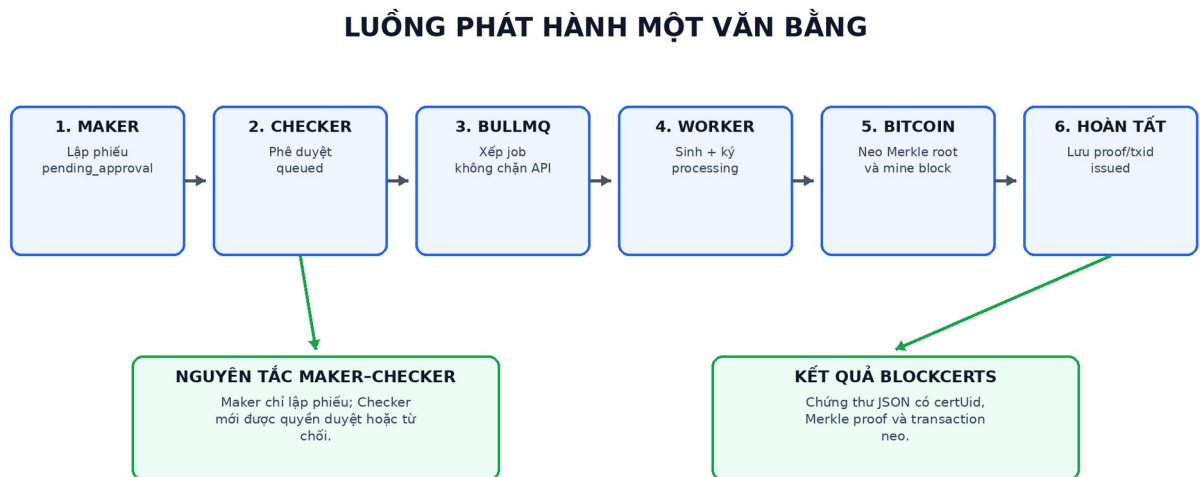
Audit được gắn vào đăng nhập, đăng ký, lập phiếu, duyệt, từ chối, phát hành, thu hồi và xác minh. Nếu riêng thao tác ghi audit lỗi, service không làm thất bại nghiệp vụ chính; lựa chọn này ưu tiên tính sẵn sàng nhưng cần cơ chế cảnh báo để không bỏ sót log âm thầm. Verification log lưu kết quả chi tiết từ verifier, adapter Bitcoin, confirmation và trạng thái cuối.

3.3. Hiện thực quy trình phát hành

3.3.1. Phát hành đơn và Maker–Checker

Maker gọi /api/issue/request. Backend lấy username từ JWT, không tin requestedBy trong body, và lưu bản ghi pending_approval. Checker gọi approve/reject. Yêu cầu được duyệt chuyển queued; BullMQ tách công việc nặng khỏi request HTTP [21]. Worker chuyển processing, tạo roster, gọi toolchain, tạo block và cập nhật issued hoặc failed.

Hình 3.2 mô tả luồng xử lý khi phát hành một văn bằng.



Hình 3.2. Luồng phát hành một văn bằng

3.3.2. Tối ưu phát hành theo lô

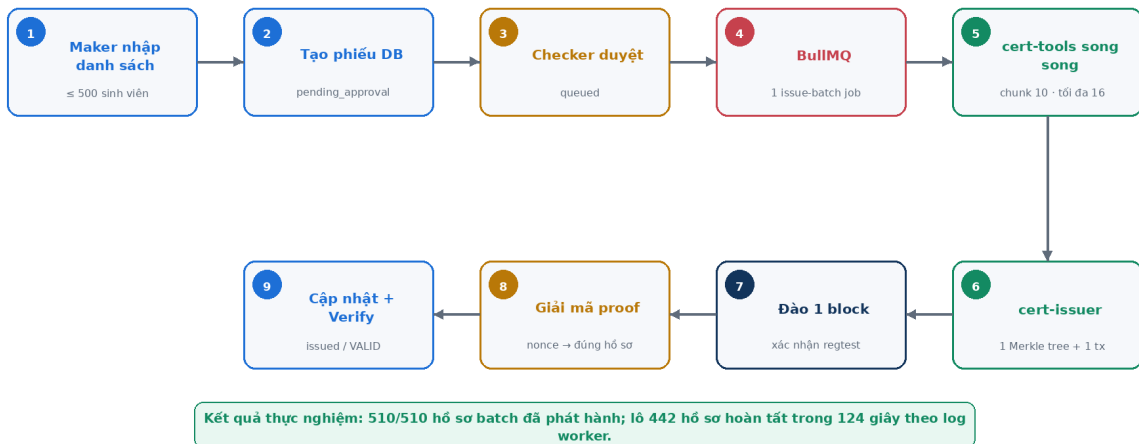
Phương án tuần tự ban đầu lập cert-tools → cert-issuer → mine cho từng hồ sơ, làm tăng chi phí khởi động container và số transaction. Một thử nghiệm roster 462 dòng chạy quá 5 phút mà chưa có output nên được hủy có kiểm soát, queue pause và bản ghi reset. Kết quả này không được dùng trong benchmark chính thức nhưng giúp xác định nút thắt.

Thiết kế cuối chia roster thành chunk 10 hồ sơ; tối đa 16 chunk tạo unsigned certificate song song, mỗi chunk dùng workspace riêng. Sau khi hoàn tất, worker gom output, đặt batch_size bằng số hồ sơ và gọi cert-issuer một lần. Tool tạo một Merkle tree và một transaction cho toàn batch. Mỗi output mang nonce để ánh xạ đúng bản ghi, không phụ thuộc tên hoặc thứ tự file.

Hình 3.3 mô tả luồng tối ưu phát hành nhiều chứng thư trong cùng một lô.

LUỒNG CẤP VẪN BẰNG THEO LÔ

Tối đa 500 hồ sơ — chia nhỏ bước sinh JSON, gom một Merkle anchor



Hình 3.3. Luồng phát hành chứng thư theo lô

Workspace có roster, unsigned, blockchain certificate, issuer work và manifest. Dữ liệu runtime được giữ cục bộ và bị loại khỏi Git. Sau khi giải mã receipt, worker lưu certUid, txid, Merkle root và issuanceBatchId; requestBatchId vẫn giữ quan hệ với nhóm phiếu ban đầu. Thao tác DB được chia nhỏ. Mỗi job hiện tối đa 500 hồ sơ, nên mục tiêu là một transaction mỗi job.

3.3.3. Cấu trúc chứng thư đầu ra

Chứng thư thực tế dùng context VC v1, Data Integrity v2 và Blockcerts v3.1; type gồm VerifiableCredential và BlockcertsCredential. Proof có type=DataIntegrityProof, cryptosuite=merkle-proof-2019, proofPurpose=assertionMethod, thời gian, verification method và proofValue. Cấu trúc được đối chiếu với VC, JSON-LD và Blockcerts [9], [11]–[14].

Nhiều chứng thư cùng txid là hành vi đúng của Merkle batching. Mỗi chứng thư có certificate ID, certUid, target hash và đường dẫn Merkle riêng; root chung mới được ghi trong OP_RETURN. SHA-256 là hàm băm nền theo FIPS 180-4 [4].

3.4. Hiện thực xác minh và thu hồi

3.4.1. Xác minh công khai

Verify portal gửi JSON đến POST /api/verify. Backend từ chối semantic không phù hợp, chuyển dữ liệu tới verifier service, nhận kết quả kiểm tra credential và adapter regtest. Adapter giải mã proofValue, lấy anchor, gọi RPC để đọc transaction, kiểm tra địa chỉ phát hành, OP_RETURN, Merkle root và ít nhất một confirmation. Sau đó backend tra thu hồi và lưu verification log/audit.

Kết quả VALID chỉ xuất hiện khi bằng chứng chứng thư và anchor đạt, đồng thời không bị thu hồi. INVALID phản ánh sai lệch được xác định; REVOKED phản ánh chứng thư hợp lệ nhưng mất hiệu lực; INDETERMINATE phản ánh không đủ dữ kiện để kết luận. Phân loại này quan trọng vì lỗi proof/tx không truy cập được không nên tự động bị diễn giải giống sửa payload.

3.4.2. Thu hồi

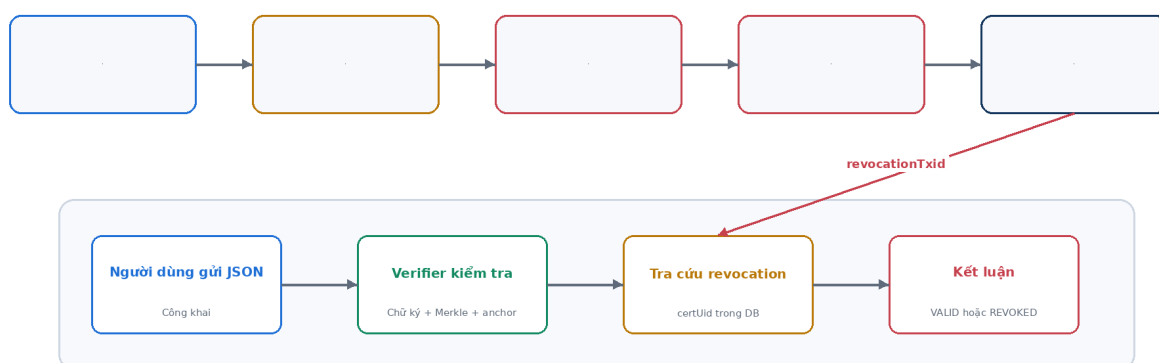
Chỉ Checker gọi được POST /api/revoke/:id. Service tạo REVOKE:<certUid>, mã hóa hex, tạo raw transaction với OP_RETURN, ký bằng signrawtransactionwithkey, broadcast và tạo một block. Việc ký trực tiếp bằng khóa được dùng vì ví regtest là watch-only. Khóa không nằm trong mã nguồn.

Sau khi giao dịch được xác nhận, PostgreSQL cập nhật revoked, transaction, actor, lý do và thời gian. Revocation List đọc các bản ghi này. Verify trước hết kiểm tra credential/anchor, sau đó trả REVOKED nếu DB ghi nhận thu hồi. Hình 3.4 mô tả luồng hai lớp.

Hình 3.4 thể hiện mối liên hệ giữa quy trình thu hồi và kết quả xác minh.

LUỒNG THU HỒI VÀ XÁC MINH

Bảng chứng OP_RETURN trên regtest kết hợp trạng thái nghiệp vụ trong PostgreSQL



Hình 3.4. Luồng thu hồi và xác minh chứng thư

3.5. Hiện thực ba frontend

Cổng quản trị hiển thị dashboard, lập phiếu đơn/lô, duyệt nhiều phiếu, danh sách văn bằng, thu hồi, người dùng và audit. Menu được lọc theo Maker/Checker. Cổng Holder dùng tài khoản Student, lấy danh sách theo khóa ngoại studentId từ JWT thay vì so tên, cho xem/tải JSON và văn hiển thị bản ghi revoked. Cổng Verify không yêu cầu đăng nhập; INDETERMINATE được hiển thị riêng là “Chưa thể kết luận”, không dùng màu/nhãn của INVALID.

Một lỗi route đã được sửa bằng cách đặt holder/certificates, batch/request, batch/approve và revoke/check trước route tham số :id. Một lỗi khác xảy ra khi RolesGuard toàn cục chạy trước JWT strategy; giải pháp là gắn AuthGuard('jwt') và RolesGuard cục bộ đúng thứ tự. Các lỗi này được giữ trong nhật ký phát triển để giải thích quyết định thiết kế.

3.6. Hardening B12

Mật khẩu được băm bằng bcrypt với cost 10, đáp ứng mức tối thiểu cho bcrypt mà OWASP nêu; đối với hệ thống mới, Argon2id vẫn là lựa chọn được ưu tiên [17]. JWT tuân theo cấu trúc RFC 7519 [15]. RBAC từ chối Student ở cổng quản trị và kiểm tra quyền tại backend; actor được lấy từ JWT. B12 bổ sung Helmet, throttling, CORS allowlist, giới hạn kích thước body, cấu hình Swagger và Nginx. Nguyên tắc kiểm quyền trên mọi request phù hợp với hướng dẫn OWASP [16]. Verifier đã giới hạn một số URL và chặn localhost/127.0.0.1; tuy nhiên, đối chiếu mã nguồn với hướng dẫn phòng chống SSRF cho thấy biện pháp hiện tại chưa bao phủ đầy đủ các dải địa chỉ nội bộ IPv4/IPv6, địa chỉ metadata và DNS rebinding [23]. Vì vậy, đây mới là mức giảm thiểu ban đầu, không phải bằng chứng SSRF đã được xử lý triệt để.

Các freeze R1/R2 cũ được giữ để truy vết lịch sử. Đợt ngày 30/09/2026 dựng lại stack cách ly, chạy đủ 10 migration và tạo bộ artifact mới; báo cáo này dùng bộ tái kiểm chứng mới thay cho việc suy diễn từ freeze cũ.

3.7. Phương pháp kiểm thử

3.7.1. Mục tiêu và nguyên tắc

Kiểm thử tập trung vào ba nhóm khẳng định: khả năng phát hiện chứng thư bị sửa và chặn Maker tự duyệt; khả năng Merkle batching phát hành đủ các lô 10, 100 và 500 chứng thư với một transaction cho mỗi lô; tính đúng quyền cùng sự nhất quán của trạng thái thu hồi tại Verify, Revocation List và audit.

Bộ dữ liệu là dữ liệu giả lập, không dùng hồ sơ cá nhân thật. Mỗi kích thước hiệu năng lặp ba lần. Thời gian đo từ lúc Checker duyệt đến khi toàn bộ bản ghi issued. Artifact tái kiểm chứng gồm JSON thô, manifest theo lô, kết quả E2E/kiểm toán và SHA-256 checksum. Những lần chạy lỗi trong quá trình sửa script không được đưa vào chín run đạt; mã lỗi vẫn được mô tả trong lịch sử phát triển thay vì tính vào thống kê.

Thời gian mỗi chứng thư và throughput được tính:

$$T_{\text{cert}} = T_{\text{batch}} / N$$

$$\text{Throughput} = N / T_{\text{batch}}$$

Trong đó T_{batch} là thời gian từ duyệt đến phát hành hoàn tất, N là kích thước batch. Độ lệch chuẩn báo cáo là độ lệch chuẩn mẫu của ba lần lặp.

3.7.2. Kiểm thử tự động và chất lượng mã

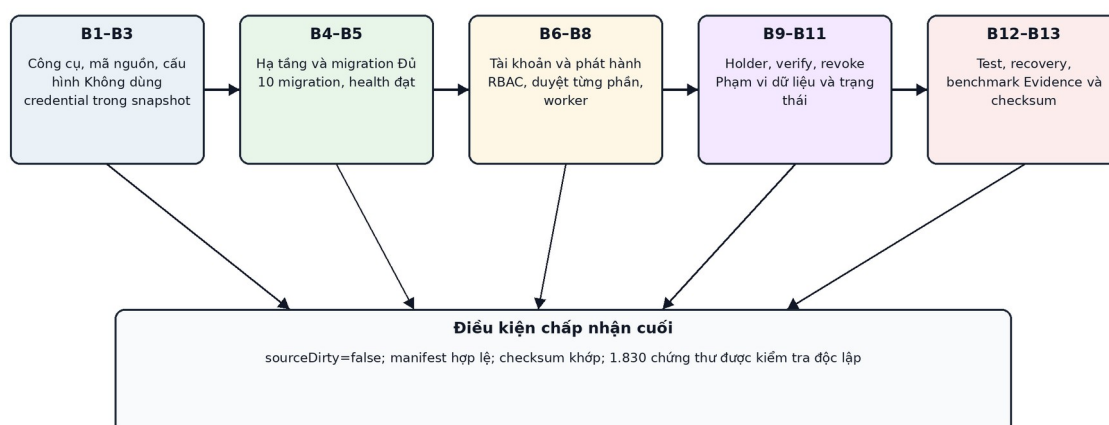
Đợt tái kiểm chứng có 12/12 backend test suite với 49/49 test đạt, gồm test hồi quy tiền điều kiện thu hồi, cạnh tranh duyệt/từ chối, duyệt từng phần, hợp nhất tạo Student, RBAC, DTO, outbox/checkpoint/recovery, audit, verifier và issuer profile. Dịch vụ verifier đạt 16/16 ca, gồm 10 ca SSRF và 6 ca phân loại trạng thái. Cả ba frontend build thành công. Kết quả này không đồng nghĩa hệ thống không có lỗi hỏng logic hoặc cấu hình. E2E trên stack regtest còn xác nhận hai lần duyệt từng phần của cùng một requestBatchId tạo hai issuanceBatchId khác nhau, luồng phát hành–xác minh–thu hồi đạt và phục hồi sau lỗi ngay sau broadcast giữ nguyên txid, chỉ tăng một block.

3.8. Quy trình triển khai và tái lập chi tiết

Quy trình được chia thành mười ba bước có thứ tự. Mỗi bước xác định đầu vào, thao tác, điểm kiểm soát và bằng chứng đầu ra. Người thực hiện không chuyển sang bước tiếp theo khi điều kiện chấp nhận của bước hiện tại chưa đạt. Mọi lệnh được chạy từ cây mã nguồn đã checkout đúng commit, còn giá trị bí mật chỉ tồn tại trong tệp runtime ngoài Git.

Quan hệ phụ thuộc giữa các bước và cổng kiểm soát được thể hiện trên Hình 3.5.

CÁC CÔNG KIỂM SOÁT QUY TRÌNH TÁI LẬP



Hình 3.5. Các công kiểm soát quy trình tái lập

3.8.1. B1 – Kiểm tra điều kiện tiên quyết

Bước đầu xác nhận máy thực nghiệm có Git, Node.js, npm, Docker Engine và Docker Compose. Docker daemon phải hoạt động, tài khoản chạy thử phải có quyền tạo container và dung lượng đĩa phải đủ cho image, volume PostgreSQL, Bitcoin regtest và artifact. Việc ghi phiên bản công cụ vào evidence giúp giải thích sai khác khi tái lập ở thời điểm khác.

```

cd /duong-dan/toi/datn-blockcerts
git --version
node --version
npm --version
docker --version
docker compose version
docker info
  
```

Điều kiện hoàn thành bước kiểm tra công cụ được nêu trong Bảng 3.2.

Bảng 3.2. Điều kiện chấp nhận B1

Hạng mục	Điều kiện đạt	Bảng chứng lưu
Git	Đọc được phiên bản và repository	git --version, git rev-parse HEAD
Node.js và npm	Chạy được script của dự án	Phiên bản trong log preflight
Docker	Daemon phản hồi	docker info có mã thoát bằng không

Hạng mục	Điều kiện đạt	Bằng chứng lưu
Compose	Nhận được tệp cấu hình	docker compose config thành công
Tài nguyên	Đủ CPU, RAM và đĩa cho thử nghiệm	Snapshot môi trường

Nếu Docker daemon chưa chạy, quy trình dừng trước khi tạo bất kỳ container nào. Nếu cổng dự kiến đã bị chiếm, phải xác định tiến trình hoặc stack cũ, sau đó giải phóng bằng quy trình dừng của bộ tái lập. Không thay đổi cổng một cách tùy ý vì URL issuer, CORS và endpoint public có liên hệ với cấu hình.

3.8.2. B2 – Cố định mốc mã nguồn

Repository được chuyển tới commit dùng cho thực nghiệm. Sau checkout, git status --short phải rỗng đối với mã nguồn nghiệp vụ. Script tạo source manifest tính fingerprint trên danh sách tệp được kiểm soát, không đưa artifact sinh ra sau chạy vào dấu vân tay nguồn.

```
git checkout 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1
git status --short
git rev-parse HEAD
python3 experiments/repro/scripts/source-manifest.py
```

Mốc được dùng trong lượt kiểm chứng cuối là 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1, trạng thái sourceDirty=false và fingerprint

6e0cd862daf5641b26995575d8e7ff9d755daf43396191dfc838fc6ffb0997cc. Ba giá trị phải xuất hiện cùng nhau trong manifest. Chỉ ghi commit mà không ghi trạng thái sạch là chưa đủ, vì file chưa commit có thể thay đổi hành vi.

3.8.3. B3 – Chuẩn bị cấu hình an toàn

Tệp mẫu được sao chép thành cấu hình runtime. Mật khẩu và secret phải được thay bằng giá trị mới, không dùng thông tin cá nhân thật. WIF phát hành được sinh trong mạng regtest và chỉ cấp cho worker. Issuer Profile chỉ nhận khóa công khai tương ứng.

```
cd experiments/repro
cp .env.repro.example .env.repro
chmod 600 .env.repro
```

Các nhóm biến và quy tắc bảo vệ được tổng hợp trong Bảng 3.3.

Bảng 3.3. Nhóm biến cấu hình runtime

Nhóm	Ví dụ tên biến	Quy tắc
PostgreSQL	POSTGRES_PASSWORD, DATABASE_URL	Giá trị riêng cho lượt chạy
Redis	REDIS_PASSWORD, REDIS_URL	Không công bố trong artifact
JWT	JWT_ACCESS_SECRET, JWT_REFRESH_SECRET	Độ dài đủ lớn và khác nhau
Bitcoin RPC	BITCOIN_RPC_USER, BITCOIN_RPC_PASSWORD	Chỉ dùng trong mạng Docker
Blockcerts	issuer URL, revocation URL, WIF	URL phải nhất quán; WIF giữ bí mật
Frontend	API base URL	Không chứa credential

Preflight quét cấu hình để phát hiện placeholder chưa thay, tệp bí mật bị Git theo dõi và giá trị có khả năng rò rỉ. Trong báo cáo, mọi ví dụ credential đều biểu diễn bằng [REDACTED].

3.8.4. B4 – Dụng hạ tầng phụ thuộc

PostgreSQL, Redis và Bitcoin Core regtest được dựng trong stack riêng. Tên project Compose tách biệt với môi trường phát triển để thao tác dừng hoặc xóa volume không ảnh hưởng hệ thống khác. Sau khi container chạy, health check kiểm tra khả năng đáp ứng chứ không chỉ kiểm tra trạng thái tiến trình.

```
cd experiments/repro
./scripts/prepare.sh
./scripts/start.sh
./scripts/health.sh
```

prepare.sh xây dựng image cần thiết, tạo cấu hình runtime, chạy migration và chuẩn bị công cụ Blockcerts. start.sh khởi động backend cùng Nginx của bộ tái lập. health.sh kiểm tra endpoint backend, PostgreSQL, Redis và Bitcoin RPC. Lượt kiểm chứng chỉ tiếp tục khi tất cả thành phần trả kết quả đạt.

3.8.5. B5 – Migration và build

Backend chạy đầy đủ mười migration theo thứ tự. Migration cuối tách requestBatchId và issuanceBatchId; vì vậy việc chỉ chờ chín migration có thể làm schema không tương thích với mã hiện hành. Script health kiểm tra số migration đã áp dụng và dừng nếu thiếu.

```
cd backend
npm ci
npm run migration:run
npm run build
cd ../frontend-admin && npm ci && npm run build
cd ../frontend-client && npm ci && npm run build
cd ../frontend-verify && npm ci && npm run build
```

Sản phẩm build và điều kiện chấp nhận được liệt kê trong Bảng 3.4.

Bảng 3.4. Sản phẩm build cần đạt

Thành phần	Sản phẩm	Điều kiện đạt
Backend	JavaScript biên dịch trong dist	TypeScript không lỗi
Admin	Bundle Maker và Checker	Build hoàn tất
Client	Bundle Student	Build hoàn tất
Verify	Bundle xác minh công khai	Build hoàn tất
Migration	Schema PostgreSQL	Đủ 10 migration

Build của bốn thành phần đã đạt trong lượt tái kiểm chứng. Kết quả này chỉ chứng minh mã có thể biên dịch và đóng gói, không thay thế test nghiệp vụ.

3.8.6. B6 – Khởi tạo tài khoản và kiểm tra RBAC

Tài khoản bootstrap Checker được tạo bằng script runtime, sau đó Checker gọi endpoint được bảo vệ để tạo Maker và Student. /api/auth/register không phải chức năng đăng ký công khai; request phải có JWT hợp lệ và vai trò Checker. Đường tạo Student sử dụng AuthService.register() thống nhất để các trường recipientName, studentCode, dateOfBirth, email và cohort được lưu đầy đủ.

```
cd experiments/repro
node scripts/prepare-accounts.mjs
```

Functional E2E đăng nhập bằng các tài khoản thử nghiệm, gọi endpoint /api/auth/me và thử các hành động được phép hoặc bị cấm. Student không được truy cập API quản trị. Maker không được tự duyệt phiếu của mình. Checker có thể

duyet, từ chối, quản lý tài khoản và thu hồi theo thiết kế. Credential bootstrap không được đưa vào báo cáo hoặc evidence công khai.

3.8.7. B7 – Tạo nhóm yêu cầu phát hành

Maker lập nhiều phiếu trong một request, mỗi phiếu chứa dữ liệu người nhận và thông tin văn bằng. Backend xác thực DTO, kiểm tra quan hệ Student và tạo requestBatchId. Trạng thái ban đầu của từng phiếu là pending_approval. Audit ghi actor, hành động và đối tượng mà không ghi token hay mật khẩu.

Các đầu ra cần đối chiếu sau khi tạo nhóm phiếu được nêu trong Bảng 3.5.

Bảng 3.5. Kiểm tra đầu ra B7

Kiểm tra	Kết quả mong đợi
Các phiếu cùng request	Có cùng requestBatchId
Trạng thái ban đầu	pending_approval
Lô phát hành	Chưa có issuanceBatchId trước khi duyệt
Audit	Ghi Maker và số lượng phiếu
Dữ liệu Student	Khớp subject đã chọn

Một phiếu không hợp lệ phải bị từ chối trước khi vào hàng đợi. Việc chỉ dựa vào validation phía frontend không được chấp nhận vì request có thể được tạo trực tiếp qua API.

3.8.8. B8 – Duyệt từng phần và phát hành

Checker chọn một tập con đang ở pending_approval. Dịch vụ cập nhật có điều kiện và tạo issuanceBatchId mới cho tập được duyệt. Các phiếu còn lại giữ requestBatchId cũ và có thể được duyệt hoặc từ chối ở lượt sau. Cơ chế này cho phép một nhóm Maker tạo sinh nhiều lô phát hành mà không vi phạm khóa chính.

Worker nhận job theo issuanceBatchId, tạo payload chứng thư, tính target hash và dựng cây Merkle. Merkle root được neo bằng một transaction Bitcoin regtest. Sau khi có anchor, worker gắn Merkle proof và receipt vào từng chứng thư, lưu tệp đầu ra và cập nhật cơ sở dữ liệu.

Các checkpoint bắt buộc của worker được mô tả trong Bảng 3.6.

Bảng 3.6. Checkpoint của worker phát hành

Checkpoint	Dữ liệu tối thiểu	Ý nghĩa phục hồi
Chuẩn bị payload	Danh sách certificate ID	Tái tạo đầu vào xác định
Tạo Merkle tree	Merkle root và proof	Phát hiện thay đổi dữ liệu
Trước broadcast	Raw transaction	Có thể gửi giao dịch đã chuẩn bị
Sau broadcast	Transaction ID	Không tạo giao dịch mới khi chạy lại
Hoàn tất	Vị trí chứng thư và trạng thái DB	Đối chiếu kết quả cuối

3.8.9. B9 – Kiểm tra cổng Holder

Student đăng nhập cổng Client và chỉ nhận chứng thư có recipient liên kết với subject JWT của mình. Backend không chấp nhận studentId tùy ý từ query làm nguồn quyết định phạm vi. Functional E2E dùng ít nhất hai Student để chứng minh tài khoản thứ nhất không đọc được chứng thư của tài khoản thứ hai.

Tập chứng thư tải xuống phải trùng với artifact được lưu cho bản ghi phát hành. Các trường hiển thị gồm người nhận, tên văn bằng, ngày cấp, đơn vị cấp, trạng thái và định danh chứng thư. Giao diện không hiển thị credential, WIF, raw transaction nội bộ hoặc dữ liệu audit không liên quan.

3.8.10. B10 – Xác minh công khai

Cổng Verify cho phép tải tập chứng thư mà không cần tài khoản. Backend hoặc verifier thực hiện tuần tự kiểm tra cấu trúc, target hash, Merkle proof, anchor, Issuer Profile và danh sách thu hồi. Kết quả trả về gồm trạng thái tổng hợp và chi tiết các bước để người dùng phân biệt lỗi nội dung với lỗi phụ thuộc tạm thời.

```
cd frontend-verify
npm test -- --runInBand
npm run build
```

Bộ test verifier có 16 ca, gồm 10 ca SSRF và 6 ca phân loại trạng thái. Trạng thái INDETERMINATE được hiển thị là “Chưa thể kết luận” và có thao tác thử lại riêng. Không có nhánh giao diện nào đổi trạng thái này thành INVALID.

3.8.11. B11 – Thu hồi và kiểm tra sau thu hồi

Checker gửi yêu cầu thu hồi một chứng thư đã phát hành và cung cấp lý do. Dịch vụ kiểm tra quyền, trạng thái hiện tại và dữ liệu anchor trước khi gọi Bitcoin

RPC. Nếu lỗi xảy ra ở giai đoạn tiền điều kiện, chứng thư giữ nguyên trạng thái và không có tác động ngoài hệ thống.

Sau khi broadcast, transaction ID và checkpoint được lưu. Khi job được phục hồi, worker sử dụng transaction hiện có thay vì tạo transaction thứ hai. Danh sách thu hồi và trạng thái trong PostgreSQL được cập nhật nhất quán. Phép xác minh tiếp theo trả REVOKED và giữ riêng các kết quả INVALID hoặc INDETERMINATE.

3.8.12. B12 – Kiểm thử chức năng, an toàn và phục hồi

Bộ tái lập gom lệnh kiểm thử vào script có mã thoát rõ ràng. Backend Jest chạy tuần tự để log dễ đối chiếu. Functional E2E kiểm tra chuỗi đăng nhập, Maker–Checker, duyệt từng phần, Holder và thu hồi. Recovery E2E tạo điểm lỗi sau broadcast rồi khởi động lại worker.

```
cd experiments/repro
./scripts/run-tests.sh
node scripts/functional-e2e.mjs
node scripts/recovery-e2e.mjs
```

Kết quả chốt gồm 12 trên 12 suite backend với 49 trên 49 test đạt, 16 trên 16 test verifier đạt, functional E2E đạt và recovery E2E đạt. Ca phục hồi giữ nguyên transaction ID và số block chỉ tăng một lần. Kết quả unit test, frontend test và E2E được báo cáo theo đúng phạm vi, không gộp thành một con số kiểm thử toàn hệ thống.

3.8.13. B13 – Benchmark và đóng băng evidence

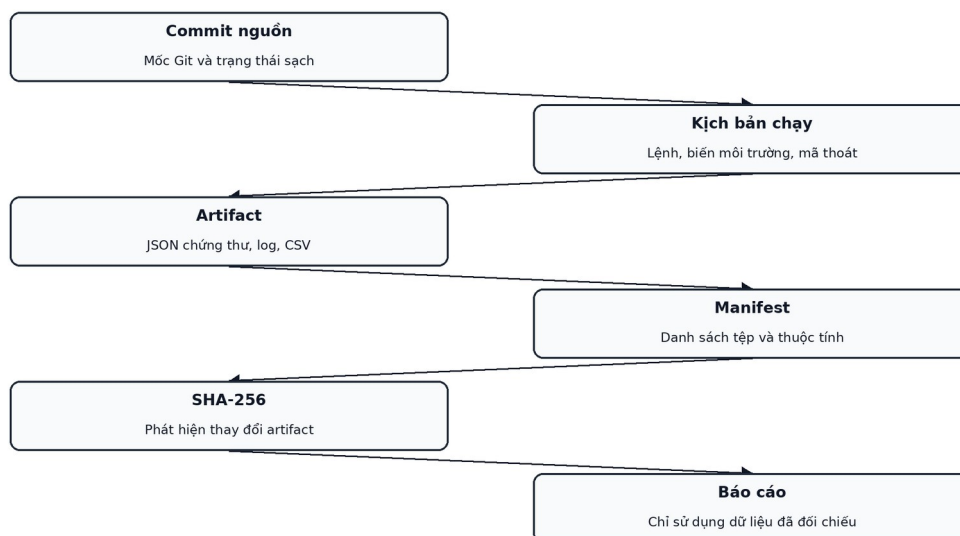
Benchmark chạy ba kích thước lô 10, 100 và 500; mỗi kích thước lặp ba lần. Trước mỗi run, script tạo dữ liệu thử nghiệm độc lập, ghi thời gian bắt đầu–kết thúc, batch ID, transaction ID, Merkle root và vị trí artifact. Tổng cộng chín run phát hành 1.830 chứng thư trên chín transaction.

```
cd experiments/repro
node scripts/benchmark.mjs
node scripts/audit-benchmark-artifacts.mjs
node scripts/full-verifier-batch.cjs
python3 scripts/freeze-evidence.py
```

Kiểm toán sau chạy không chỉ đếm file. Script kiểm tra certificate ID, target hash, Merkle proof, quan hệ batch–transaction và manifest. Bộ full verifier đọc toàn bộ chứng thư bằng cert-verifier-js. Sau khi kiểm tra đạt, freeze-evidence.py tạo manifest và SHA-256 để phát hiện thay đổi về sau.

Chuỗi truy vết từ commit nguồn tới kết luận báo cáo được thể hiện trên Hình 3.6.

CHUỖI BẢNG CHỨNG TỪ MÃ NGUỒN ĐẾN KẾT LUẬN



Một kết luận chỉ được chấp nhận khi truy ngược được tới artifact và mốc mã nguồn tương ứng

Hình 3.6. Chuỗi bằng chứng từ mã nguồn đến kết luận

Các chỉ số kiểm soát cuối của bước B13 được tổng hợp trong Bảng 3.7.

Bảng 3.7. Kết quả kiểm soát cuối B13

Kiểm soát	Kết quả
Run benchmark	9 trên 9 đạt
Chứng thư phát hành	1.830
Transaction neo	9
Full verifier	1.830 trên 1.830 VALID
Payload manifest	35 trên 35 khớp
Tệp SHA-256	37 trên 37 khớp

Quy trình chỉ công bố artifact sau khi loại credential và dữ liệu runtime không cần thiết. Kết quả phản ánh nguyên mẫu Bitcoin regtest trên máy 4 vCPU và 19 GiB RAM; không được dùng để khẳng định hiệu năng mainnet hoặc trạng thái sẵn sàng production.

3.9. Kết quả Kịch bản 1 – tính toàn vẹn và Maker–Checker

Kịch bản bắt đầu từ chứng thư gốc hợp lệ, lần lượt sửa tên người nhận, tên văn bằng, proofValue, transaction id và thử để Maker tự duyệt. Sáu trên sáu ca đạt.

Kết quả kiểm tra tính toàn vẹn và cơ chế Maker–Checker được tổng hợp trong Bảng 3.8.

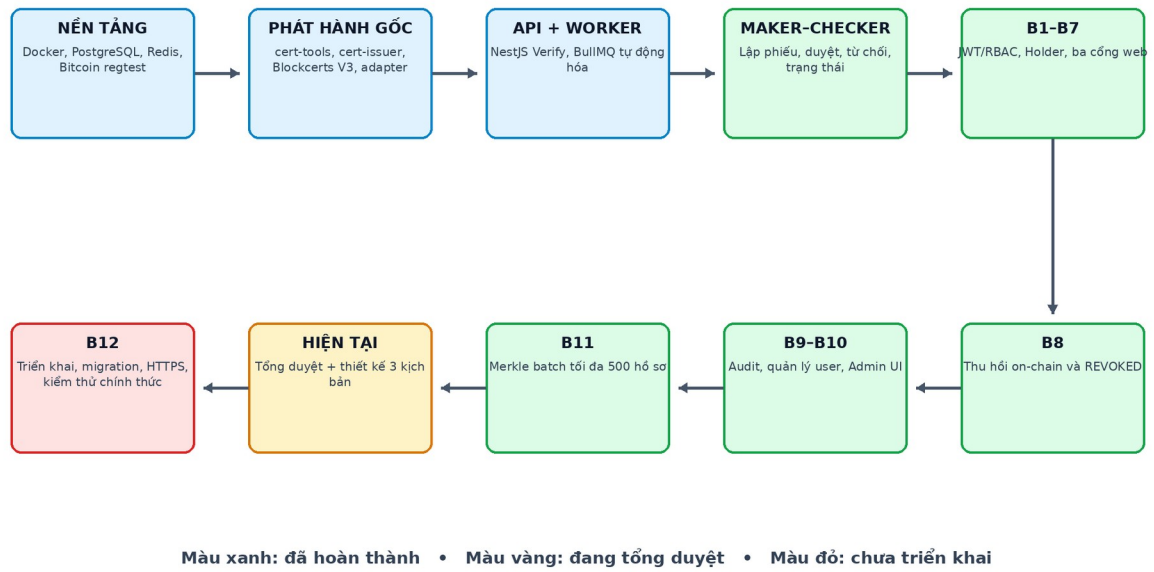
Bảng 3.8. Kết quả Kịch bản 1

Mã	Cá kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S1-01	Chứng thư gốc	VALID	VALID	Đạt
S1-02	Sửa tên người nhận	Không VALID	INVALID	Đạt
S1-03	Sửa tên văn bản	Không VALID	INVALID	Đạt
S1-04	Sửa proofValue	Không VALID	INVALID	Đạt
S1-05	Sửa txid trong proof	Không VALID	INVALID	Đạt
S1-06	Maker tự duyệt	HTTP 403	HTTP 403	Đạt

Bốn trường hợp sửa nội dung, proof hoặc txid đều trả INVALID; lỗi hạ tầng/RPC trong bộ test riêng mới trả INDETERMINATE. Cá S1-06 cho thấy guard Maker–Checker đã chặn đúng hành vi tự duyệt trong phạm vi API và cấu hình được kiểm thử, không chỉ ở giao diện.

Hình 3.7 khái quát trình tự chuẩn bị, thực thi và kiểm toán thực nghiệm.

TIẾN TRÌNH XÂY DỰNG CHƯƠNG 3



Hình 3.7. Tiến trình thực nghiệm Chương 3

3.10. Kết quả Kịch bản 2 – hiệu năng Merkle batching

3.10.1. Kết quả từng lần

Kết quả của chín lần chạy hiệu năng tái kiểm chứng được trình bày trong Bảng 3.9.

Bảng 3.9. Chín lần chạy hiệu năng tái kiểm chứng

Kích thước	Lần	Duyệt → issued (giây)	Giây/ chứng thư	Chứng thư/giây	Thành công	Tx
10	1	6,106	0,6106	1,6377	10/10	1
10	2	6,102	0,6102	1,6388	10/10	1
10	3	6,094	0,6094	1,6410	10/10	1
100	1	10,379	0,1038	9,6348	100/100	1
100	2	10,368	0,1037	9,6451	100/100	1
100	3	10,548	0,1055	9,4805	100/100	1
500	1	35,943	0,0719	13,9109	500/500	1
500	2	36,112	0,0722	13,8458	500/500	1

Kích thước	Lần	Duyệt → issued (giây)	Giây/ chứng thư	Chứng thư/giây	Thành công	Tx
500	3	35,798	0,0716	13,9673	500/500	1

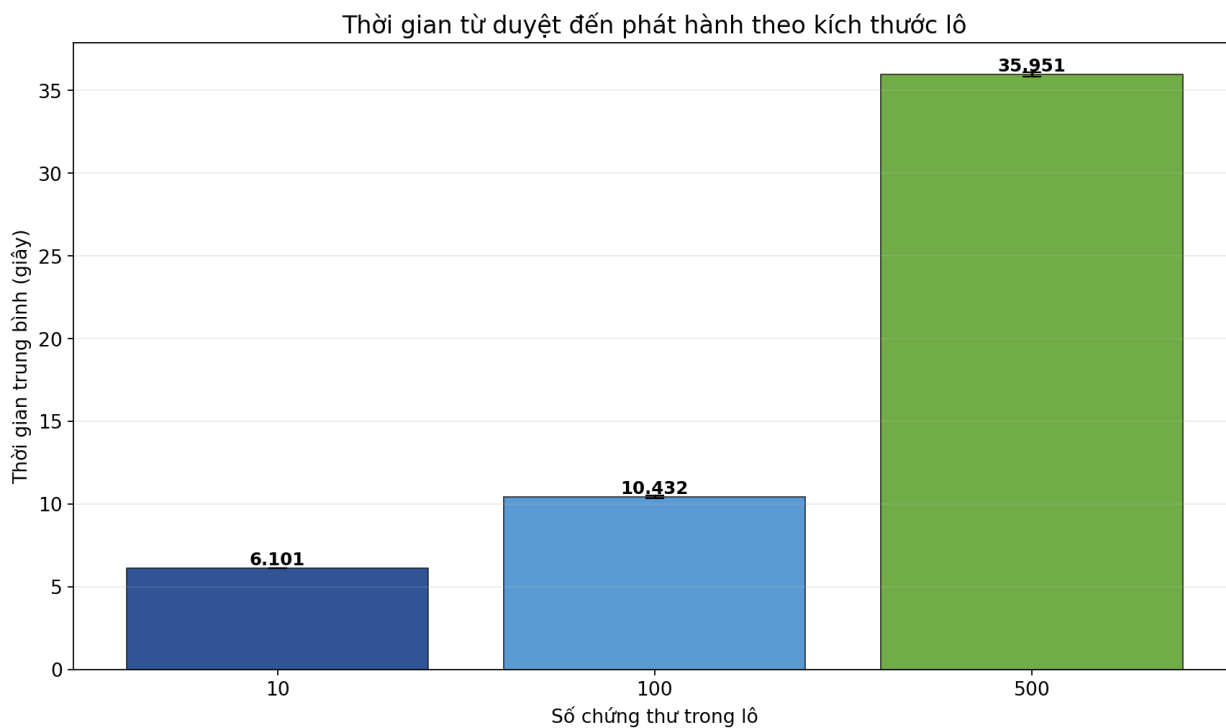
3.10.2. Giá trị tổng hợp

Các giá trị tổng hợp theo kích thước lô được trình bày trong Bảng 3.10.

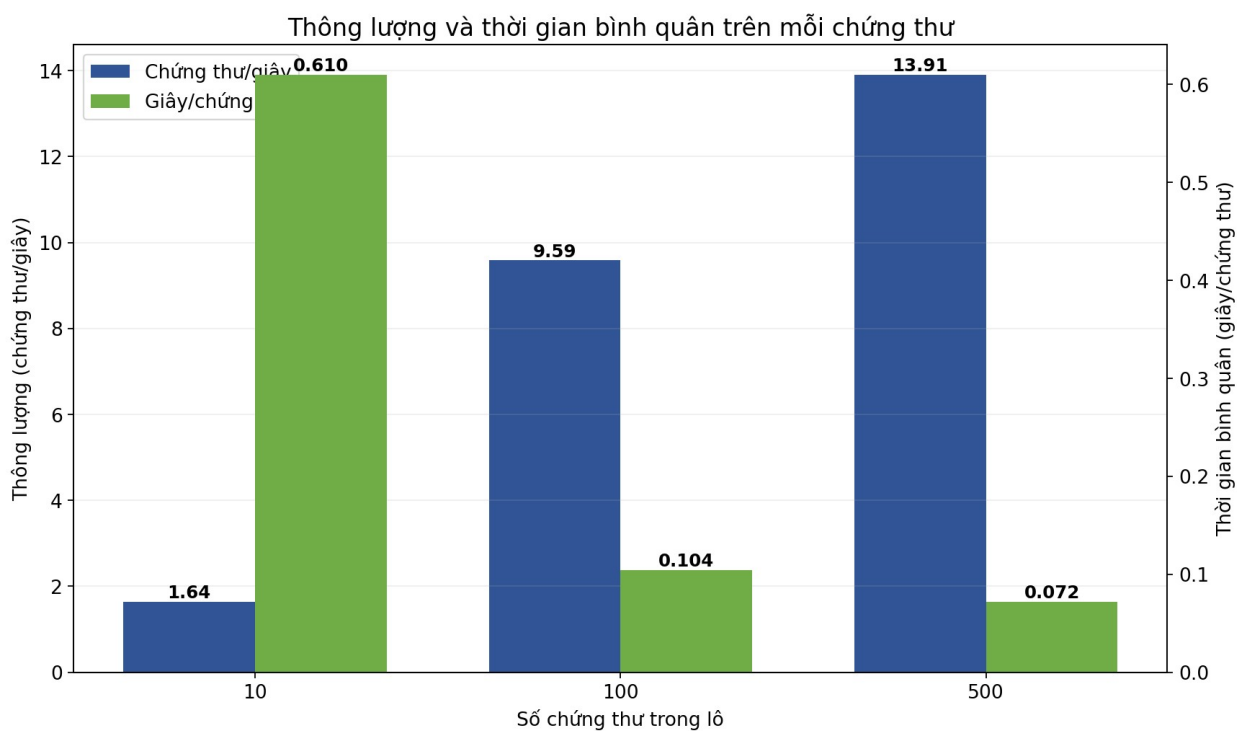
Bảng 3.10. Tổng hợp theo kích thước batch

Batch	Số lần	Tổng chứng thư	Thành công	Thời gian TB $\pm$ SD (giây)	Giây/ chứng thư TB	Thông lượng TB	Tx/batch
10	3	30	100%	6,101 $\pm$ 0,005	0,6101	1,6392	1,00
100	3	300	100%	10,432 $\pm$ 0,082	0,1043	9,5868	1,00
500	3	1.500	100%	35,951 $\pm$ 0,128	0,0719	13,9080	1,00

Khi batch tăng từ 10 lên 500, kích thước tăng 50 lần nhưng thời gian trung bình tăng khoảng 5,89 lần. Throughput tăng khoảng 8,48 lần; thời gian trung bình mỗi chứng thư giảm khoảng 88,21%. Kết quả phản ánh lợi ích phân bổ chi phí cố định và batching trong môi trường thử nghiệm, không phải cam kết hiệu năng production.



Hình 3.8. Thời gian phát hành trung bình theo kích thước batch



Hình 3.9. Thông lượng và thời gian trung bình trên mỗi chứng thư

3.10.3. Tài nguyên

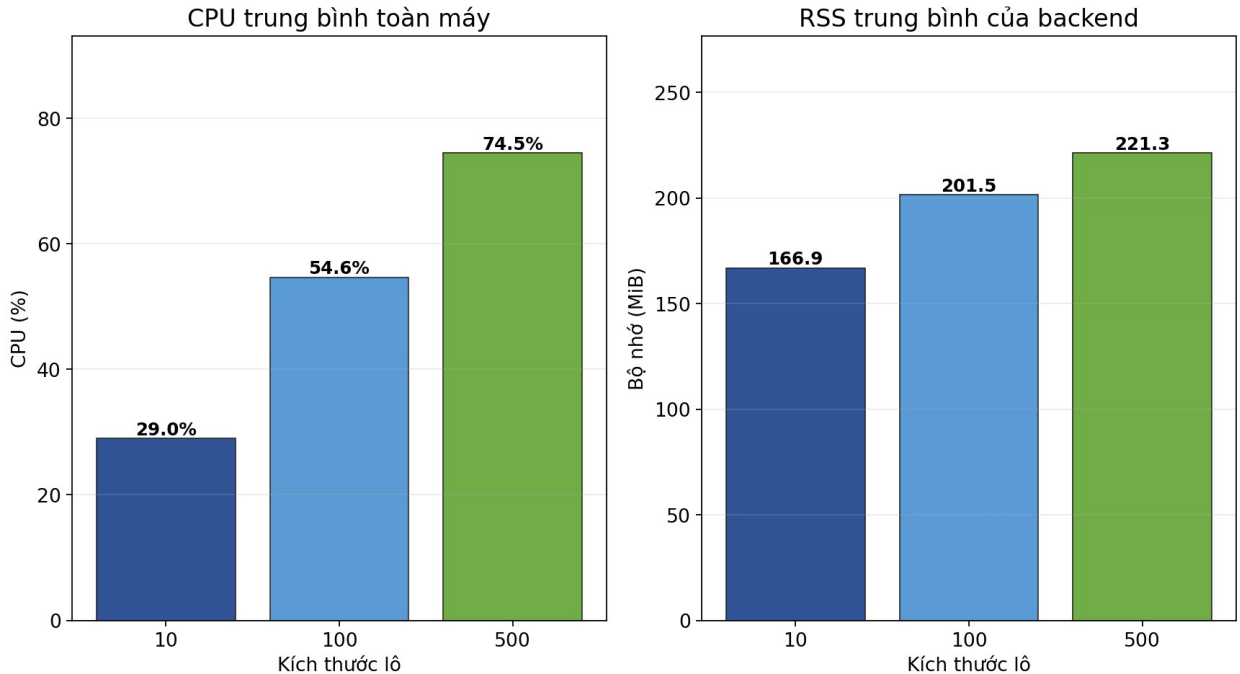
Mức sử dụng tài nguyên theo kích thước lô được tổng hợp trong Bảng 3.11.

Bảng 3.11. Tài nguyên hệ thống

Batch	CPU TB	CPU cực đại TB	Bộ nhớ dùng TB (MB)	Bộ nhớ cực đại TB (MB)	RSS backend TB (MB)	RSS backend cực đại (MB)
10	29,00%	42,71%	2.508,9	2.543,7	166,9	169,4
100	54,62%	100,00%	2.705,6	3.042,8	201,5	205,3
500	74,45%	100,00%	2.912,7	3.323,4	221,3	235,6

CPU và bộ nhớ tăng theo kích thước batch nhưng nằm trong giới hạn máy thử nghiệm. CPU cực đại chạm 100% ở nhóm 100 và 500; bộ nhớ dùng cực đại trung bình của nhóm 500 khoảng 3.323,4 MB. Số liệu chỉ đại diện cấu hình 4 vCPU và concurrency 16 đã nêu.

Mức sử dụng tài nguyên theo kích thước lô



Hình 3.10. Mức sử dụng tài nguyên theo kích thước batch

3.10.4. Kiểm toán artifact

Script kiểm toán chạy lại trên dữ liệu vừa sinh và ghi nhận 9/9 run đạt, 1.830/1.830 chứng thư được phát hành, 1.830 certificate ID cùng target hash duy

nhất, 1.830/1.830 Merkle proof khớp, 9 lô phát hành tương ứng 9 transaction đã xác nhận và 1.830/1.830 chứng thư được bộ cert-verifier-js kết luận VALID. Kiểm toán còn đối chiếu checksum từng file với manifest, dữ liệu DB theo issuanceBatchId và Merkle root trong OP_RETURN. MANIFEST.json đối chiếu đúng 35/35 payload của lượt chạy; SHA256SUMS kiểm tra đúng 37/37 tệp công bố (gồm manifest và bảng tổng hợp bổ sung) được lưu trong repository.

Kết quả kiểm toán artifact được tổng hợp trong Bảng 3.12.

Bảng 3.12. Kết quả kiểm toán

Chỉ số	Kết quả
Lần chạy	9/9 đạt
Chứng thư phát hành	1.830/1.830
Certificate ID/target hash duy nhất	1.830/1.830
Merkle proof được đối chiếu	1.830/1.830
Xác minh đầy đủ bằng cert-verifier-js	1.830/1.830 VALID
Lô phát hành/transaction đã xác nhận	9/9
Payload trong manifest lượt chạy	35/35
Tệp công bố qua SHA256SUMS	37/37

Tổng 1.830 là tổng lượt/chứng thư của chín run: $3 \times 10 + 3 \times 100 + 3 \times 500$. Không có việc cộng 1.830 chứng thư rồi nhân thêm ba lần; từng run sinh tập ID riêng. Kết quả áp dụng cho cây nguồn và stack regtest được ghi trong artifact, không phải tuyên bố cho mainnet hoặc production. Lượt chạy được thực hiện từ commit nguồn 2816de8aac13492b614dbcc6b3b3e1f3eba52bc1, trạng thái Git sạch (sourceDirty=false), với fingerprint cây nguồn 6e0cd862daf5641b26995575d8e7ff9d755daf43396191dfc838fc6ffb0997cc; manifest bao phủ cả benchmark.mjs, run-tests.sh, audit-benchmark-artifacts.mjs, full-verifier-batch.cjs và source-manifest.py.

3.11. Kết quả Kịch bản 3 – thu hồi, RBAC và audit

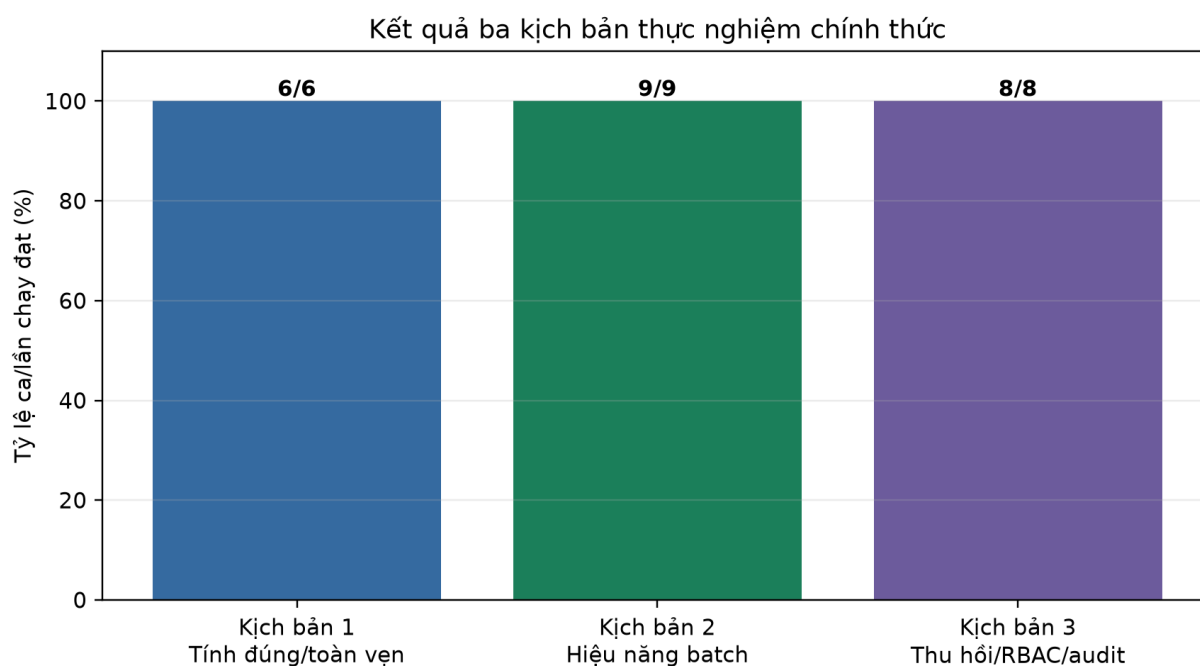
Kết quả kiểm tra thu hồi, phân quyền và audit được tổng hợp trong Bảng 3.13.

Bảng 3.13. Kết quả Kịch bản 3

Mã	Ca kiểm thử	Kỳ vọng	Quan sát	Đánh giá
S3-01	Trước thu hồi	VALID	VALID	Đạt
S3-02	Student thu hồi	HTTP 403	HTTP 403	Đạt
S3-03	Maker thu hồi	HTTP 403	HTTP 403	Đạt
S3-04	Checker thu hồi	HTTP 201	HTTP 201	Đạt
S3-05	Sau thu hồi	REVOKED	REVOKED	Đạt
S3-06	Tx thu hồi xác nhận	≥ 1	1	Đạt
S3-07	Có trong Revocation List	Có	Có	Đạt
S3-08	Audit đủ role/action	4 action	revoke/issue/ approve/request	Đạt

Tám trên tám ca đạt. Trong phạm vi môi trường tái kiểm chứng và các ca đã thiết kế, kết quả ghi nhận chỉ Checker thực hiện được thao tác thu hồi; transaction có xác nhận; Verify, Revocation List và audit trả kết quả nhất quán.

Hình 3.11 tổng hợp tỷ lệ đạt của ba nhóm kịch bản kiểm thử đã thực hiện.



Hình 3.11. Tổng hợp tỷ lệ đạt của ba kịch bản kiểm thử

3.12. Đánh giá kết quả

3.12.1. Mức đáp ứng

Mức đáp ứng yêu cầu và bằng chứng tương ứng được đối chiếu trong Bảng 3.14.

Bảng 3.14. Đối chiếu yêu cầu và bằng chứng

Yêu cầu	Bằng chứng	Kết quả
Maker không tự duyệt	S1-06	Đạt
Phát hiện sửa chứng thư/bằng chứng	S1-02 đến S1-05	Đạt
Phát hành 10/100/500	9 run, 1.830 chứng thư	Đạt
Một transaction mỗi batch	9 batch/9 transaction	Đạt
Chỉ Checker thu hồi	S3-02 đến S3-04	Đạt
Verify nhận biết thu hồi	S3-05 đến S3-07	Đạt
Audit actor/action	S3-08	Đạt
Build/test/health	49/49 backend test; 16/16 verifier test; 3 frontend; health đạt với 10 migration	Đạt

3.12.2. Hạn chế

Thứ nhất, regtest không phản ánh phí, độ trễ và kinh tế an ninh của mainnet. Thứ hai, thử nghiệm chạy trên một VPS và ba lần lặp mỗi kích thước; chưa đại diện tải dài hạn hoặc hệ thống phân tán nhiều node. Thứ ba, khóa chưa ở HSM/KMS, chưa có MFA, HTTPS công khai, secret manager và quy trình xoay khóa. Thứ tư, API và worker vẫn cùng tiến trình; worker có quyền Docker gần tương đương quyền quản trị host.

Thứ năm, Revocation List dựa vào DB; chưa có indexer phục hồi toàn bộ từ blockchain. Thứ sáu, audit hash chain phát hiện sửa cục bộ nhưng DBA vẫn có thể viết lại chuỗi hoặc xóa đuôi nếu head hash không được neo ra kho độc lập. Thứ bảy, issuance/revocation đã có checkpoint, idempotency và reconciliation nhưng chưa có fault-injection bằng cách kill worker đúng thời điểm hoặc kiểm thử tải/concurrency quy mô lớn. Thứ tám, SSRF regression test đã bao phủ loopback, private/link-local/metadata IP, DNS kết quả trộn, redirect, timeout/giới hạn dung

lượng; chưa thay thế pentest độc lập. Thứ chín, khả năng tương tác với verifier Blockcerts bên ngoài regtest cần tiếp tục đánh giá.

3.12.3. Hướng phát triển

Hệ thống đã dùng studentCode/khóa ngoại ổn định, transactional outbox và checkpoint/idempotency; bước tiếp theo là tách bảng request/batch/certificate/revocation, bổ sung fault-injection và indexer phục hồi thu hồi, đưa khóa vào HSM/KMS, tách API khỏi worker đặc quyền, thêm MFA, HTTPS/HSTS, backup/restore, giám sát và kiểm thử xâm nhập. Verifier cần phân giải DNS có kiểm soát, từ chối toàn bộ địa chỉ private/link-local/loopback/metadata sau mỗi lần phân giải và sau chuyển hướng, đồng thời chỉ cho phép hostname/scheme/port đã định trước theo khuyến nghị OWASP [23]. Trước mainnet cần kiểm thử testnet, xác định phí, confirmation policy, quyền riêng tư và trách nhiệm pháp lý. Có thể bổ sung verifier độc lập phía người dùng để giảm phụ thuộc dịch vụ trung tâm.

Tài liệu hỗ trợ được tổ chức theo vai trò sử dụng. Danh mục API và quyền truy cập nằm tại Phụ lục 1. Khung cấu hình triển khai nằm tại Phụ lục 2. Quy trình vận hành và xử lý lỗi nằm tại Phụ lục 3. Bộ bằng chứng thực nghiệm được mô tả tại Phụ lục 4. Ma trận truy vết yêu cầu nằm tại Phụ lục 5. Nguồn của hình và biểu đồ nằm tại Phụ lục 6.

3.13. Kết luận Chương 3

Chương 3 đã trình bày môi trường, cách hiện thực backend, ba frontend, worker, Blockcerts toolchain, xác minh, thu hồi và hardening B12; đồng thời báo cáo kết quả thực nghiệm có artifact kiểm toán. Kịch bản 1 đạt 6/6, Kịch bản 2 có 9/9 run, 1.830/1.830 proof được đối chiếu và 1.830/1.830 chứng thư được xác minh VALID, Kịch bản 3 đạt 8/8. Kết quả cung cấp bằng chứng thực nghiệm rằng nguyên mẫu đáp ứng các yêu cầu đã kiểm thử và batching hoạt động trong môi trường regtest; kết quả không xác nhận tính đúng toàn diện và không được suy rộng thành mức sẵn sàng production.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết quả của đề án

Đề án đã nghiên cứu cơ sở blockchain, hàm băm, chữ ký số, cây Merkle, cơ chế đồng thuận và mô hình Verifiable Credentials; từ đó phân tích, thiết kế và xây dựng nguyên mẫu quản lý văn bằng số dựa trên Blockcerts V3 và Bitcoin. Hệ thống có ba vai trò xác thực Maker–Checker–Student, một nhóm người xác minh công khai, ba cổng web, phát hành theo lô, holder portal, thu hồi và audit.

Về kỹ thuật, hệ thống chuẩn hóa và băm từng chứng thư, tạo cây Merkle, neo một root cho cả lô lên Bitcoin regtest và gắn proof riêng vào từng credential. Bộ xác minh tính lại payload hash, proof, anchor, confirmation và trạng thái thu hồi. Thực nghiệm chính thức hoàn thành 6/6 phép thử nghiệp vụ, 8/8 phép thử xác minh–thu hồi, chín lần benchmark không lỗi, kiểm toán 1.830/1.830 proof và xác minh đầy đủ 1.830/1.830 chứng thư là VALID.

Kết quả cho thấy mô hình phù hợp để minh họa cách blockchain tăng khả năng phát hiện sửa đổi và giảm phụ thuộc vào tra cứu thủ công. Tuy nhiên, blockchain không tự chứng minh dữ liệu đầu vào là đúng và không thay thế quản trị của cơ sở đào tạo. Giá trị của chứng thư vẫn phụ thuộc vào danh tính issuer, quy trình phê duyệt, bảo vệ khóa và khả năng vận hành dịch vụ.

2. Đóng góp chính

Đóng góp của đề án gồm: (1) mô hình hóa quy trình phát hành văn bằng có phân tách Maker–Checker; (2) hiện thực Blockcerts V3 theo lô với Merkle proof và Bitcoin anchor; (3) xây ba cổng giao diện phù hợp ba nhóm người dùng; (4) kết hợp kiểm tra on-chain với trạng thái nghiệp vụ, thu hồi và audit; (5) cung cấp migration, scripts và artifacts để tái lập, kiểm toán kết quả.

3. Kiến nghị

Trước khi triển khai thực tế, cần đánh giá pháp lý và quy trình nghiệp vụ của cơ sở đào tạo; quản lý khóa bằng HSM/KMS; bổ sung chữ ký tổ chức; triển khai indexer và phục hồi dữ liệu thu hồi; tăng độ phủ kiểm thử; kiểm thử bảo mật độc lập; xây quy trình backup, giám sát, luân chuyển khóa và ứng cứu sự cố. Việc chuyển sang testnet/mainnet chỉ nên thực hiện sau khi các điều kiện này được đáp ứng và chi phí, quyền riêng tư cùng chính sách lưu trữ đã được phê duyệt.

TÀI LIỆU THAM KHẢO

- [1] D. Yaga, P. Mell, N. Roby, K. Scarfone, “Blockchain Technology Overview,” NISTIR 8202, National Institute of Standards and Technology, 2018. DOI: 10.6028/NIST.IR.8202.
- [2] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008. <https://bitcoin.org/bitcoin.pdf> (truy cập ngày 13/09/2026).
- [3] R. C. Merkle, “Secrecy, Authentication, and Public Key Systems,” doctoral dissertation, Stanford University, 1979.
- [4] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.
- [5] National Institute of Standards and Technology, “Digital Signature Standard (DSS),” FIPS PUB 186-5, 2023. DOI: 10.6028/NIST.FIPS.186-5.
- [6] M. Castro, B. Liskov, “Practical Byzantine Fault Tolerance,” Proceedings of the Third Symposium on Operating Systems Design and Implementation, 1999, pp. 173–186. <https://www.usenix.org/conference/osdi-99/practical-byzantine-fault-tolerance>.
- [7] D. Ongaro, J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” 2014 USENIX Annual Technical Conference, 2014, pp. 305–319. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>.
- [8] E. Androulaki và cộng sự, “Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains,” Proceedings of the Thirteenth EuroSys Conference, 2018. DOI: 10.1145/3190508.3190538.
- [9] M. Sporny, D. Longley, D. Chadwick (biên tập), “Verifiable Credentials Data Model v2.0,” W3C Recommendation, 15/05/2025. <https://www.w3.org/TR/vc-data-model-2.0/>.
- [10] T. T. A. Dinh, R. Liu, M. Zhang, G. Chen, B. C. Ooi, J. Wang, “Untangling Blockchain: A Data Processing View of Blockchain Systems,” IEEE Transactions on Knowledge and Data Engineering, vol. 30, no. 7, pp. 1366–1385, 2018. DOI: 10.1109/TKDE.2017.2781227.
- [11] Blockcerts, “cert-schema: JSON schemas and context for Blockcerts,” commit 1e390ac, 31/07/2025. <https://github.com/blockchain-certificates/cert-schema> (truy cập ngày 13/09/2026).

[12] D. Longley, G. Kellogg, M. Sporny, M. Lanthaler, P.-A. Champin, N. Lindström (biên tập), “JSON-LD 1.1,” W3C Recommendation, 16/07/2020. <https://www.w3.org/TR/json-ld11/>.

[13] M. Sporny, D. Longley (biên tập), “Verifiable Credential Data Integrity 1.0,” W3C Recommendation, 15/05/2025. <https://www.w3.org/TR/vc-data-integrity/>.

[14] Blockcerts, “jsonld-signatures-merkleproof2019: MerkleProof2019 suite,” commit 26a9c30, 25/08/2026. <https://github.com/blockchain-certificates/jsonld-signatures-merkleproof2019> (truy cập ngày 13/09/2026).

[15] M. Jones, J. Bradley, N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, 2015. <https://www.rfc-editor.org/rfc/rfc7519>.

[16] OWASP Foundation, “Authorization Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (truy cập ngày 13/09/2026).

[17] OWASP Foundation, “Password Storage Cheat Sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (truy cập ngày 13/09/2026).

[18] Bitcoin Project, “Bitcoin Developer Guide — Testing Applications.” <https://developer.bitcoin.org/examples/testing.html> (truy cập ngày 13/09/2026).

[19] NestJS, “NestJS Documentation.” <https://docs.nestjs.com/> (truy cập ngày 13/09/2026).

[20] PostgreSQL Global Development Group, “PostgreSQL 17 Documentation — Transactions.” <https://www.postgresql.org/docs/17/tutorial-transactions.html> (truy cập ngày 13/09/2026).

[21] BullMQ, “BullMQ Documentation.” <https://docs.bullmq.io/> (truy cập ngày 13/09/2026).

[22] TypeORM, “TypeORM Documentation — Migrations and Transactions.” <https://typeorm.io/docs/advanced-topics/migrations/>; <https://typeorm.io/docs/advanced-topics/transactions/> (truy cập ngày 13/09/2026).

[23] OWASP Foundation, “Server Side Request Forgery Prevention Cheat Sheet,” OWASP Cheat Sheet Series (truy cập ngày 13/09/2026).

PHỤ LỤC

Phụ lục 1. Danh mục API và quyền truy cập

Phụ lục này tóm tắt bề mặt API dùng trong nguyên mẫu. Quyền được kiểm tra ở backend bằng JWT và RolesGuard. Điều kiện hiển thị trên frontend chỉ hỗ trợ trải nghiệm, không phải ranh giới an toàn.

Nhóm API xác thực và tài khoản được trình bày trong Bảng PL1.1.

Bảng PL1.1. API xác thực và tài khoản

Phương thức	Đường dẫn	Quyền	Mục đích
POST	/api/auth/login	Công khai	Đổi thông tin đăng nhập hợp lệ lấy JWT
GET	/api/auth/me	Đã đăng nhập	Lấy hồ sơ của subject hiện tại
POST	/api/auth/register	Checker	Tạo tài khoản theo quy trình quản trị
GET	/api/admin/users	Checker	Liệt kê tài khoản phục vụ quản trị
PATCH	/api/admin/users/:id	Checker	Cập nhật trạng thái hoặc hồ sơ cho phép

Nhóm API nghiệp vụ phát hành được trình bày trong Bảng PL1.2.

Bảng PL1.2. API phát hành, Holder và audit

Phương thức	Đường dẫn đại diện	Quyền	Mục đích
POST	/api/issue/requests	Maker	Tạo nhóm yêu cầu phát hành
GET	/api/issue/requests	Maker, Checker	Tra cứu phiếu theo phạm vi vai trò
POST	/api/issue/approve	Checker	Duyệt một tập phiếu đang chờ

Phương thức	Đường dẫn đại diện	Quyền	Mục đích
POST	/api/issue/reject	Checker	Từ chối phiếu bằng cập nhật có điều kiện
GET	/api/issue/batches/:id	Maker, Checker	Theo dõi lô phát hành
GET	/api/issue/holder/certificates	Student	Lấy chứng thư của subject hiện tại
GET	/api/audit	Maker, Checker	Tra cứu log theo quyền và bộ lọc

Nhóm API công khai và thu hồi được trình bày trong Bảng PL1.3.

Bảng PL1.3. API xác minh, thu hồi và tài nguyên công khai

Phương thức	Đường dẫn đại diện	Quyền	Mục đích
POST	/api/verify	Công khai	Xác minh chứng thư được tải lên
POST	/api/revoke/:id	Checker	Khởi tạo quy trình thu hồi
GET	/api/blockcerts/issuers/kma/profile.json	Công khai	Cung cấp Issuer Profile
GET	/api/blockcerts/issuers/kma/revocation-list.json	Công khai	Cung cấp danh sách thu hồi
GET	/health	Công khai	Kiểm tra sức khỏe tối thiểu

Tên route chi tiết có thể thay đổi khi tái cấu trúc; repository tại commit nguồn là tham chiếu có thẩm quyền. Bảng không thay thế tài liệu Swagger được sinh từ decorator trong mã nguồn.

Phụ lục 2. Cấu hình triển khai tái lập

2.1. Nguyên tắc cấu hình

Cấu hình được chia thành giá trị công khai, giá trị nội bộ và bí mật. URL public của Issuer Profile và revocation list phải thống nhất với nội dung chứng thư. Hostname giữa container dùng tên service Compose, còn trình duyệt dùng địa chỉ do Nginx công bố. Không sử dụng localhost trong chứng thư nếu verifier chạy ở môi trường khác.

Các mức nhạy cảm của cấu hình được phân loại trong Bảng PL2.1.

Bảng PL2.1. Phân loại cấu hình

Nhóm	Phạm vi	Được đưa vào Git	Được đưa vào báo cáo
Tên service và cổng mẫu	Kiến trúc	Có	Có
URL issuer công khai	Chứng thư	Có thể	Có
Mật khẩu PostgreSQL và Redis	Runtime	Không	Không
JWT secret	Runtime	Không	Không
Bitcoin RPC password	Runtime	Không	Không
WIF phát hành	Worker	Không	Không
Khóa công khai issuer	Public profile	Có	Có

2.2. Khung biến môi trường

Đoạn dưới chỉ thể hiện tên biến. Giá trị thật phải được sinh riêng và không sao chép từ tài liệu.

```
NODE_ENV=production
PORT=4000
DATABASE_URL=[REDACTED]
REDIS_URL=[REDACTED]
JWT_ACCESS_SECRET=[REDACTED]
JWT_REFRESH_SECRET=[REDACTED]
BITCOIN_RPC_URL=[REDACTED]
BITCOIN_RPC_USER=[REDACTED]
```

```

BITCOIN_RPC_PASSWORD=[REDACTED]
ISSUER_WIF=[REDACTED]
PUBLIC_BASE_URL=http://127.0.0.1:18088

```

Tập runtime được cấp quyền đọc hạn chế. Trước commit cần kiểm tra git status --short, vùng staging và mẫu chuỗi nhảy cảm. Nếu credential từng xuất hiện trong Git, chỉ xóa khỏi phiên bản hiện tại là chưa đủ; giá trị phải bị thu hồi hoặc thay mới.

2.3. Các service triển khai

Vai trò và dữ liệu bền vững của từng service được trình bày trong Bảng PL2.2.

Bảng PL2.2. Vai trò service

Service	Vai trò	Dữ liệu bền vững
PostgreSQL	Trạng thái nghiệp vụ, người dùng, chứng thư và audit	Volume cơ sở dữ liệu
Redis	Hàng đợi BullMQ 6 và trạng thái job ngắn hạn	Có thể tái tạo từ DB và checkpoint
Bitcoin Core	Blockchain regtest và ví thử nghiệm	Volume chain, wallet
Backend	API xác thực, phát hành, xác minh và thu hồi	Không lưu cục bộ lâu dài
Worker	Điều phối cert-tools, cert-issuer và checkpoint	Workspace của từng job
Nginx	Điểm vào của bộ tái lập	Không có dữ liệu nghiệp vụ
Ba frontend	Admin, Client và Verify	Bundle tĩnh

Phụ lục 3. Quy trình vận hành và xử lý lỗi

3.1. Trình tự chuẩn

Kiểm tra công cụ và Docker daemon.

Checkout đúng commit và xác nhận cây nguồn sạch.

Tạo cấu hình runtime bằng credential mới.

Chạy experiments/repro/scripts/prepare.sh.

Chạy `experiments/repro/scripts/start.sh`.
Chờ `experiments/repro/scripts/health.sh` đạt.
Chạy bộ test chức năng và phục hồi.
Chạy benchmark khi cần tái tạo số liệu hiệu năng.
Kiểm toán artifact bằng hai script kiểm tra.
Đóng băng evidence và tạo SHA-256.
Dừng stack bằng `experiments/repro/scripts/stop.sh`.

3.2. Backend không kết nối PostgreSQL

Kiểm tra trạng thái container, health và log trước khi thay cấu hình.

```
docker compose logs --tail=100 postgres
```

Ba nguyên nhân thường gặp là password runtime không đồng nhất, volume cũ được tạo bởi cấu hình khác và backend khởi động trước khi PostgreSQL sẵn sàng. Việc xóa volume chỉ thực hiện với stack thử nghiệm và sau khi xác định không cần giữ dữ liệu.

3.3. Redis hoặc BullMQ không hoạt động

Backend có thể nhận request nhưng batch đứng ở queued khi worker không kết nối Redis hoặc không chạy. Cần đối chiếu REDIS_URL, password, tên queue và log worker. Redis chỉ điều phối job; không sửa trạng thái cơ sở dữ liệu bằng tay để bỏ qua worker.

3.4. Bitcoin RPC không phản hồi

Kiểm tra container Bitcoin Core, chain đang là regtest, ví đã được nạp và RPC credential giữa client với server khớp. Thiếu coin thử nghiệm làm transaction không thể được tài trợ. Coin regtest được tạo bằng mining trong môi trường cô lập và không có giá trị ngoài thử nghiệm.

3.5. Migration chưa đủ

Schema phải có đủ mười migration. Dấu hiệu thiếu migration cuối gồm không có cột tách định danh hoặc lỗi khóa khi duyệt từng phần. Cách xử lý đúng là chạy migration từ mã nguồn cùng commit, không sửa thủ công bảng để vượt qua test.

3.6. Phát hành dừng trước broadcast

Khi chưa có transaction ID, worker có thể xây dựng lại payload và transaction từ checkpoint phù hợp. Các file trung gian phải gắn với

issuanceBatchId để không trộn dữ liệu giữa job. Trạng thái DB phản ánh lỗi có thể thử lại nhưng không tuyên bố chứng thư đã neo.

3.7. *Phát hành dừng sau broadcast*

Khi checkpoint đã có transaction ID, lần chạy lại phải tra cứu và sử dụng giao dịch này. Tạo giao dịch mới sẽ làm sai quan hệ một lô–một anchor và gây khó kiểm toán. Recovery E2E kiểm tra chính xác thuộc tính giữ nguyên transaction ID và chỉ tăng một block.

3.8. *Verifier trả INDETERMINATE*

Trạng thái này yêu cầu kiểm tra phụ thuộc tạm thời như issuer URL, revocation URL, Bitcoin RPC, DNS, redirect và timeout. Giao diện giữ nguyên nhãn “Chưa thể kết luận”. Chỉ khi có bằng chứng xác định payload, proof hoặc anchor sai mới chuyển sang INVALID.

3.9. *Checksum không khớp*

Một tệp bị thay đổi sau thời điểm đóng băng evidence phải được xem là không còn thuộc bộ bằng chứng đã xác nhận. Cần xác định nguồn thay đổi, chạy lại bước kiểm toán tương ứng và tạo manifest mới. Không chỉnh trực tiếp tệp checksum để làm cho kết quả khớp.

Phụ lục 4. Bộ bằng chứng thực nghiệm

Bộ tái kiểm chứng chính thức nằm tại experiments/results/final-revalidation-20260930/. Artifact cũ trong experiments/results/ được giữ để truy vết lịch sử nhưng không được dùng thay cho kết quả cuối.

Thành phần và vai trò của bộ evidence được liệt kê trong Bảng PL4.1.

Bảng PL4.1. Thành phần evidence chính

Thành phần	Nội dung	Vai trò
Functional E2E	Kết quả luồng đăng nhập, Maker–Checker, Holder và duyệt từng phần	Chứng minh tích hợp nghiệp vụ
Recovery E2E	Transaction ID và số block trước, sau phục hồi	Chứng minh không broadcast lặp
Backend Jest	12 suite, 49 test	Chứng minh hành vi đơn vị

Thành phần	Nội dung	Vai trò
Verifier tests	16 test	Chứng minh SSRF và phân loại trạng thái
Benchmark runs	Chín run cho batch 10, 100 và 500	Cơ sở số liệu hiệu năng
Audit benchmark	Kiểm tra ID, hash, proof, batch và transaction	Chứng minh cấu trúc artifact
Full verifier	1.830 chứng thư VALID	Kiểm tra độc lập bằng verifier
Manifest	Mô tả nguồn, cấu hình và payload	Truy vết lượt chạy
SHA-256	37 trên 37 tệp khớp	Phát hiện thay đổi evidence

Các số liệu đã được đối chiếu và được phép sử dụng được chốt trong Bảng PL4.2.

Bảng PL4.2. Số liệu chốt được phép sử dụng

Chỉ số	Giá trị
Backend Jest	12 trên 12 suite; 49 trên 49 test
Verifier	16 trên 16 test
Functional E2E	Đạt
Recovery E2E	Đạt
Benchmark	9 trên 9 run
Tổng chứng thư	1.830
Full verifier	1.830 trên 1.830 VALID
Manifest payload	35 trên 35 khớp
SHA-256 evidence	37 trên 37 khớp
Thời gian batch 10	6,101 ± 0,005 giây
Thời gian batch 100	10,432 ± 0,082 giây

Chỉ số	Giá trị
Thời gian batch 500	35,951 ± 0,128 giây

Số liệu trên thuộc nguyên mẫu Bitcoin regtest trên máy 4 vCPU và 19 GiB RAM. Không có phép đo mainnet, tải Internet diện rộng, HSM, high availability hoặc kiểm thử xâm nhập độc lập trong phạm vi evidence này.

Phụ lục 5. Ma trận truy vết yêu cầu và bằng chứng

Tình trạng đáp ứng yêu cầu chức năng được truy vết trong Bảng PL5.1.

Bảng PL5.1. Truy vết yêu cầu chức năng

Mã	Yêu cầu	Thành phần	Bằng chứng	Kết quả
FR-01	Đăng nhập và xác định vai trò	Auth module, JWT strategy	Backend Jest, E2E	Đạt
FR-02	Checker tạo tài khoản	Register có JWT và RolesGuard	E2E tài khoản	Đạt
FR-03	Maker tạo nhóm phiếu	Issuance controller và service	Functional E2E	Đạt
FR-04	Checker duyệt hoặc từ chối	Conditional update	Jest và E2E	Đạt
FR-05	Duyệt từng phần	Hai loại batch ID	E2E hai lần duyệt	Đạt
FR-06	Phát hành theo lô	Worker và Blockcerts tools	Benchmark, artifact audit	Đạt trong regtest
FR-07	Student xem chứng thư của mình	Holder endpoint	Functional E2E	Đạt
FR-08	Xác minh công khai	Verify service và portal	Verifier tests, full verifier	Đạt
FR-09	Thu hồi	Revocation service	Jest và E2E	Đạt

Mã	Yêu cầu	Thành phần	Bằng chứng	Kết quả
FR-10	Audit thao tác	Audit module	DB và functional E2E	Đạt
FR-11	Phục hồi sau lỗi	Worker checkpoint	Recovery E2E	Đạt

Tình trạng đáp ứng yêu cầu an toàn và phi chức năng được truy vết trong Bảng PL5.2.

Bảng PL5.2. Truy vết yêu cầu an toàn và phi chức năng

Mã	Yêu cầu	Kiểm soát	Bằng chứng	Phạm vi kết luận
NFR-01	Phân quyền phía server	JWT, RBAC và guard	Unit test, E2E	Vai trò trong nguyên mẫu
NFR-02	Chống SSRF	Kiểm tra URL, IP và redirect	10 ca verifier	Các biến thể đã kiểm thử
NFR-03	Toàn vẹn chứng thư	Hash, Merkle proof và anchor	1.830 artifact	Bộ benchmark cuối
NFR-04	Không phát lập sau lỗi	Checkpoint sau broadcast	Recovery E2E	Kịch bản lỗi được mô phỏng
NFR-05	Khả năng tái lập	Commit, manifest và checksum	Source manifest, SHA-256	Môi trường đã công bố
NFR-06	Hiệu năng theo batch	Benchmark ba kích thước	Chín run	Máy 4 vCPU, 19 GiB RAM
NFR-07	Bảo vệ bí mật	Tập runtime ngoài Git, redaction	Quét repository và artifact	Không thay thế secret manager production

Phụ lục 6. Danh mục hình và tệp nguồn

Các hình lý thuyết, kiến trúc, quy trình và biểu đồ hiệu năng nằm trong report-assets/. Hình benchmark được sinh từ dữ liệu CSV hoặc JSON của bộ evidence cuối. Hình kiến trúc được dựng lại theo module, endpoint và luồng đang

tồn tại trong repository. Logo bìa được trích từ mẫu CDCS của nhà trường và chỉ dùng cho tài liệu này.

Nguồn của từng nhóm tài sản hình ảnh được tổng hợp trong Bảng PL6.1.

Bảng PL6.1. Nhóm tài sản hình ảnh

Nhóm	Tiền tố tệp	Nguồn
Lý thuyết Blockchain	ch1-	Mô hình hóa từ tài liệu tham khảo
Thiết kế hệ thống	ch2-	Kiến trúc và mã nguồn hiện hành
Triển khai và luồng	ch3-01 đến ch3-05	Quy trình hiện thực
Biểu đồ benchmark	ch3-06 đến ch3-09	Evidence cuối ngày 30 tháng 9 năm 2026
Kiểm soát tái lập	ch3-10, ch3-11	Script và manifest hiện hành

Không có vị trí ảnh trống trong bản xuất. Nếu một ảnh không thể dựng hoặc không có nguồn kiểm chứng, nội dung tương ứng được trình bày bằng bảng và văn bản thay vì chèn khung ảnh không có bằng chứng.